

Tomasz Przemysław Chojnacki

Numer albumu: 094004

**Projekt i realizacja szkoleniowej gry symulacyjnej w
wirtualnej rzeczywistości dla szkolenia początkujących
strażaków**

**Praca dyplomowa
na studiach I-go stopnia
na kierunku Informatyka**

Promotor pracy dyplomowej:

dr inż. Ludomir Tuszyński

Katedra Informatyki Stosowanej

POLITECHNIKA ŚWIĘTOKRZYSKA
WYDZIAŁ ELEKTROTECHNIKI, AUTOMATYKI I INFORMATYKI

Studia stacjonarne
Kierunek: INFORMATYKA

Zatwierdzam:

PRODZIEKAN
ds. Studenckich i Dydaktyki
Wydziału Elektrotechniki, Automatyki i Informatyki

.....
Prodziekan ds. studenckich i dydaktyki

Rok akademicki: 2025/26

ZADANIE NA PRACĘ DYPLOMOWĄ
STUDIÓW PIERWSZEGO STOPNIA

Wydano studentowi:

Chojnacki Tomasz

nr albumu: 94004

I. Temat pracy:

Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków.

Design and implementation of a virtual reality training simulation game for training novice firefighters.

II. Cel pracy:

Celem pracy jest stworzenie gry symulacyjnej w wirtualnej rzeczywistości (VR). Gra będzie pełnić funkcję szkoleniową dla początkujących strażaków.

III. Plan pracy (zakres pracy):

1. Omówienie problematyki wykorzystania wirtualnej rzeczywistości (VR) w grach komputerowych.
2. Opracowanie projektu gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków.
3. Realizacja i testowanie gry.
4. Uwagi i wnioski.

IV. Uwagi dotyczące pracy: brak uwag.

V. Termin oddania pracy: zgodnie z Regulaminem Studiów.

VI. Konsultant: praca nie wymaga konsultanta.

Opiekun merytoryczny

Dr hab. inż. Paweł Sitek, prof. PŚk

Katedry Informatyki Stosowanej,
Wydziału Elektrotechniki, Automatyki i Informatyki

.....
dr hab. inż. Paweł Sitek, prof. PŚk
(podpis)

Promotor pracy dyplomowej

Dr inż. Ludomir Tuszyński

.....
(podpis)

Temat pracy dyplomowej celem jej wykonania otrzymałem(am):

Kielce, dnia 05.06.2025 r.

.....
Tomasz Chojnacki
czytelny podpis studenta

Kielce, dnia 23.01.2026r.

Tomasz Przemysław Chojnacki, 094004
Imię i nazwisko studenta, nr albumu
ul. Wesoła 13, Starachowice 29-200
Adres zamieszkania
Studia I stopnia, stacjonarne
Studia pierwszego/drugiego* stopnia, forma studiów stacjonarne/nie-stacjonarne*
Informatyka, Grafika, Komputerowa
Kierunek, zakres
dr inż. Ludomir Tuszyński
Promotor pracy dyplomowej

OŚWIADCZENIE

Przedkładając w roku akademickim 2025/26 promotorowi pracy dyplomowej studiów pierwszego/drugiego* stopnia, powołanemu przez Dziekana Wydziału Elektrotechniki, Automatyki i Informatyki Politechniki Świętokrzyskiej, pracę dyplomową pod tytułem: Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków oświadczam, że:

- 1) przedstawiona praca dyplomowa została opracowana przeze mnie samodzielnie, stosownie do wskazówek merytorycznych opiekuna pracy,
- 2) przy wykonywaniu pracy dyplomowej wykorzystano materiały źródłowe, w granicach dozwolonego użytku wymieniając autora, tytuł pozycji i miejsce jej publikacji,
- 3) praca dyplomowa nie zawiera żadnych danych, informacji i materiałów, których publikacja nie jest prawnie dozwolona,
- 4) przedstawiona praca nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego/stopnia naukowego w wyższej uczelni,
- 5) niniejsza wersja pracy jest identyczna z treścią elektroniczną w systemie Archiwum Prac Dyplomowych.

Przyjmuję do wiadomości, że w przypadku ujawnienia w mojej pracy dyplomowej, stanowiącej podstawę nadania tytułu zawodowego, przypisania sobie przeze mnie autorstwa istotnego fragmentu lub innych elementów cudzego utworu lub ustalenia naukowego, rektor, w drodze decyzji administracyjnej, stwierdzi nieważność dyplomu.

Zostałem uprzedzony:

- 1) o odpowiedzialności karnej wynikającej z art. 115 ust. 1 i 2 ustawy z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (t. j. Dz. U. z 2022 r. poz. 2509 ze zm.): „Kto przywłaszcza sobie autorstwo albo wprowadza w błąd co do autorstwa całości lub części cudzego utworu albo artystycznego wykonania, podlega grzywnie, karze ograniczenia wolności albo pozbawienia wolności do lat 3. Tej samej karze podlega, kto rozpowszechnia bez podania nazwiska lub pseudonimu twórcy cudzy utwór w wersji oryginalnej albo w postaci opracowania, artystyczne wykonanie albo publicznie zniekształca taki utwór, artystyczne wykonanie, fonogram, wideogram lub nadanie.”,
- 2) o odpowiedzialności dyscyplinarnej na podstawie art. 307 ust. 1 ustawy z dnia 20 lipca 2018 r. Prawo o szkolnictwie wyższym i nauce (t. j. Dz. U. z 2022 r. poz. 574, ze zm.): „Student podlega odpowiedzialności dyscyplinarnej za naruszenie przepisów obowiązujących w uczelni oraz za czyn uchybiający godności studenta.”

Tomasz Chojnacki
Czytelny podpis studenta

*niepotrzebne skreślić

Kielce, dnia 23.01.2026r.

Tomasz Przemysław Chojnacki 094004
Imię i nazwisko studenta, nr albumu
ul. Klesza 18 Starachowice 27-200
Adres zamieszkania
Studia I stopnia, stacjonarne
Studia pierwszego/ drugiego stopnia, forma studiów stacjonarne/nie-stacjonarne
Informatyka, Grafika Komputerowa
Kierunek, zakres
dr. inż. Ludomir Tuszyński
Promotor pracy dyplomowej

OŚWIADCZENIE AUTORA PRACY

Zgodnie z ustawą z dnia 4 lutego 1994r. o prawie autorskim i prawach pokrewnych (t.j. Dz. U. 2022 poz. 2509), wyrażam zgodę na udostępnianie mojej pracy dyplomowej dla celów naukowych i dydaktycznych.

Tomasz Chojnacki
Czytelny podpis studenta

Projekt i realizacja szkoleniowej gry symulacyjnej wirtualnej rzeczywistości dla szkolenia początkujących strażaków

Streszczenie

Celem niniejszej pracy było zaprojektowanie oraz wykonanie szkoleniowej gry symulacyjnej wykorzystującej technologię wirtualnej rzeczywistości, przeznaczonej do wspomagania procesu szkolenia początkujących strażaków. Opracowane rozwiązanie umożliwia użytkownikowi udział w zasymulowanej rzeczywistości szkoleniowej odwzorowującej wybraną sytuację operacyjną, z zachowaniem zasad bezpieczeństwa oraz możliwością wielokrotnego powtarzania ćwiczenia. Zastosowanie środowiska wirtualnego pozwala na symulację zdarzeń niebezpiecznych do przeprowadzenia w warunkach rzeczywistych, a także na wstępne przygotowanie potencjalnego użytkownika gry symulacyjnej do zadań objętych ryzykiem w pracy strażaka. System został zaprojektowany jako narzędzie szkoleniowe umożliwiające stopniowe nabywanie umiejętności oraz ocenę poprawności podejmowanych decyzji w trakcie symulacji.

Słowa kluczowe: gra symulacyjna, wirtualna rzeczywistość, szkolenie początkujących strażaków, unity, C#

Design and implementation of a virtual reality training simulation game for training novice firefighters

Summary

The aim of this thesis was to design and implement a training simulation game using virtual reality technology, intended to support the training process of beginner firefighters.

The developed solution enables the user to participate in a simulated training reality reflecting a selected operational situation, while maintaining safety principles and allowing repeated execution of the exercise. The use of a virtual environment makes it possible to simulate events that are dangerous to conduct in real conditions, as well as to provide initial preparation of a potential user of the simulation game for tasks involving risk in the work of a firefighter. The system was designed as a training tool enabling the gradual acquisition of skills and the evaluation of the correctness of decisions made during the simulation.

Keywords: simulation game, virtual reality, training of beginner firefighters, Unity, C#

SPIS TREŚCI

SPIS TREŚCI	11
1. WSTĘP	13
2. CEL I ZAKRES PRACY	14
3. WPROWADZENIE DO TECHNOLOGII VR I NARZĘDZI	16
3.1. Podstawy działania rzeczywistości wirtualnej	16
3.2. Możliwości oraz ograniczenia Meta Quest 2 w projektach szkoleniowych	19
3.3. Unity jako narzędzie do tworzenia gier i symulacji	20
3.4. Omówienie wykorzystanych narzędzi	21
4. PROJEKT SYMULATORA STRAŻACKIEGO VR	21
4.1. Założenia projektowe – czego ma nauczyć symulator	22
4.2. Opis scenariusza pożaru w hali dystrybucyjnej.....	23
4.3. Scenariusze wybranych przypadków użycia systemu	25
4.4. Interakcje użytkownika	30
5. IMPLEMENTACJA APLIKACJI VR	31
5.1. Konfiguracja Unity pod Meta Quest 2	31
5.2. Implementacja sterowania VR	34
5.2.1. Implementacja chodzenia	35
5.2.2. Implementacja obracania	37
5.2.3. Implementacja teleportacji	39
5.2.4. Implementacja chwytania	42
5.3. Modele 3D w symulacji	44
5.4. Główny interfejs użytkownika VR	54
5.4.1. Interakcja z menu w VR	55
5.4.2. Uruchamianie symulacji i ekran ładowania	56
5.4.3. Okno ustawień	56
5.4.4. Okno informacji	57
5.4.5. Zamykanie aplikacji	58
5.5. System autopompy	59
5.5.1. Implementacja zachowania węża strażackiego.....	62
5.5.2. System cząstek wody z zaworu końcowego	64
5.6. System powietrza	68
5.6.1. Panel stanu zdrowia i powietrza	69
5.6.2. Aparaty powietrzne	71
5.7. System prądu.....	73
5.8. System ognia	74
5.9. Sceny końcowe	77
5.10. Optymalizacja i dostosowanie pod wydajność gogli VR.....	79
6. TESTOWANIE SYMULATORA	82
6.1. Testowanie funkcjonalne	82

6.2. Opis odczuć użytkowników po testach symulatora.....	84
6.3. Możliwości rozwoju.....	88
7. PODSUMOWANIE.....	88
LITERATURA	90
SPIS TABEL	91
SPIS RYSUNKÓW	91

1. WSTĘP

W ostatnich latach technologie cyfrowe w coraz większym stopniu zmieniają podejście do edukacji i szkoleń praktycznych. W szczególności wirtualna rzeczywistość (VR) jako narzędzie immersyjne zdobywa uznanie nie tylko w rozrywce, lecz także w dydaktyce, szkoleniach zawodowych i symulacjach sytuacji kryzysowych. VR umożliwia tworzenie trójwymiarowych środowisk, w których uczestnik może wchodzić w interakcje z elementami symulacji, doświadczać realistycznych scenariuszy oraz uczyć się poprzez działanie. Wszystko to bez narażania się na faktyczne ryzyko związane z realnymi sytuacjami. W literaturze [7, 9] wirtualna rzeczywistość jest opisywana jako technologia, która dzięki efektowi „zanurzenia” pozwala uczestnikowi odczuwać obecność w wirtualnym świecie i reagować tak, jak w warunkach rzeczywistych, co znacznie zwiększa efektywność procesu uczenia się.

Rozwój VR znajduje swoje odzwierciedlenie także w badaniach i publikacjach dotyczących edukacji oraz szkoleń specjalistycznych [4]. Analizując informacje związane z wykorzystaniem symulatorów VR można dostrzec, że tego rodzaju technologie dają możliwość efektywnego kształcenia praktycznych umiejętności. Jest to szczególnie ważne w obszarach, gdzie tradycyjne metody nauczania napotykają ograniczenia logistyczne lub bezpieczeństwa uczestników. W literaturze wskazuje się również na wyzwania związane z integracją VR z istniejącymi programami szkoleniowymi, wymagające zmian metodycznych i organizacyjnych w instytucjach edukacyjnych oraz szkoleniowych.

Jednym z elementów, w których potencjał VR jest szczególnie dostrzegany, są szkolenia ratownicze, szkolenia służb bezpieczeństwa oraz szkolenia strażaków. Straż Pożarna jako jeden z oddziałów zawodowych działa w środowisku o wysokim stopniu ryzyka. Pożary, wybuchy, katastrofy budowlane, czy zagrożenia chemiczne to scenariusze, które ciężko odwzorować w ramach tradycyjnych ćwiczeń bez kosztów logistycznych i potencjalnego zagrożenia dla uczestników. W artykule badawczym „VR w straży pożarnej – korzyści, wyzwania i najlepsze praktyki wdrożenia szkoleń wirtualnych” [9] który poświęcony jest wykorzystaniu VR w szkoleniu strażaków wskazuje się, że technologia ta może stać się wartościowym narzędziem wspierającym doskonalenie umiejętności, zwłaszcza w sytuacjach, które w realnym świecie byłyby niebezpieczne, kosztowne lub trudne do odtworzenia w warunkach szkoleniowych.

Szkolenia VR w kontekście straży pożarnej mają kilka kluczowych zalet. Po pierwsze, umożliwiają bezpieczne wielokrotne przećwiczenie procedur, które w rzeczywistości niosą ze sobą ryzyko urazów lub uszkodzenia sprzętu. Po drugie, poprzez realistyczne scenariusze VR pozwala na trenowanie reakcji emocjonalnej i decyzyjnej, co

jest szczególnie istotne w zawodach, gdzie działania pod presją czasu mogą decydować o życiu i zdrowiu. Po trzecie, VR może być stosowana jako etap pośredni pomiędzy szkoleniem teoretycznym a ćwiczeniami praktycznymi w rzeczywistych warunkach, co sprzyja ugruntowaniu właściwych nawyków i procedur bez pierwszego kontaktu z realnym zagrożeniem.

W Polsce obserwujemy pierwsze przykłady wykorzystania VR w szkoleniach strażaków, zarówno w jednostkach zawodowych, jak i ochotniczych, gdzie gogle VR są wykorzystywane do ćwiczenia scenariuszy operacyjnych, takich jak działania gaśnicze czy ewakuacyjne. Przykłady takie pokazują, że technologia ta może przyczynić się do zwiększenia jakości przygotowania zawodowego oraz do budowania kompetencji, które w tradycyjnym systemie szkolenia byłyby trudne do osiągnięcia. Niemniej jednak rozwiązania tego typu nie są jeszcze powszechnie stosowane i często stanowią jedynie uzupełnienie standardowych metod szkoleniowych, co wynika zarówno z kosztów implementacji, jak i konieczności adaptacji treści dydaktycznych do specyfiki VR.

Poza wyzwaniem szkoleniowymi związanymi z treścią, VR jako technologia stawia również wyzwania technologiczne i sprzętowe. Urządzenia, takie jak Meta Quest 2, ze względu na brak konieczności podłączania do komputera zewnętrznego, stają się coraz częściej wybieranym sprzętem do realizacji symulacji VR w środowisku szkoleniowym. Jednak ograniczenia wydajności, pola widzenia czy też poruszania po świecie VR wpływają na sposób projektowania aplikacji. Konieczna jest optymalizacja treści w taki sposób, aby użytkownik nie doświadczał negatywnych skutków symulacji, takich jak nudności, czy też zawrotów głowy wynikających z zaburzenia błędnika. Problemy te są przedmiotem badań i publikacji dotyczących adaptacji technologii VR do zastosowań praktycznych w edukacji i szkoleniach specjalistycznych.

2. CEL I ZAKRES PRACY

Celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz realizacja szkoleniowej gry symulacyjnej w technologii wirtualnej rzeczywistości, przeznaczonej do wspomagania procesu szkolenia początkujących strażaków, która umożliwia naukę i doskonalenie podstawowych umiejętności niezbędnych podczas działań gaśniczych hali dystrybucyjnej. Opracowana aplikacja ma stanowić narzędzie wspierające proces szkolenia, umożliwiając użytkownikowi udział w realistycznym scenariuszu pożarowym w bezpiecznym, kontrolowanym środowisku wirtualnym.

Celem symulatora jest przede wszystkim umożliwienie użytkownikowi praktycznego zapoznania się z wybranymi elementami pracy strażaka, takimi jak obsługa

sprzętu gaśniczego - autopompy oraz poruszanie się w przestrzeni objętej zagrożeniem wynikającym z zatrucia oparami gazów pożarowych, reagowanie na zmieniające się warunki otoczenia oraz podejmowanie decyzji pod presją czasu poprzez ograniczony dostęp powietrza. Gra symulacyjna została zaprojektowana w taki sposób, aby użytkownik mógł wielokrotnie powtarzać ćwiczenia, analizować popełniane błędy oraz stopniowo doskonalić swoje umiejętności bez ryzyka konsekwencji, jakie mogłyby wystąpić w realnych warunkach. Istotnym założeniem projektu było również uwzględnienie czynników wpływających na bezpieczeństwo strażaka, takich jak ograniczona ilość powietrza, obecność stref zagrożenia czy ryzyko wynikające z niewłaściwej kolejności wykonywania czynności. Dzięki temu symulator nie tylko uczy obsługi sprzętu, lecz także kształtuje świadomość zagrożeń oraz odpowiedzialnego podejmowania decyzji w sytuacjach kryzysowych.

Praca koncentruje się na praktycznym wykorzystaniu technologii VR jako narzędzia umożliwiającego bezpieczne i realistyczne odwzorowanie scenariuszy działań gaśniczych, które w rzeczywistych warunkach wiązałyby się z wysokim poziomem ryzyka, kosztami organizacyjnymi lub ograniczeniami logistycznymi.

Zakres pracy obejmuje zarówno część teoretyczną, jak i praktyczną. W części teoretycznej przedstawione są podstawy działania technologii wirtualnej rzeczywistości, charakterystyka zastosowanego sprzętu VR oraz omówione są narzędzia programistyczne wykorzystane do realizacji projektu. Przedstawię również możliwości oraz ograniczenia wynikające z zastosowania autonomicznych gogli VR, które mają bezpośredni wpływ na projektowanie interakcji, realizm symulacji oraz optymalizację aplikacji.

Część praktyczna pracy skupia się na procesie projektowania i implementacji szkoleniowej gry symulacyjnej. Obejmuje ona opracowanie scenariusza szkoleniowego, zaprojektowanie interakcji użytkownika w środowisku VR, implementację kluczowych systemów symulacyjnych oraz dostosowanie aplikacji do wymagań wydajnościowych platformy sprzętowej. W ramach pracy uwzględniono również aspekty związane z ergonomią użytkownika, komfortem przebywania w środowisku wirtualnym oraz stabilnością działania aplikacji.

Zakres projektu został celowo ograniczony do jednego, wybranego scenariusza szkoleniowego, który pozwala skupić się na jakości i realizmie symulacji, zamiast na liczbie dostępnych sytuacji. Takie podejście umożliwiło dokładniejsze odwzorowanie procedur, interakcji ze sprzętem oraz zagrożeń środowiskowych, z jakimi może spotkać się początkujący strażak podczas działań operacyjnych w hali dystrybucyjnej.

3. WPROWADZENIE DO TECHNOLOGII VR I NARZĘDZI

Technologia wirtualnej rzeczywistości jest dobrym, współczesnym rozwiązaniem do tworzenia symulacji. Jej głównym założeniem jest stworzenie immersyjnego środowiska cyfrowego, w którym użytkownik ma poczucie fizycznej obecności oraz możliwość naturalnej interakcji z wirtualnym otoczeniem. W przeciwieństwie do klasycznych symulatorów komputerowych, VR angażuje nie tylko wzrok, lecz również ruchy głowy, rąk oraz całego ciała, co znacząco wpływa na sposób odbioru treści i przyswajania wiedzy. Podstawą działania systemów VR są gogle wyposażone w zestaw wyświetlaczy, czujników oraz kamer, które wspólnie odpowiadają za śledzenie ruchów użytkownika i ich odwzorowanie w środowisku wirtualnym. W projekcie zastosowałem autonomiczne gogle Meta Quest 2, które łączą się z komputerem za pomocą okablowania i aplikacji „Meta Horizon Link”. Tego typu rozwiązanie wpływa zarówno na płynność symulacji jak i dobre odczucia użytkownika, w szczególności w kontekście wydajności, optymalizacji oraz realizmu symulacji.

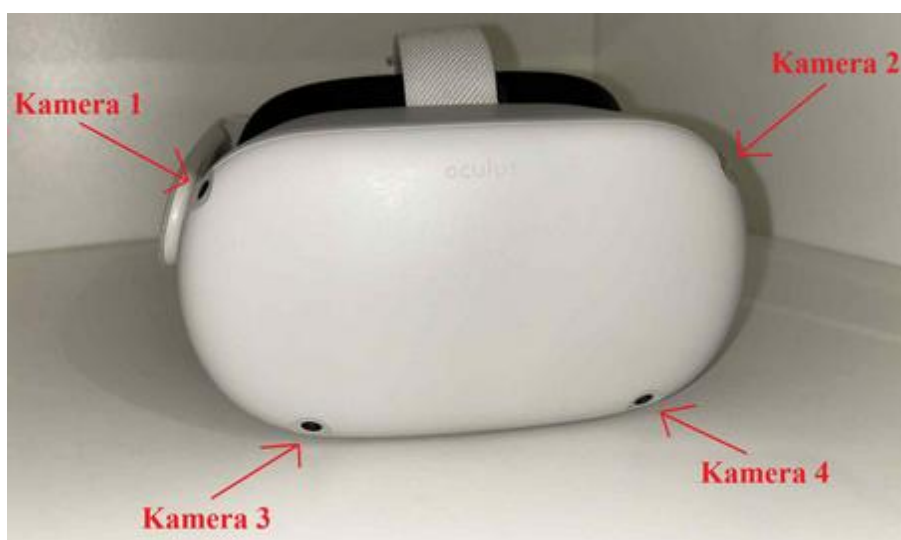
3.1. Podstawy działania rzeczywistości wirtualnej

Rzeczywistość wirtualna opiera się na połączeniu obrazu stereoskopowego, śledzenia ruchu użytkownika oraz systemów interakcji, które wspólnie tworzą wrażenie przebywania w wirtualnym świecie. Gogle VR wyświetlają dwa nieco różniące się obrazy osobny dla każdego oka.



Rysunek 3.1. Soczewki gogli Meta Quest 2 [Źródło własne].

Dzięki temu możliwe jest uzyskanie efektu głębi oraz realistycznego postrzegania przestrzeni trójwymiarowej. Obrazy te są dynamicznie aktualizowane w czasie rzeczywistym w zależności od ruchów głowy użytkownika, co pozwala zachować spójność pomiędzy ruchem a wyświetlaną sceną. Gogle wyposażone są również w zestaw czujników inercyjnych takich jak: akcelerometry, żyroskopy, które odpowiadają za precyzyjne śledzenie orientacji głowy w przestrzeni. Dzięki nim system rejestruje ruchy obrotowe, takie jak pochylanie, skręcanie czy obracanie głowy i natychmiast odwzorowuje je w środowisku wirtualnym. Oprócz śledzenia obrotu, istotnym elementem jest również śledzenie położenia użytkownika w przestrzeni. Meta Quest 2 realizuje śledzenie pozycyjne w oparciu o system tzw. „inside-out tracking”. Oznacza to, że kamery odpowiedzialne za analizę otoczenia znajdują się bezpośrednio w goglach.



Rysunek 3.2. Kamery gogli Meta Quest 2 [Źródło własne].

Takie rozwiązanie eliminuje zapotrzebowanie na dawniej używane zewnętrzne stacje bazowe. Gogle wyposażone są w cztery szerokokątne kamery umieszczone na obudowie, które nieustannie obserwują otoczenie użytkownika. Na podstawie obrazu z kamer oraz danych z czujników inercyjnych system jest w stanie określić pozycję gogli w przestrzeni oraz wykrywać zmiany położenia względem otoczenia. Kamery gogli pełnią ważną rolę w śledzeniu kontrolerów ruchu. Kontrolery Meta Quest 2 wyposażone są w pierścienie z diodami podczerwieni, które nie są widoczne dla ludzkiego oka, lecz są wykrywane przez kamery gogli. Na podstawie położenia tych punktów świetlnych system oblicza dokładną pozycję oraz orientację kontrolerów w przestrzeni. Dzięki temu możliwe jest naturalne odwzorowanie ruchów rąk użytkownika w wirtualnym świecie, takich jak chwytanie obiektów, naciskanie przycisków, wskazywanie czy wykonywanie gestów.



Rysunek 3.3. Kontrolery gogli Meta Quest 2 [Źródło własne].

Dodatkowym elementem wpływającym na realizm VR jest możliwość rozpoznawania przestrzeni użytkownika, określanej jako Guardian. System ten pozwala na wyznaczenie bezpiecznego obszaru (granicy) poruszania się w rzeczywistym pomieszczeniu. Gdy użytkownik zbliża się do granicy wyznaczonej przestrzeni, gogle wyświetlają ostrzeżenie, graficzne o wyglądzie niebieskiej siatki, która w miejscu kolizji z granicą podświetla się na czerwono. Minimalizuje to ryzyko kolizji z obiektami rzeczywistymi. Informacje o granicach przestrzeni są wykorzystywane również przez aplikacje VR, które dostosowują sposób poruszania się użytkownika w świecie wirtualnym. Opóźnienia pomiędzy ruchem głowy a aktualizacją obrazu mogą prowadzić do dyskomfortu, a nawet objawów choroby symulatorowej. Z tego względu gogle Meta Quest 2 działają z częstotliwością 90 Hz odświeżania obrazu na każdą soczewkę, a aplikacje VR muszą być odpowiednio zoptymalizowane, aby zapewnić płynność animacji oraz stabilność wyświetlania. Ma to szczególne znaczenie w aplikacjach szkoleniowych, gdzie użytkownik spędza w środowisku VR dłuższy czas. Do spełnienia wymogów płynności gry szkoleniowej użyłem kabla o minimalnej przepustowości 5 Gb/s (Gigabitów na sekundę), co jest jednym z niezbędnych elementów do płynnej rozgrywki VR.

3.2. Możliwości oraz ograniczenia Meta Quest 2 w projektach szkoleniowych

Meta Quest 2 wyposażone są w procesor Qualcomm Snapdragon XR2 [7]. Układ ten łączy w sobie jednostkę centralną, układ graficzny oraz moduły przetwarzania obrazu. W połączeniu z pamięcią operacyjną oraz pamięcią wewnętrzną urządzenie jest w stanie samodzielnie przetwarzać sceny trójwymiarowe, obsługiwać wszystkie wewnętrzne systemy do prawidłowego działania w czasie rzeczywistym. Dzięki temu możliwe jest tworzenie w pełni funkcjonalnych aplikacji szkoleniowych.

Wykorzystanie gogli w trybie autonomicznym umożliwia realizację średniej wielkości scen oraz zamkniętych środowisk symulacyjnych, takich jak pomieszczenia, czy też ograniczone obszary zewnętrzne [5]. Przy odpowiedniej optymalizacji modeli, tekstur oraz efektów wizualnych możliwe jest osiągnięcie stabilnej płynności działania, co ma znaczenie dla wydajności. W praktyce oznacza to konieczność stosowania uproszczonej geometrii, ograniczonej liczby źródeł światła oraz świadomego wykorzystania systemów cząstek i shaderów. Jedną z głównych zalet trybu autonomicznego jest pełna swoboda ruchu użytkownika [4]. Brak kabla łączącego gogle z komputerem eliminuje ryzyko zaplątania się w przewód oraz pozwala na naturalne poruszanie się w wyznaczonej przestrzeni treningowej.

Oprócz wymienionych zalet gogli Meta Quest 2 istnieją również ograniczenia w ich wykorzystaniu. Najważniejszym z nich jest ograniczona moc obliczeniowa w porównaniu do zestawów VR podłączonych do wydajnego komputera PC [5]. W przypadku bardziej rozbudowanych projektów szkoleniowych o wyższym stopniu realistyczności tekstur, obejmujących duże mapy, złożone sceny otwarte, zaawansowane systemy oświetlenia czy rozbudowaną fizykę, zasoby sprzętowe gogli mogą okazać się niewystarczające. W takich sytuacjach najlepiej wybrać połączenie kablowe gogli z komputerem, co pozwala przenieść główne obciążenie obliczeniowe na zewnętrzną jednostkę i znacząco zwiększyć możliwości aplikacji.

Kolejnym elementem, który wpływa na możliwości jak i ograniczenia jest zasilanie urządzenia. Meta Quest 2 wyposażone są w wbudowany akumulator litowo jonowy [7], który umożliwia całkowicie bezprzewodową pracę przez ograniczony czas. Z jednej strony jest to duża zaleta, ponieważ zapewnia użytkownikowi swobodę i niezależność od źródeł zasilania. Z drugiej strony, pojemność akumulatora stanowi istotne ograniczenie w kontekście długotrwałych sesji szkoleniowych. Podczas korzystania z aplikacji VR, zwłaszcza symulacji o wysokim stopniu złożoności, gogle pobierają znaczną ilość energii, co prowadzi do stosunkowo szybkiego rozładowywania akumulatora. Jednakże podłączenie gogli do źródła zasilania za pomocą kabla podczas działania aplikacji VR nie powoduje pełnego ładowania akumulatora. W praktyce takie podłączenie jedynie spowalnia proces

jego rozładowywania, jednak nie eliminuje problemu ograniczonego czasu pracy. Z tego względu planowanie dłuższych sesji szkoleniowych wymaga uwzględnienia przerw na ładowanie urządzenia.

3.3. Unity jako narzędzie do tworzenia gier i symulacji

Unity jest jednym z najczęściej wykorzystywanych silników do tworzenia aplikacji w wirtualnej rzeczywistości [6, 8], szczególnie w projektach symulacyjnych. Wynika to przede wszystkim z dobrego wsparcia od strony silnika z platformami VR. Unity oferuje szerokie wsparcie dla urządzeń oraz elastyczność w zakresie projektowania. Jednym z głównych powodów wyboru przeze mnie Unity, jako narzędzia do realizacji projektu VR jest jego bardzo dobre wsparcie dla gogli Meta Quest 2. Silnik ten oferuje rozwiązania przeznaczone do pracy z architekturą ARM. W porównaniu do silnika Unreal Engine 5, Unity charakteryzuje się mniejszym narzutem wydajnościowym [6], co przekłada się na łatwiejsze osiągnięcie stabilnej płynności działania aplikacji na sprzęcie. Unreal Engine 5, mimo bardzo wysokiej jakości renderingu, jest silnikiem wymagającym znacznie większej mocy obliczeniowej, co w praktyce utrudnia jego wykorzystanie w aplikacjach VR. Kolejną istotną zaletą Unity jest dostępność rozbudowanego ekosystemu narzędzi dedykowanych VR.

Jednym z przyspieszających pracę rozwiązań jest XR Interaction Toolkit [8], który stanowi zestaw gotowych komponentów umożliwiających szybkie tworzenie interakcji charakterystycznych dla środowisk wirtualnych. Toolkit ten pozwala na implementację takich mechanizmów jak chwytanie obiektów, interakcje z elementami interfejsu użytkownika, obsługa kontrolerów ruchu oraz podstawowe systemy lokomocji. Dzięki temu możliwe jest skupienie się na logice symulacji i aspektach szkoleniowych projektu, zamiast na implementowaniu mechanizmów interakcji od podstaw. Unity oferuje również bardzo szeroki dostęp do gotowych zasobów [8] w postaci modeli 3D, materiałów, które dostępne są w oficjalnym sklepie Unity Asset Store. Dostępność dużej liczby assetów znacząco skraca czas realizacji projektu oraz pozwala na wykorzystanie sprawdzonych rozwiązań w obszarach takich jak efekty wizualne, interfejs użytkownika czy systemy cząstek. W projektach szkoleniowych możliwe jest wykorzystanie gotowych elementów, a następnie ich dostosowanie do specyfiki symulacji.

Istotną zaletą Unity jest zastosowanie języka C# [3], który cechuje się czytelną składnią oraz szerokim wsparciem dokumentacyjnym. Język ten umożliwia łatwe wprowadzanie nowych funkcjonalności, co jest istotne w projektach symulacyjnych, gdzie często zachodzi potrzeba dołożenia nowej logiki w trakcie testów.

3.4. Omówienie wykorzystanych narzędzi

Do utworzenia szkoleniowej gry symulacyjnej VR potrzebne były inne programy wspomagające proces tworzenia zasobów graficznych, animacji oraz elementów dźwiękowych aplikacji. Dobór odpowiednich narzędzi podyktowany był ich dostępnością i funkcjonalnością oraz możliwością bezproblemowej integracji z silnikiem Unity.

Pierwszym z wykorzystanych narzędzi był Blender [2], który umożliwia tworzenie modeli 3D, teksturowanie, rigging, animację oraz eksport zasobów do formatu fbx. W moim projekcie użyłem go do tworzenia oraz edycji modeli trójwymiarowych wykorzystywanych w symulacji. Obejmowało to przygotowanie wybranych elementów wyposażenia oraz edycji pojedynczych modeli. Zostały one dostosowane do wymagań aplikacji VR oraz zmodyfikowane pod indywidualne potrzeby symulacji poprzez odpowiednią optymalizację geometrii oraz poprawne przygotowanie siatek. Dodatkowo w Blenderze realizowałem animacje postaci strażaka [2], w tym animacje ruchu oraz pozycji uzupełniających tło symulacji.

Kolejnym narzędziem użytym w projekcie był Mixamo, czyli internetowa platforma umożliwiająca automatyczne riggowanie modeli postaci oraz generowanie animacji szkieletowych. W tym projekcie Mixamo wykorzystałem do pozyskania bazowych animacji. Następnie animacje te zostały zaimportowane do programu Blender, gdzie poddałem je dalszej edycji i dostosowaniu do potrzeb symulacji.

Do realizacji warstwy dźwiękowej symulacji wykorzystałem program Audacity [1], będący narzędziem do edycji i obróbki dźwięku. Umożliwia on nagrywanie, przycinanie, modyfikowanie oraz eksport plików audio w formatach kompatybilnych z silnikiem Unity. W ramach projektu użyłem go do przygotowania dźwięków wykorzystywanych w symulacji, takich jak odgłosy wody, ognia oraz sygnały ostrzegawcze. Dźwięki te zostały odpowiednio przycięte, oczyszczone oraz dostosowane pod względem głośności, aby mogły być poprawnie odtwarzane w środowisku VR.

4. PROJEKT SYMULATORA STRAŻACKIEGO VR

Projekt symulatora VR został opracowany z myślą o wsparciu procesu szkolenia początkujących strażaków poprzez wykorzystanie technologii wirtualnej rzeczywistości. Na etapie projektowania należało określić cele dydaktyczne symulatora oraz dostosować je do możliwości technologicznych sprzętu i narzędzi programistycznych. Projekt symulatora uwzględnia zarówno realizm odwzorowywanego scenariusza, jak i bezpieczeństwo

użytkownika oraz możliwość wielokrotnego powtarzania ćwiczenia w kontrolowanym środowisku.

Symulator został zaprojektowany jako aplikacja szkoleniowa, a nie gra rozrywkowa, co miało bezpośredni wpływ na przyjęte rozwiązania projektowe. Priorytetem nie była rozbudowana fabuła czy elementy rywalizacji, lecz poprawne odwzorowanie procedur, zachowań oraz interakcji charakterystycznych dla działań strażackich.

4.1. Założenia projektowe – czego ma nauczyć symulator

Głównym założeniem projektowym szkoleniowego symulatora strażackiego VR było stworzenie narzędzia umożliwiającego naukę podstawowych umiejętności niezbędnych podczas działań gaśniczych. Symulator ma za zadanie wprowadzić użytkownika w realia pracy strażaka poprzez praktyczne ćwiczenie procedur w środowisku wirtualnym, które odzwierciedla warunki panujące podczas akcji pożarowej.

Pierwszym celem dydaktycznym symulatora jest zapoznanie użytkownika z obsługą podstawowego sprzętu strażackiego. Projekt zakłada możliwość interakcji z elementami wyposażenia wykorzystywanymi podczas gaszenia pożaru, takimi jak wąż gaśniczy czy aparatura powietrzna. Użytkownik ma możliwość wykonywania czynności w odpowiedniej kolejności oraz obserwowania konsekwencji swoich działań w trakcie symulacji, co sprzyja utrwalaniu poprawnych nawyków.

Kolejnym założeniem projektu jest nauka poruszania się i orientacji w środowisku objętym zagrożeniem. Symulator umożliwia użytkownikowi funkcjonowanie w przestrzeni o ograniczonej dostępności powietrza, w obecności ognia oraz innych czynników utrudniających bezpieczne poruszanie się. Dzięki temu użytkownik może ćwiczyć zachowanie ostrożności, analizę otoczenia oraz podejmowanie decyzji w warunkach presji czasu.

Symulator ma również na celu rozwijanie umiejętności podejmowania decyzji oraz reagowania na warunki symulacji. Projekt zakłada, że użytkownik musi dostosowywać swoje działania do aktualnej sytuacji, w tym do poziomu zagrożenia, dostępnych zasobów oraz czasu pobytu w strefie zagrożenia. Zaprojektowany symulator umożliwia wielokrotne powtarzanie scenariusza szkoleniowego oraz naukę metodą prób i błędów. Brak realnych zagrożeń fizycznych pozwala użytkownikowi skupić się na nauce procedur i analizie własnych działań, co jest szczególnie istotne na początkowym etapie szkolenia. Takie podejście pozwala na kształtowanie świadomości konsekwencji błędnych decyzji.

4.2. Opis scenariusza pożaru w hali dystrybucyjnej

Środowisko symulacji w hali dystrybucyjnej zostało zaprojektowane w taki sposób, aby odwzorować realne warunki pracy w obiekcie przemysłowym, gdzie występuje zróżnicowane wyposażenie, instalacje elektryczne oraz materiały łatwopalne.

Głównym celem scenariusza jest ugaszenie pożaru poprzez neutralizację wszystkich aktywnych punktów ognia rozmieszczonych w obrębie hali. Ogniska pożaru występują na elementach wyposażenia takich jak opakowania składowane na paletach, wózki widłowe, metalowe szafki o podwyższonej wytrzymałości mechanicznej oraz lampy oświetleniowe zamontowane na suficie obiektu. Każdy z tych elementów stanowi oddzielny punkt pożarowy, który musi zostać ugaszony, aby możliwe było ukończenie symulacji z wynikiem pozytywnym.

Istotnym elementem scenariusza jest obecność czynnej instalacji elektrycznej w hali dystrybucyjnej. Na początku symulacji zasilanie elektryczne pozostaje włączone, co wpływa na sposób prowadzenia działań gaśniczych. W przypadku polewania wodą lamp oświetleniowych znajdujących się pod napięciem użytkownik narażony jest na porażenie prądem elektrycznym, co w symulacji skutkuje zmniejszaniem poziomu zdrowia. Długotrwałe kierowanie strumienia wody na elementy oświetleniowe przy włączonym zasilaniu może doprowadzić do przegrania symulacji. Jednym z zadań użytkownika jest zatem odnalezienie transformatorowni i wyłączenie głównego źródła zasilania hali, co eliminuje ryzyko porażenia prądem i umożliwia bezpieczne gaszenie wszystkich punktów ognia.

Scenariusz uwzględnia również obecność stref zagrożenia związanych z wysokim zadymieniem oraz istniejącymi gazami pożarowymi. Wejście użytkownika do takiej strefy bez założonej aparatury powietrznej powoduje szybkie zmniejszanie poziomu dostępnego powietrza. W przypadku korzystania z aparatury powietrznej zużycie powietrza spada znacznie wolniej, co odzwierciedla realne działanie sprzętu ochronnego. W chwili, gdy poziom powietrza w butli aparatury powietrznej osiągnie minimalne ciśnienie, rozlega się dźwięk gwizdka, oznaczającego informację o małej ilości powietrza, dlatego należy natychmiast ewakuować się ze strefy zagrożenia. Mechanizm ten ma na celu uświadomienie użytkownikowi znaczenia prawidłowego przygotowania do działań w środowisku zanieczyszczonym gazami pożarowymi.

W sytuacji, gdy poziom powietrza osiągnie wartość zerową, a użytkownik nadal przebywa w strefie zagrożenia, rozpoczyna się stopniowa utrata zdrowia. Spadek poziomu zdrowia do zera skutkuje zakończeniem symulacji niepowodzeniem. Takie rozwiązanie

wymusza na użytkowniku kontrolowanie zasobów oraz świadome podejmowanie decyzji dotyczących dalszego prowadzenia działań lub wycofania się z obszaru zagrożenia.

Proces gaszenia pożaru w symulatorze został zaprojektowany w sposób zbliżony do rzeczywistych działań strażackich. Aby rozpocząć gaszenie, użytkownik musi najpierw rozwinąć wąż gaśniczy, a następnie uruchomić dopływ wody poprzez włączenie ciśnienia. Dopiero po wykonaniu tych czynności możliwe jest skuteczne gaszenie ognisk pożaru. Taki przebieg symulacji wymusza zachowanie właściwej kolejności działań oraz zwiększa realizm szkolenia.

4.3. Scenariusze wybranych przypadków użycia systemu

Do generowania scenariuszy przypadków użycia w formie tabel wykorzystano narzędzie UML tools - Generator scenariuszy przypadków użycia [10].

Scenariusz przypadku użycia: I

Author name:	Tomasz
Author surname:	Chojnacki
Author group:	Group 1
Subject:	Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków
Use Case:	Uruchomienie symulacji
Language:	Polish
Preconditions:	• Aplikacja VR jest uruchomiona • Użytkownik znajduje się w menu głównym
Postconditions:	• Scena hali dystrybucyjnej została wczytana i uruchomiona • Użytkownik znajduje się w symulacji
Actors:	• Użytkownik
Main flow:	1. Użytkownik wyświetla menu główne aplikacji. 2. Użytkownik wybiera przycisk Start. 3. System wyświetla okno pośrednie z informacją o zapoznaniu się z informacjami oraz przyciski: Start i Powrót do menu. 4. Użytkownik wybiera przycisk Start. 5. System wyświetla ekran ładowania z paskiem postępu. 6. System ładuje scenę hali dystrybucyjnej. 7. System przenosi użytkownika do sceny hali dystrybucyjnej i rozpoczyna symulację.
Alternate flow:	• A1 Użytkownik rezygnuje z rozpoczęcia Użytkownik wybiera Powrót do menu w oknie pośrednim, system wraca do menu głównego. GOTO 1

Rysunek 4.1. Scenariusz przypadku użycia: „Uruchomienie symulacji” [Opracowanie własne].

Scenariusz przypadku użycia: II

Author name:	Tomasz
Author surname:	Chojnacki
Author group:	Group 1
Subject:	Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków
Use Case:	Zmiana ustawień
Language:	Polish
Preconditions:	• Aplikacja VR jest uruchomiona • Użytkownik znajduje się w menu głównym
Postconditions:	• Wybrane ustawienia zostały zapisane i zastosowane • Użytkownik wrócił do menu głównego
Actors:	• Użytkownik
Main flow:	1. Użytkownik wybiera w menu głównym przycisk Ustawienia. 2. System wyświetla panel ustawień. 3. Użytkownik wybiera jakość grafiki: wysoka/średnia/niska. 4. Użytkownik wybiera sposób obracania: skokowe/ciągłe. 5. Użytkownik zatwierdza ustawienia wyjściem z panelu. 6. System wyświetla menu główne.
Alternate flow:	• A1 Wyjście bez zatwierdzenia Użytkownik opuszcza panel ustawień bez wprowadzania zmian, system pozostawia poprzednie ustawienia i wraca do menu głównego. END FLOW

Rysunek 4.2. Scenariusz przypadku użycia: „Zmiana ustawień” [Opracowanie własne].

Scenariusz przypadku użycia: III

Author name:	Tomasz
Author surname:	Chojnacki
Author group:	Group 1
Subject:	Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków
Use Case:	Przegląd informacji
Language:	Polish
Preconditions:	- Aplikacja VR jest uruchomiona - Użytkownik znajduje się w menu głównym
Postconditions:	- Użytkownik zapoznał się z informacjami lub z nich zrezygnował - Użytkownik wrócił do menu głównego
Actors:	Użytkownik
Main flow:	1. Użytkownik wybiera w menu głównym przycisk Informacje. 2. System wyświetla listę przycisków tematów: PROJEKT, CHODZENIE, CHWYTANIE, OBRACANIE, TELEPORTACJA, WĄŻ STRAŻACKI, POWIETRZE, ZDROWIE, SYSTEM PRĄDU oraz POWRÓT DO MENU.. 3. Użytkownik wybiera jeden z tematów. 4. System wyświetla okno z opisem wybranej funkcji oraz przycisk POWRÓT. 5. Użytkownik wybiera przycisk POWRÓT. 6. System wraca do listy tematów. 7. Użytkownik wybiera przycisk POWRÓT DO MENU. 8. System wraca do menu głównego.
Alternate flow:	A1 Użytkownik nie wybiera tematu Użytkownik od razu wybiera przycisk POWRÓT DO MENU, system wraca do menu głównego. END FLOW

Rysunek 4.3. Scenariusz przypadku użycia: „Przegląd informacji” [Opracowanie własne].

Scenariusz przypadku użycia: IV

Author name:	Tomasz
Author surname:	Chojnacki
Author group:	Group 1
Subject:	Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków
Use Case:	Symulacja w hali dystrybucyjnej - osiągnięcie wygranej
Language:	Polish
Preconditions:	• Scena hali dystrybucyjnej została uruchomiona • Użytkownik steruje postacią w środowisku symulacji
Postconditions:	• Scena Wygrana została uruchomiona • Użytkownik wrócił do menu głównego po wybraniu przycisku
Actors:	• Użytkownik
Main flow:	1. System uruchamia scenę hali dystrybucyjnej i rozpoczyna symulację. 2. Użytkownik przygotowuje się do działań (może założyć aparaturę powietrzną). 3. Użytkownik rozwija wąż i uruchamia dopływ wody (ciśnienie). 4. Użytkownik wyłącza zasilanie w hali, aby bezpiecznie gasić lampy bez ryzyka utraty zdrowia. 5. Użytkownik lokalizuje i gasi punkty ognia występujące na: opakowaniach, wózkach widłowych, metalowych szafkach oraz lampach. 6. System wykrywa, że wszystkie punkty ognia zostały całkowicie ugaszone. 7. System kończy scenę symulacji i ładuje scenę Wygrana. 8. System wyświetla ekran Wygrana z przyciskiem POWRÓT DO MENU. 9. Użytkownik wybiera przycisk POWRÓT DO MENU. 10. System ładuje menu główne aplikacji.
Alternate flow:	• A1 Użytkownik nie wyłącza prądu i gasi lampy pod napięciem System zmniejsza zdrowie, możliwe przejście do sceny przegranej. GOTO A2 • A2 Zdrowie spada do zera System ładuje scenę Przegrana i wyświetla przycisk POWRÓT DO MENU. END FLOW

Rysunek 4.4. Scenariusz przypadku użycia: „Symulacja w hali dystrybucyjnej – osiągnięcie wygranej” [Opracowanie własne].

Scenariusz przypadku użycia: V

Author name:	Tomasz
Author surname:	Chojnacki
Author group:	Group 1
Subject:	Projekt i realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości dla szkolenia początkujących strażaków
Use Case:	Symulacja w hali dystrybucyjnej - osiągnięcie przegranej
Language:	Polish
Preconditions:	• Scena hali dystrybucyjnej została uruchomiona • Użytkownik steruje postacią w środowisku symulacji
Postconditions:	• Scena Przegrana została uruchomiona • Użytkownik wrócił do menu głównego po wybraniu przycisku
Actors:	• Użytkownik
Main flow:	1. System uruchamia scenę hali dystrybucyjnej i rozpoczyna symulację. 2. Użytkownik wchodzi do hali dystrybucyjnej bez założonej aparatury powietrznej. 3. System wykrywa wejście użytkownika do strefy zagrożenia. 4. System zaczyna szybko zmniejszać poziom powietrza. 5. Poziom powietrza osiąga wartość zerową. 6. System rozpoczyna zmniejszanie poziomu zdrowia (użytkownik pozostaje w strefie zagrożenia). 7. Poziom zdrowia osiąga wartość zerową. 8. System ładuje scenę Przegrana. 9. System wyświetla ekran Przegrana z przyciskiem POWRÓT DO MENU. 10. Użytkownik wybiera przycisk POWRÓT DO MENU. 11. System ładuje menu główne aplikacji.
Alternate flow:	• A1 Użytkownik opuszcza strefę zagrożenia przed wyczerpaniem zdrowia System przestaje odejmować zdrowie; użytkownik może kontynuować symulację. GOTO 2

Rysunek 4.5. Scenariusz przypadku użycia: „Symulacja w hali dystrybucyjnej – osiągnięcie przegranej” [Opracowanie własne].

4.4. Interakcje użytkownika

Jednym z głównych elementów wpływających na realizm symulacji VR są interakcje użytkownika z elementami wirtualnego otoczenia. W projektowanym symulatorze szczególnie zwrócono uwagę na odwzorowanie naturalnych zachowań rąk oraz sposób interakcji użytkownika z obiektami. Celem było uzyskanie kompromisu pomiędzy wysokim poziomem realizmu fizycznego a komfortem użytkowania aplikacji w warunkach ograniczonej precyzji sterowania, wynikającej z użycia kontrolerów VR.

Użytkownik w symulacji steruje wirtualnymi dłońmi, które reprezentowane są przez modele rąk odpowiadające położeniu i orientacji kontrolerów. Modele te służą do wykonywania interakcji takich jak chwytanie, naciskanie przycisków czy obsługa elementów wyposażenia. W celu zwiększenia intuicyjności interakcji zastosowano rozwiązanie polegające na ograniczeniu kolizji samych modeli dłoni z wybranymi elementami środowiska.

Przykładem takiego podejścia są drzwi znajdujące się przy elementach wyposażenia wozu strażackiego oraz transformatorowni. Podczas otwierania drzwi użytkownik może swobodnie przeniknąć modelem dłoni przez ich powierzchnię, co ułatwia precyzyjne uchwycenie klamki. Jednocześnie zastosowano pełną kolizję dla kapsuły postaci użytkownika, dzięki czemu sama postać nie może przeniknąć przez drzwi. Takie rozwiązanie zwiększa realizm zachowania symulatora, jednocześnie eliminuje negatywne emocje związane z koniecznością bardzo dokładnego ustawienia dłoni w przestrzeni wirtualnej.

Podobne podejście zastosowano w przypadku zaworu końcowego węża strażackiego. Element uchwytu zaworu został zaprojektowany w taki sposób, aby model dłoni mógł przez niego przeniknąć i go chwycić. Rozwiązanie to zapobiega sytuacjom, w których zawór mógłby zostać przypadkowo odrzucony w wyniku nieprecyzyjnego kontaktu z dłonią. Jednocześnie sama końcówka zaworu oraz wąż gaśniczy posiadają aktywne kolizje zarówno z rękami użytkownika, jak i z modelem postaci. Dzięki temu zachowanie całego zestawu gaśniczego jest zbliżone do rzeczywistego, a ruch węża reaguje na położenie użytkownika oraz elementy otoczenia.

W symulacji zastosowano również przyciski odpowiedzialne za obsługę systemów, takich jak uruchamianie ciśnienia w systemie autopompy czy wyłączanie transformatora zasilającego halę. W przypadku tych elementów wybrano optymalne rozwiązanie dotyczące kolizji. Zamiast pełnej kolizji całej dłoni z przyciskiem, aktywny obszar interakcji ograniczono do centralnego punktu dłoni o stałym promieniu od jej środka. Taki sposób

działania lepiej odwzorowuje rzeczywiste zachowanie, w którym duże przyciski przemysłowe wciskane są całą dłonią, a nie pojedynczym palcem.

Zastosowanie ograniczenia kolizji miało na celu poprawienie ergonomii symulacji, gdyż użytkownik nie posiada pełnej kontroli nad ruchami palców, a sterowanie odbywa się za pomocą kontrolerów. Ograniczenie punktu interakcji do centralnej części dłoni znacząco poprawia precyzję obsługi przycisków oraz zmniejsza ryzyko niezamierzonych aktywacji lub problemów z trafieniem w niewielki obszar kolizji.

5. IMPLEMENTACJA APLIKACJI VR

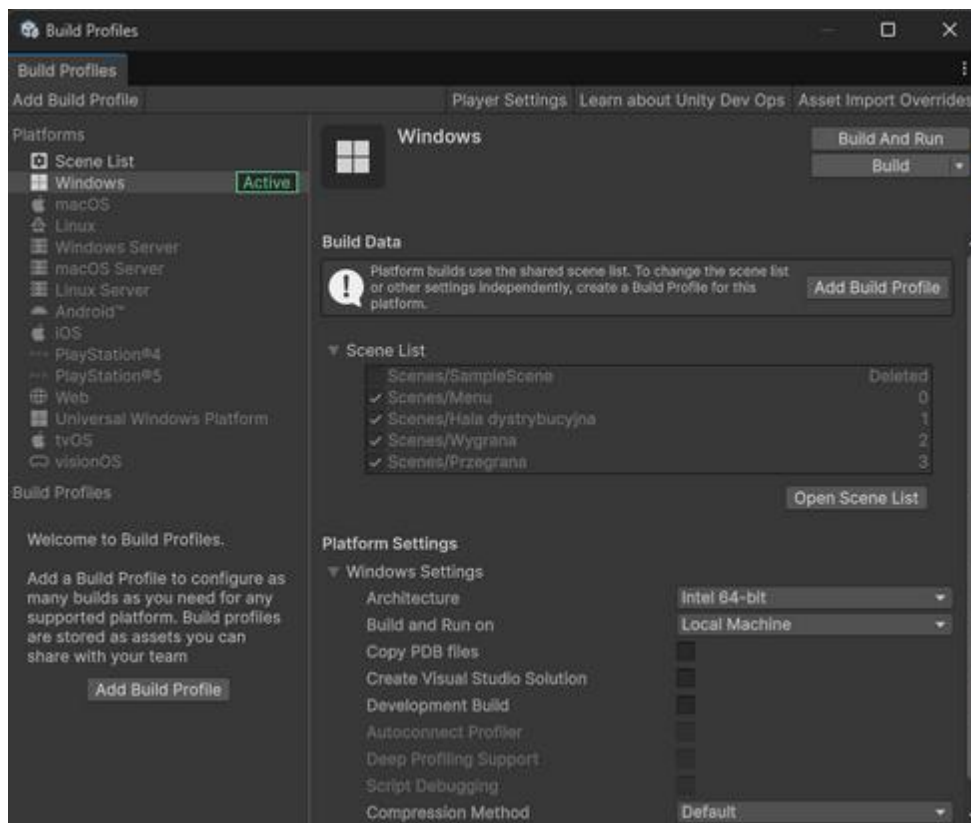
5.1. Konfiguracja Unity pod Meta Quest 2

Konfiguracja środowiska Unity została rozpoczęta od wyboru odpowiedniej wersji silnika. W projekcie wykorzystano wersję Unity 6000.0.58f2, należącą do jednej z najnowszych generacji silnika Unity. Wybór tej wersji był podyktowany stabilnością działania, dostępnością aktualnych pakietów XR oraz zgodnością z goglami Meta Quest 2 wykorzystywanymi w konfiguracji PC VR [8].

Po zainstalowaniu silnika Unity wraz z modułami przeznaczonymi dla systemu Windows, utworzono nowy projekt o nazwie „Training Simulator For Beginner Firefighters VR”, który następnie został dostosowany do pracy w środowisku wirtualnej rzeczywistości.

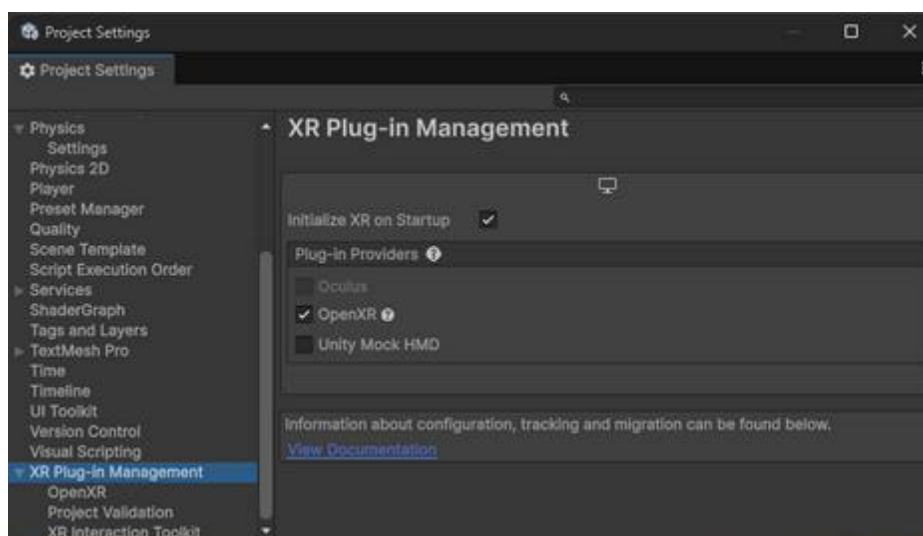
W Unity 6000 konfiguracja platformy docelowej realizowana jest z wykorzystaniem mechanizmu Build Profiles, który pozwala na niezależne definiowanie ustawień kompilacji dla poszczególnych platform oraz ułatwia zarządzanie projektem w przypadku rozbudowy [8].

W ramach projektu aktywowano profil kompilacji dla platformy Windows, który został ustawiony jako aktywny profil budowania aplikacji. W profilu tym zdefiniowano listę scen wykorzystywanych w aplikacji, obejmującą scenę menu głównego, scenę hali dystrybucyjnej oraz sceny końcowe odpowiadające wygranej i przegranej. Takie podejście umożliwia poprawne przełączanie się pomiędzy scenami w trakcie działania symulacji. W ustawieniach profilu Windows określono architekturę docelową jako Intel 64-bit, co jest standardowym rozwiązaniem dla aplikacji VR uruchamianych na komputerach osobistych. Pozostałe opcje, takie jak tryb developerski czy profilowanie, pozostawiono wyłączone w wersji docelowej aplikacji.



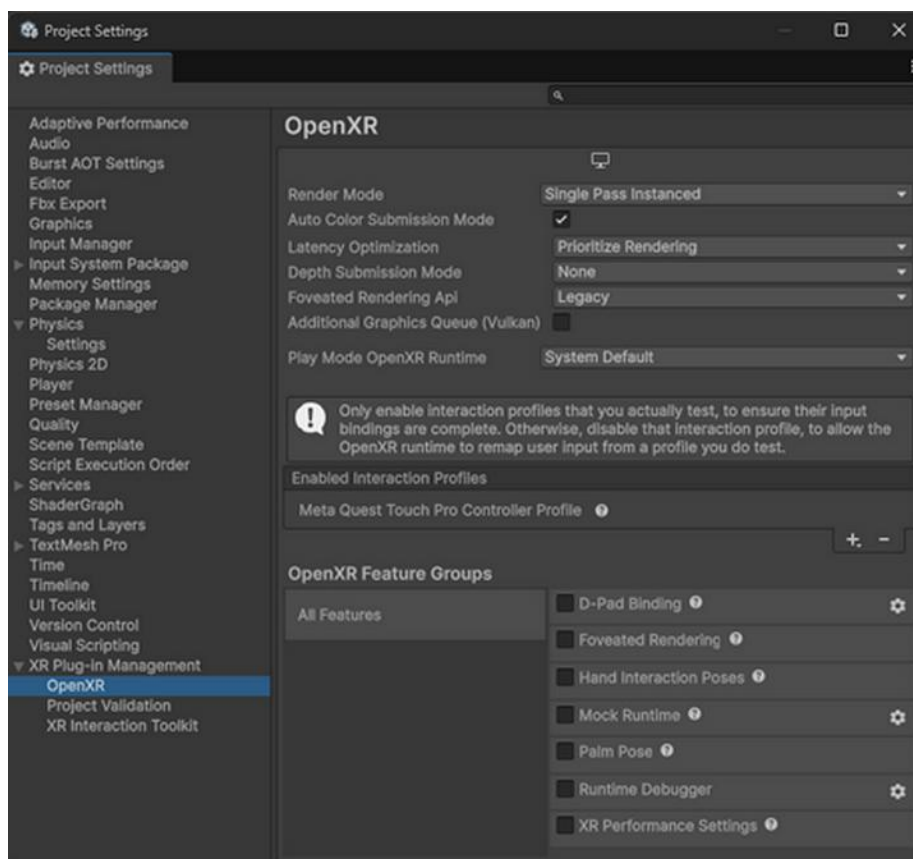
Rysunek 5.1. Konfiguracja Build Profiles [Źródło własne].

Kolejnym etapem było skonfigurowanie systemu obsługi wirtualnej rzeczywistości. W projekcie wykorzystano standard OpenXR, który stanowi uniwersalne rozwiązanie do komunikacji pomiędzy aplikacją a sprzętem VR. Zastosowanie OpenXR pozwala na obsługę gogli Meta Quest 2. W ustawieniach Project Settings w sekcji XR Plug-in Management aktywowano obsługę OpenXR dla platformy PC. Dzięki temu środowisko wirtualnej rzeczywistości jest automatycznie uruchamiane wraz ze startem symulacji.



Rysunek 5.2. XR Plug-in Management – OpenXR dla platformy PC [Źródło własne].

W ramach konfiguracji OpenXR wybrano profil interakcji „Meta Quest Touch Pro Controller Profile”, który odpowiada kontrolerom wykorzystywanym przez gogle Meta Quest 2. Profil ten zapewnia poprawne mapowanie przycisków, triggerów oraz osi joysticka, co jest niezbędne do prawidłowego działania interakcji użytkownika w symulacji.

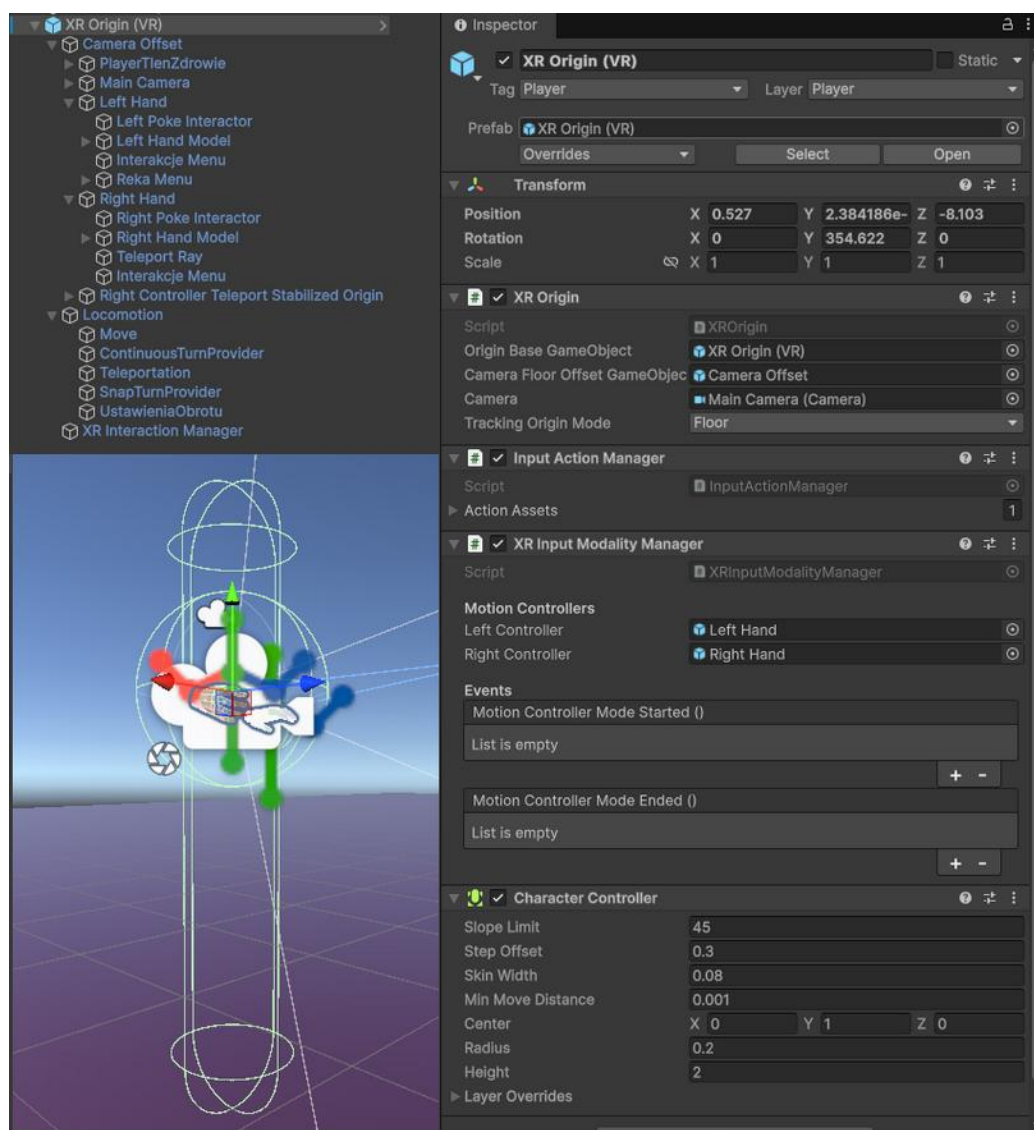


Rysunek 5.3. OpenXR – Interaction Profiles [Źródło własne].

W kolejnym kroku do projektu dodano pakiet XR Interaction Toolkit w wersji 3.0.8, który stanowi podstawowe narzędzie do implementacji interakcji w środowisku VR. Pakiet ten umożliwia obsługę takich mechanizmów jak chwytanie obiektów, interakcja z elementami interfejsu użytkownika w przestrzeni trójwymiarowej oraz systemy lokomocji [8]. Po zainstalowaniu pakietu przeprowadzono jego inicjalizację, co pozwoliło na automatyczne skonfigurowanie wymaganych komponentów projektu, w tym systemów wejścia oraz menedżera interakcji XR.

5.2. Implementacja sterowania VR

Sterowanie użytkownika zostało zaimplementowane z wykorzystaniem pakietu XR Interaction Toolkit. Głównym elementem odpowiedzialnym za reprezentację użytkownika w scenie jest obiekt XR Origin (VR), który pełni rolę kontenera dla kamery, kontrolerów oraz komponentów odpowiedzialnych za poruszanie się i kolizje z otoczeniem. Struktura XR Origin została zaprojektowana w sposób umożliwiający jednocześnie śledzenie ruchów głowy użytkownika, pozycji kontrolerów oraz poprawne odwzorowanie fizycznej obecności postaci w świecie wirtualnym. W skład XR Origin wchodzi między innymi: kamera główna, modele dłoni, komponenty wejścia oraz system lokomocji. Takie rozwiązanie pozwala na zachowanie spójności pomiędzy ruchem fizycznym użytkownika a jego pozycją w symulacji.

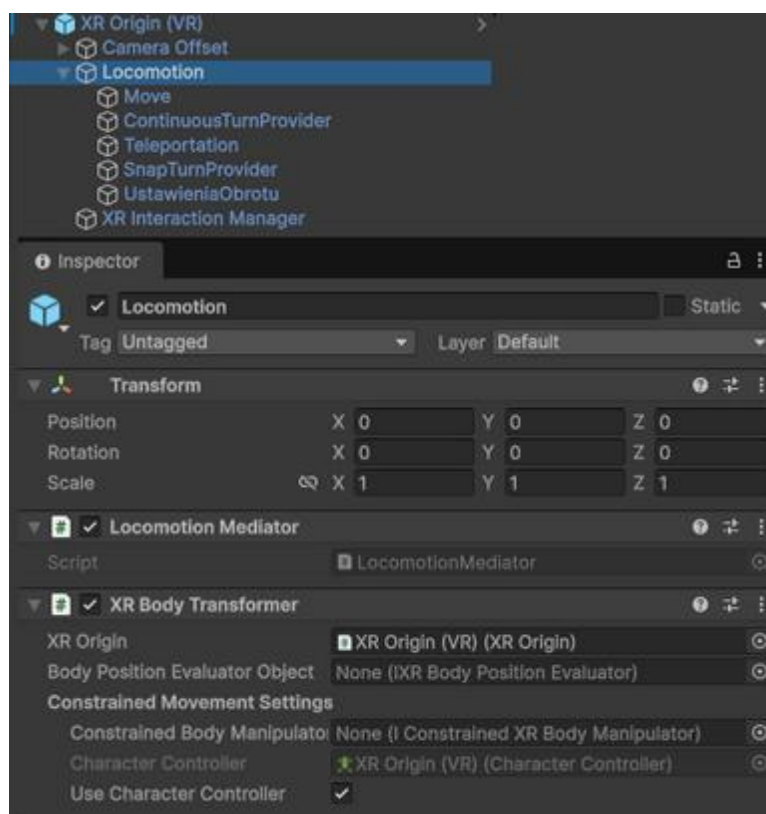


Rysunek 5.4. Hierarchia XR Origin (VR) oraz widok inspektora [Źródło własne].

5.2.1. Implementacja chodzenia

Implementacja chodzenia w symulatorze została zrealizowana z wykorzystaniem systemu lokomocji dostępnego w XR Interaction Toolkit. Głównym elementem odpowiedzialnym za ruch użytkownika jest komponent Dynamic Move Provider, który umożliwia płynne przemieszczanie się w przestrzeni wirtualnej na podstawie sygnałów wejściowych z kontrolerów VR.

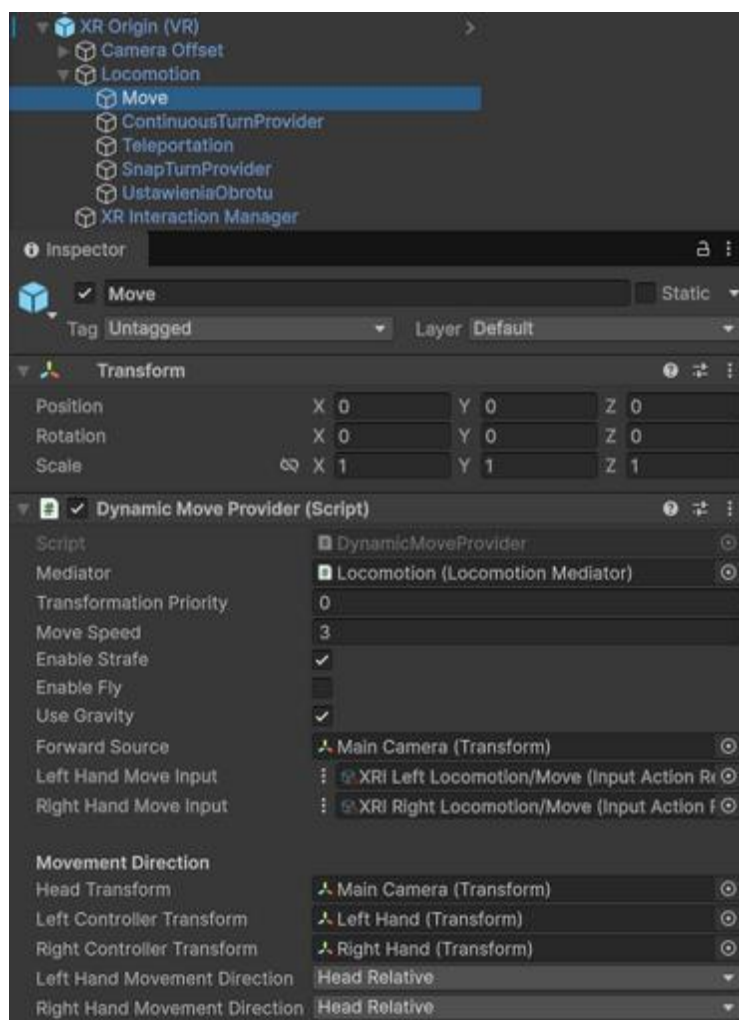
Komponent Dynamic Move Provider został umieszczony w strukturze obiektu Locomotion, który pełni rolę nadrzędnego kontenera dla wszystkich mechanizmów związanych z ruchem użytkownika. Ruch jest zarządzany przez Locomotion Mediator, który koordynuje działanie poszczególnych dostawców lokomocji i zapobiega konfliktom pomiędzy różnymi trybami poruszania się.



Rysunek 5.5. Widok inspektora obiektu Locomotion [Źródło własne].

Ruch użytkownika realizowany jest w oparciu o sygnały wejściowe z joysticka lewego kontrolera. Kierunek poruszania się został ustawiony jako head-relative, co oznacza, że wektor ruchu wyznaczany jest na podstawie aktualnego kierunku patrzenia użytkownika. Takie rozwiązanie zwiększa intuicyjność sterowania i pozwala na naturalne poruszanie się w środowisku symulacji bez konieczności ciągłej korekty orientacji ciała. Prędkość z jaką porusza się użytkownik została dobrana w taki sposób, aby umożliwić sprawne

przemieszczanie się po hali dystrybucyjnej, jednocześnie nie powodując nadmiernego przyspieszenia, które mogłoby negatywnie wpływać na komfort korzystania z symulatora. Dodatkowo włączono obsługę grawitacji, co pozwala na realistyczne zachowanie postaci podczas poruszania się po nierównym terenie lub rampach.



Rysunek 5.6. Widok inspektora obiektu Move (poruszania) [Źródło własne].

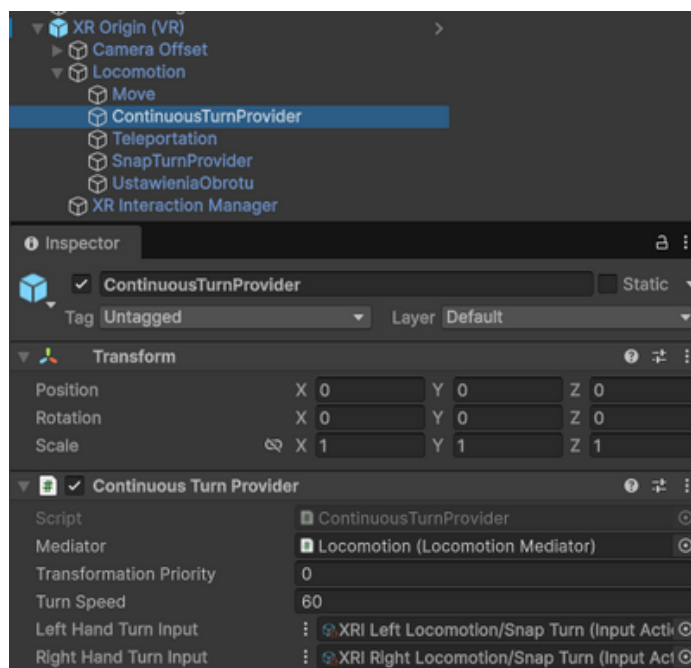
Aby zapewnić realistyczną kolizję postaci z elementami otoczenia, do obiektu XR Origin (VR) dodano komponent Character Controller [Rysunek 5.4]. Komponent ten odpowiada za fizyczną reprezentację użytkownika w świecie wirtualnym oraz zapobiega przenikaniu postaci przez ściany, obiekty czy elementy infrastruktury hali. Parametry takie jak wysokość, promień oraz punkt środka kontrolera zostały dostosowane do sylwetki postaci, co umożliwia poprawne odwzorowanie pozycji użytkownika względem otoczenia.

Dodatkowym elementem systemu chodzenia jest XR Body Transformer [Rysunek 5.5], który odpowiada za synchronizację pozycji ciała użytkownika z ruchem kamery oraz kontrolerów. Zastosowane rozwiązanie umożliwia płynne, ciągłe poruszanie się po hali dystrybucyjnej, co jest istotne z punktu widzenia symulacji.

5.2.2. Implementacja obracania

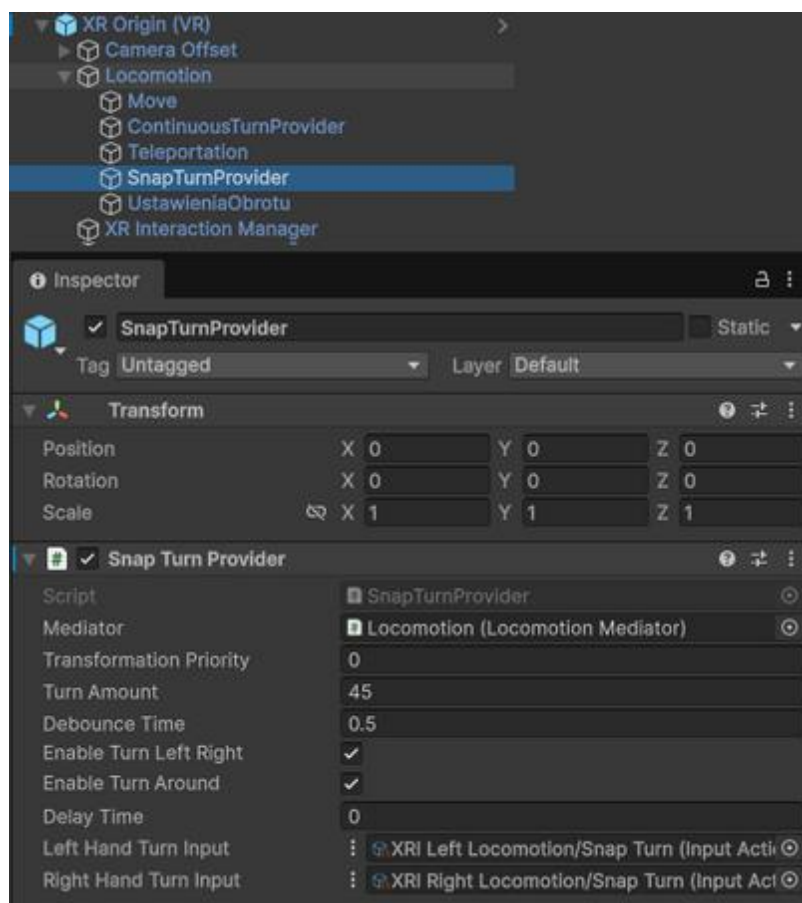
Obracanie użytkownika w środowisku symulacyjnym stanowi istotny element systemu sterowania, który został zaimplementowany na dwa sposoby: obracanie ciągłe oraz obracanie skokowe (snap turn). Oba tryby zostały zrealizowane z wykorzystaniem mechanizmów lokomocji dostępnych w pakiecie XR Interaction Toolkit i są zarządzane przez wspólny system lokomocji. Podobnie jak w przypadku chodzenia, komponenty odpowiedzialne za obracanie zostały umieszczone w strukturze obiektu Locomotion, a ich działanie koordynowane jest przez Locomotion Mediator. Takie rozwiązanie zapobiega konfliktom pomiędzy różnymi trybami sterowania oraz umożliwia dynamiczne przełączanie aktywnego sposobu obracania w trakcie działania aplikacji [Rysunek 5.5].

Obracanie ciągłe zostało zaimplementowane przy użyciu komponentu Continuous Turn Provider. Komponent ten umożliwia płynną, nieprzerwaną zmianę orientacji użytkownika wokół osi pionowej na podstawie sygnałów wejściowych z osi analogowej kontrolera. W symulatorze obrót realizowany jest za pomocą joysticka prawego kontrolera, co zapewnia intuicyjne sterowanie zgodne z powszechnie stosowanymi standardami w aplikacjach VR. Prędkość obrotu została określona w parametrach komponentu Continuous Turn Provider i dobrana w taki sposób, aby umożliwić sprawną zmianę kierunku patrzenia bez wywoływania dezorientacji.



Rysunek 5.7. Widok inspektora komponentu Continuous Turn Provider [Źródło własne].

Obracanie skokowe zostało zaimplementowane z wykorzystaniem komponentu Snap Turn Provider. W tym trybie obrót realizowany jest w sposób skokowy, o określony z góry kąt wynoszący 45 stopni. Obracanie skokowe również jest sterowane za pomocą joysticka prawego kontrolera, jednak zamiast płynnej zmiany orientacji następuje natychmiastowy obrót o ustaloną wartość. Dodatkowo występuje też czas opóźnienia pomiędzy kolejnymi obrotami. Takie rozwiązanie zapobiega przypadkowemu wielokrotnemu obracaniu się w krótkim czasie oraz zwiększa kontrolę użytkownika nad zmianą orientacji.



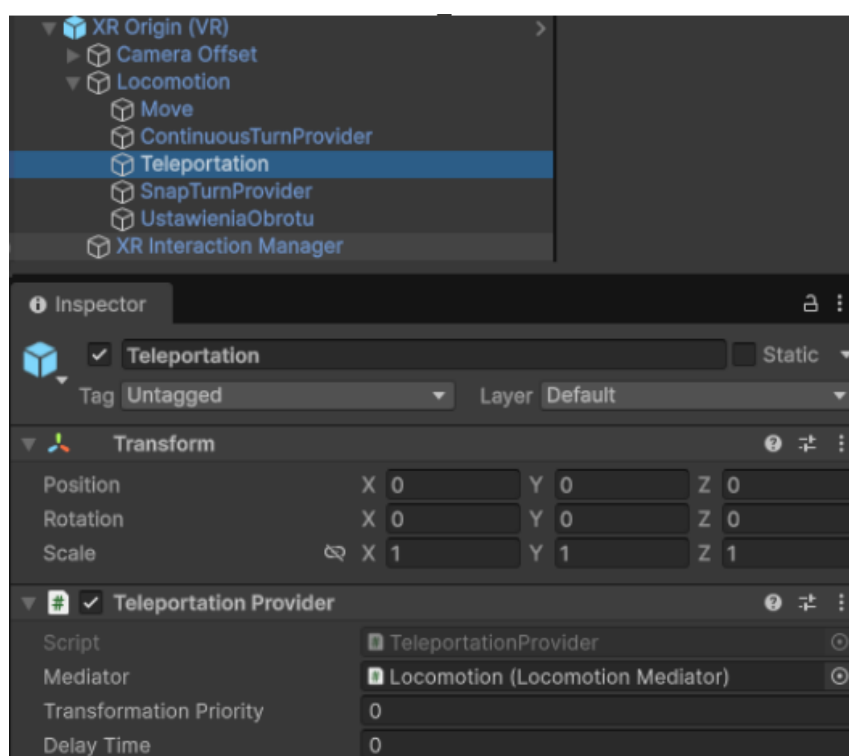
Rysunek 5.8. Widok inspektora komponentu Snap Turn Provider [Źródło własne].

Obracanie skokowe zostało zaimplementowane z myślą o użytkownikach, którzy są bardziej podatni na dyskomfort związany z płynnym ruchem w wirtualnej rzeczywistości. Nagła, ale krótka zmiana orientacji ogranicza ryzyko wystąpienia choroby symulacyjnej, takiej jak zawroty głowy czy nudności [5,7].

Zarówno obracanie ciągłe, jak i skokowe zostały zintegrowane z systemem lokomocji w taki sposób, aby w danym momencie aktywny był tylko jeden tryb obracania. Przełączanie pomiędzy nimi odbywa się z poziomu menu ustawień aplikacji, jednak logika interfejsu użytkownika została omówiona w dalszej części pracy.

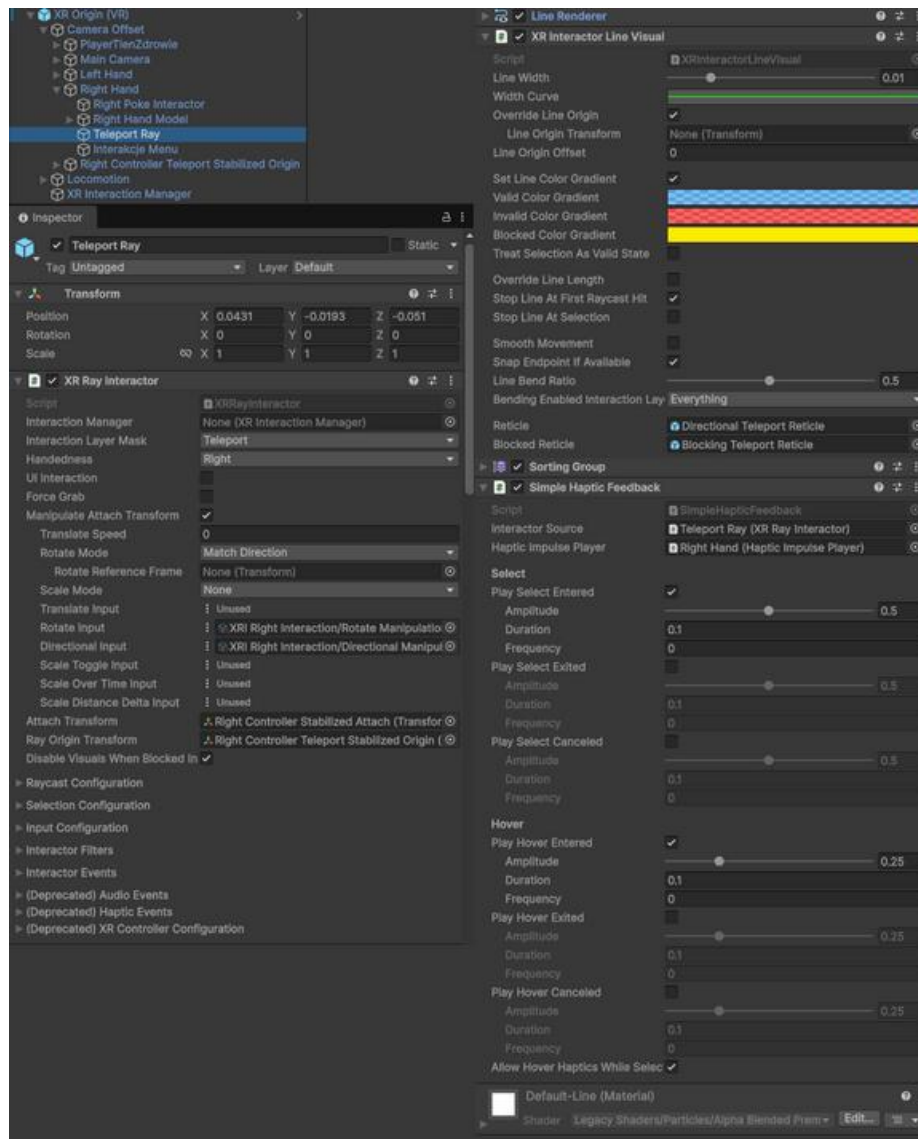
5.2.3. Implementacja teleportacji

Teleportacja w symulatorze została zaimplementowana jako alternatywny mechanizm przemieszczania się użytkownika, mający na celu zwiększenie komfortu korzystania z aplikacji VR oraz ograniczenie ryzyka wystąpienia dyskomfortu związanego z ruchem ciągłym. Rozwiązanie to zostało zrealizowane z wykorzystaniem narzędzi dostępnych w XR Interaction Toolkit, w szczególności komponentu Teleportation Provider, który odpowiada za przenoszenie użytkownika do wybranego punktu w przestrzeni wirtualnej.



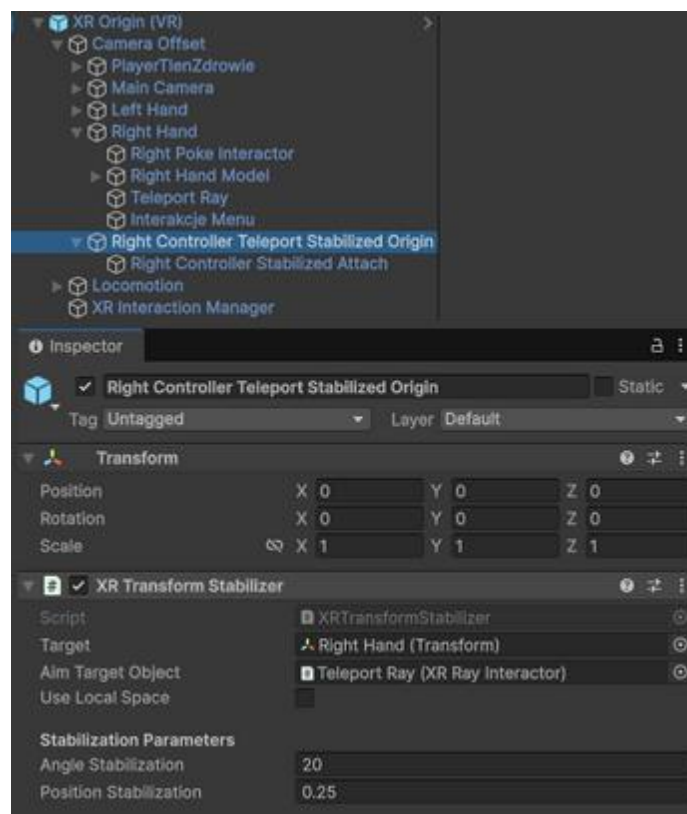
Rysunek 5.9. Widok inspektora komponentu Teleportation [Źródło własne].

System teleportacji, podobnie jak pozostałe mechanizmy lokomocji, został umieszczony w strukturze obiektu Locomotion i jest zarządzany przez Locomotion Mediator [Rysunek 5.5]. Aktywacja teleportacji odbywa się za pomocą joysticka prawego kontrolera VR. Przechylenie joysticka do góry powoduje uruchomienie trybu teleportacji. W tym momencie z prawej ręki emitowany jest promień, który umożliwia wskazanie miejsca docelowego. Funkcjonalność ta została zrealizowana przy użyciu komponentu XR Ray Interactor, przypisanego do prawej dłoni użytkownika.

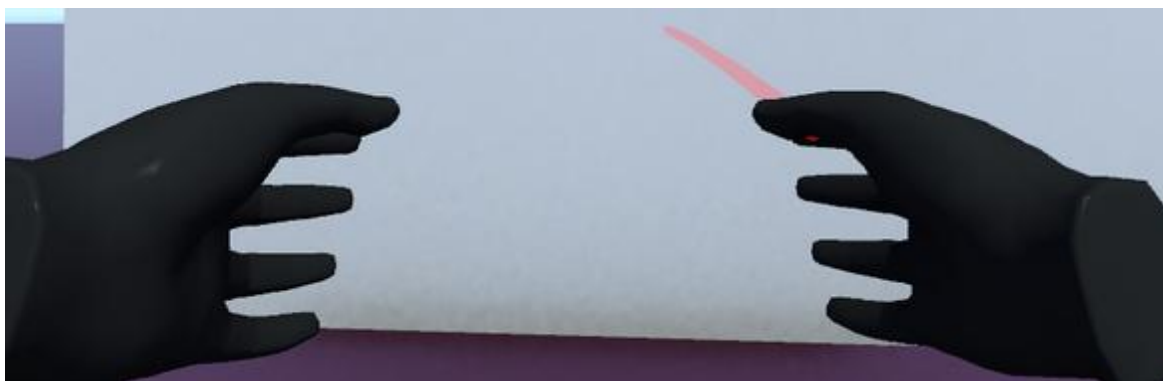


Rysunek 5.10. Widok inspektora obiektu Teleport Ray [Źródło własne].

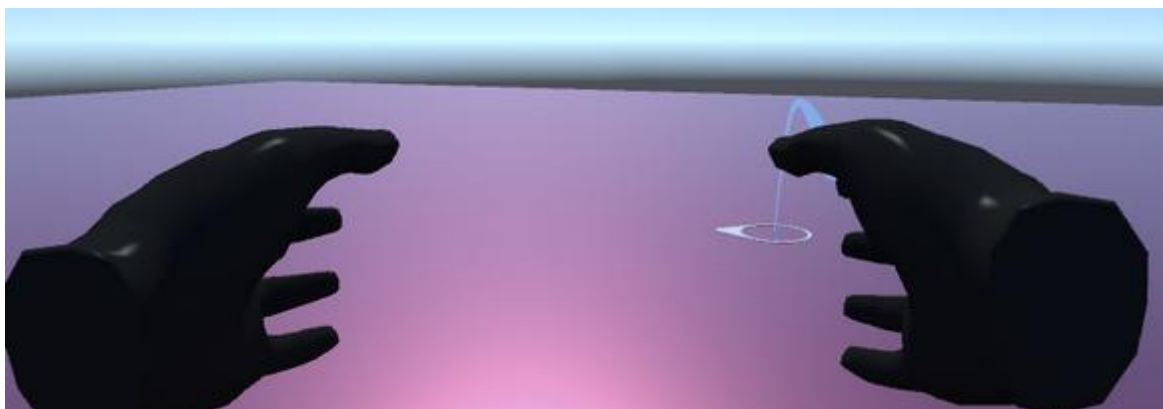
W miejscu, w które skierowany jest promień, na powierzchni podłoża wyświetlany jest okrągły wskaźnik teleportacji, tzw. Directional Teleport Reticle. Wskaźnik ten informuje użytkownika, czy wybrane miejsce jest poprawne i możliwe do wykorzystania jako punkt docelowy. W przypadku obszarów niedozwolonych linia teleportacji zmienia się na czerwoną, co uniemożliwia teleportację. Dodatkowo jest możliwość określenia kierunku obrotu użytkownika po zakończeniu teleportacji. Kierunek ten wyznaczany jest na podstawie wychylenia joysticka w trakcie wyboru miejsca docelowego, co pozwala na lepsze dopasowanie orientacji użytkownika względem otoczenia.



Rysunek 5.11. Widok inspektora obiektu, w którym przyczepiony jest punkt startowy promienia Right Controller Teleport Stabilized Origin [Źródło własne].



Rysunek 5.12. Czerwony wskaźnik teleportacji [Źródło własne].



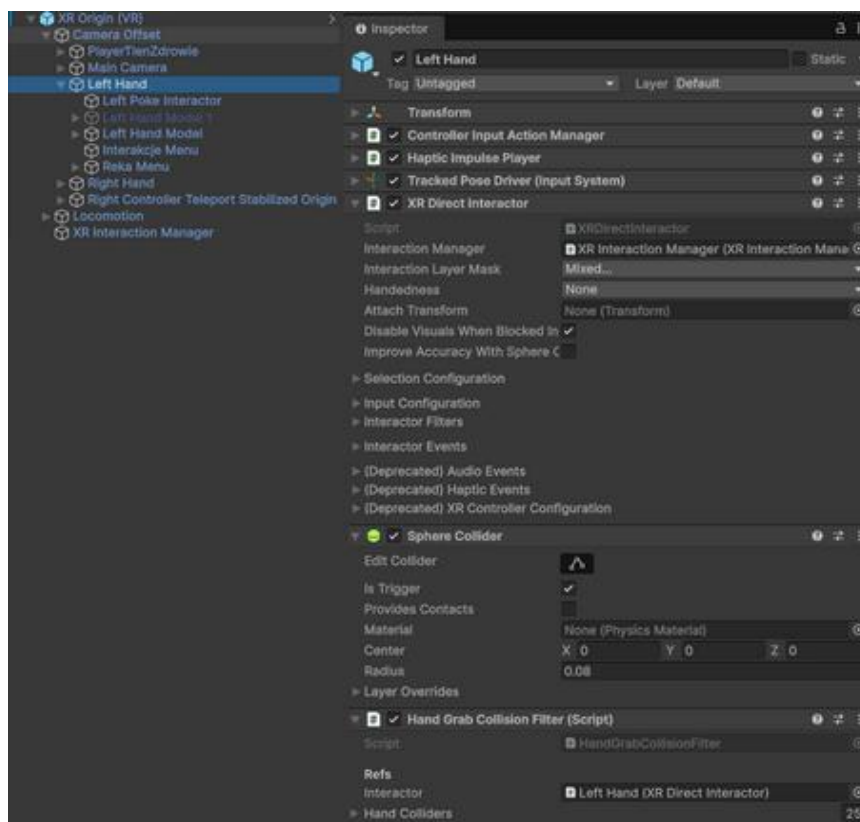
Rysunek 5.13. Wskaźnik teleportacji Directional Teleport Reticle [Źródło własne].

Po puszczeniu joysticka następuje natychmiastowe przeniesienie użytkownika do wskazanego punktu. Teleportacja realizowana jest bez animacji przejścia, co minimalizuje ryzyko dezorientacji. Dla zwiększenia czytelności interakcji zastosowano także odczucie haptyczne, generowane w momencie zatwierdzenia teleportacji za pomocą komponentu Simple Haptic Feedback przypisanego do prawego kontrolera [Rysunek 5.10].

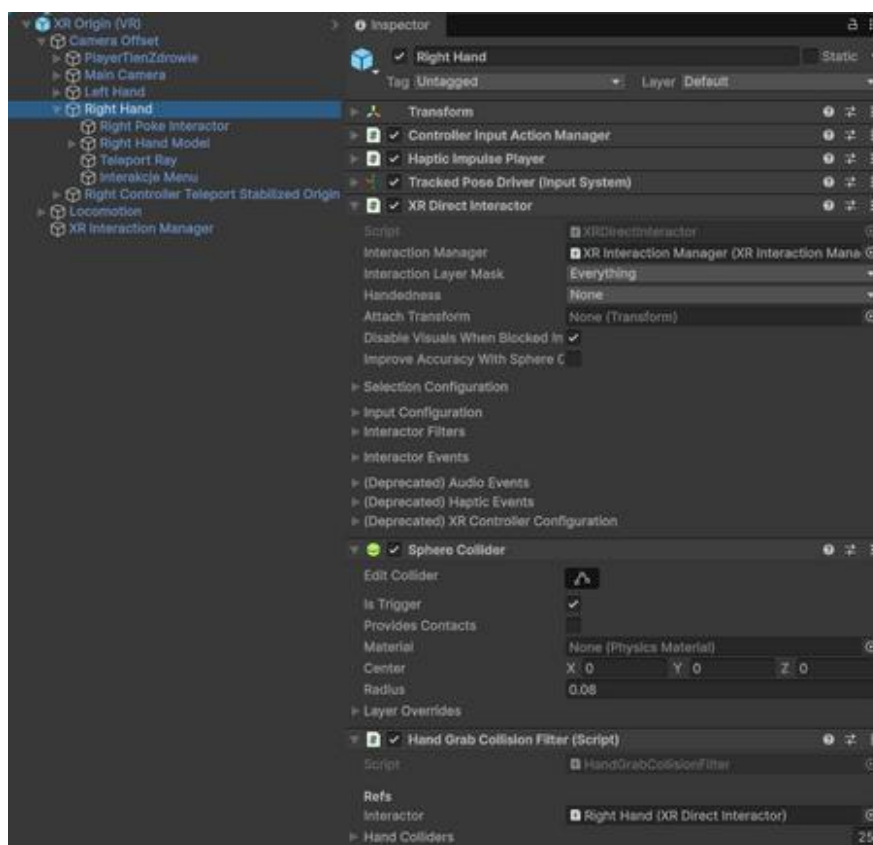
5.2.4. Implementacja chwytania

Implementacja chwytania obiektów w symulatorze została wykonana w oparciu o mechanizmy interakcji dostarczane przez XR Interaction Toolkit. Funkcjonalność ta pozwala użytkownikowi na wykonywanie podstawowych czynności operacyjnych, takich jak podnoszenie narzędzi oraz otwieranie drzwi.

W aplikacji chwytanie realizowane jest poprzez użycie przycisku Grab, znajdującego się pod palcami na bocznej części kontrolera. W praktyce użytkownik zbliża wirtualną dłoń do wybranego obiektu, a następnie wciska przycisk Grab, co powoduje przechwycenie obiektu przez dłoń. Zwolnienie przycisku skutkuje puszczeniem przedmiotu, co pozwala na jego odłożenie lub upuszczenie w wybranym miejscu. Za wykrywanie możliwości chwytu odpowiada komponent XR Direct Interactor, przypisany do każdej z dłoni użytkownika. Komponent ten umożliwia interakcję z obiektami znajdującymi się w bezpośrednim zasięgu ręki. Dzięki temu chwytanie zbliżone jest do rzeczywistego, gdzie konieczne jest fizyczne „sięgnięcie” po przedmiot. W projekcie do dłoni dodano także Sphere Collider, który pełni rolę kolizji wykorzystywanej do wykrycia obiektów w zasięgu chwytu.



Rysunek 5.14. Widok inspektora Left Hand, XR Direct Interactor oraz Sphere Collider [Źródło własne].



Rysunek 5.15. Widok inspektora Right Hand, XR Direct Interactor oraz Sphere Collider [Źródło własne].

W celu poprawy stabilności interakcji oraz ograniczenia niepożądanych kolizji pomiędzy rękami a chwytanymi obiektami zastosowano dodatkowy komponent Hand Grab Collision Filter. Jego zadaniem jest kontrolowanie kolizji dłoni w trakcie chwytania i minimalizowanie problemów takich jak blokowanie obiektów przez własne kolidery dłoni lub niekontrolowane zachowanie fizyki podczas podnoszenia. Rozwiązanie to poprawia płynność użytkowania i zwiększa przewidywalność zachowania obiektów podczas interakcji.

Każdy obiekt, który w symulatorze może być podniesiony lub przenoszony, musiał zostać odpowiednio przygotowany od strony fizyki i interakcji. W tym celu dodano do nich trzy podstawowe komponenty:

- Box Collider lub Sphere Collider – odpowiadające za kolizję, wykorzystywaną do wykrywania kontaktu oraz poprawnego zachowania obiektu w przestrzeni,
- Rigidbody – umożliwiający działanie fizyki, podczas podnoszenia, upuszczania oraz przenoszenia obiektu,
- XR Grab Interactable – komponent interakcji, który definiuje obiekt jako możliwy do przechwycenia przez XR Direct Interactor.

Tak przygotowane obiekty mogą być wykrywane przez dłonie użytkownika i obsługiwane w czasie rzeczywistym zgodnie z regułami systemu interakcji XR [8].

5.3. Modele 3D w symulacji

Jednym z elementów wpływających na realizm oraz wartość dydaktyczną symulatora strażackiego VR są zastosowane modele 3D. Odpowiednie odwzorowanie postaci, wyposażenia oraz otoczenia ma istotne znaczenie dla wiarygodności prezentowanych procedur szkoleniowych. W dalszej części podrozdziału przedstawiono modele 3D znajdujące się w symulacji.

Model strażaka [11] został przygotowany w zakresie wyglądu zewnętrznego oraz kolorystyki. Zmieniono barwy umundurowania na odpowiadające rzeczywistym strojom strażackim, stosowanym w Polsce. Na hełmie dodano napis „Kielce University of Technology”, pełniący funkcję dekoracyjną. Model został też przygotowany w taki sposób, aby mógł być wielokrotnie wykorzystywany w różnych scenach symulacji. W każdej z nich przypisano mu odrębne animacje, które odpowiadają kontekstowi danej sytuacji oraz aktualnemu stanowi rozgrywki. Zastosowanie zróżnicowanych animacji zwiększa przekazywanie informacji i emocji bez konieczności stosowania dodatkowych komunikatów tekstowych.



Rysunek 5.16. Model strażaka [Źródło własne].



Rysunek 5.17. Jednak klatka przedstawiająca animacje modelu strażaka [Źródło własne].

Model wozu strażackiego [12] został zaadaptowany i zmodyfikowany na potrzeby symulatora zarówno na etapie modelowania 3D w środowisku Blender, jak i podczas integracji w silniku Unity. Celem wprowadzonych zmian było zwiększenie realizmu wizualnego oraz dostosowanie modelu do wymagań funkcjonalnych symulacji.

W środowisku Blender dokonano modyfikacji geometrii pojazdu poprzez usunięcie jednej ze ścian bocznych zabudowy. Działanie to umożliwiło późniejsze umieszczenie interaktywnego panelu ciśnienia oraz drzwi autopompy. Dodatkowo w tylnej części pojazdu przygotowano przestrzeń umożliwiającą logiczne rozmieszczenie elementów instalacji wodnej oraz aparatury powietrznej.

Po zaimportowaniu modelu do silnika Unity wykonano dalsze prace adaptacyjne. Sprzęt aparatury powietrznej został wizualnie zasłonięty drzwiami, żeby zachować realistyczność symulacji. Zmieniono również tekstury modelu w celu zwiększenia liczby

detali i odświeżenia samego pojazdu. Na pojeździe dodano nazwę wozu „Tomasz 25”, gdzie liczba oznacza rok rozpoczęcia prac nad projektem. Umieszczono także oznaczenia jednostki „STRAŻ PŚK WEAiI” oraz numer alarmowy „998”, co dodatkowo podkreśla polski charakter symulacji.

W silniku dodano również punkty świetlne w miejscach odpowiadających rzeczywistym sygnałom świetlnym pojazdu strażackiego. Zastosowane źródła światła pełnią funkcję wizualną oraz nadają bardziej realistyczny odbiór symulacji w ciemniejszych warunkach sceny hali dystrybucyjnej. Na tylnej części wozu został podłączony wąż strażacki wraz z końcowym zaworem. Element ten stanowi integralną część systemu wodnego, którego szczegółowa implementacja została opisana w dalszych podrozdziałach pracy.



Rysunek 5.18. Model wozu strażackiego [Źródło własne].

Model transformatora [13] został zmodyfikowany na potrzeby symulatora w celu zwiększenia interakcji użytkownika z elementami infrastruktury. Adaptacja modelu obejmowała zarówno zmiany w geometrii obiektu, jak i jego przygotowanie do obsługi interakcji podczas rozgrywki w silniku Unity. Podstawowy model transformatora został pozostawiony w niezmienionej formie, zgodnie z oryginalnym modelem. Następnie została utworzona druga część transformatorowni. W powielonym modelu usunięto dwie przeciwległe ściany. Jedna z nich została zastąpiona jednolitą ścianą bez otworów wentylacyjnych, co lepiej odpowiada końcowej części obiektu transformatora. Druga ściana została całkowicie usunięta, aby możliwe było połączenie obu segmentów w jedną, większą bryłę. W celu uniknięcia nakładania się geometrii skrócono dach dołączanego elementu. Dzięki tym modyfikacjom oba segmenty wizualnie tworzą jeden spójny obiekt, mimo że technicznie składają się z dwóch połączonych modeli.

Kolejną zmianą była również modyfikacja drzwi transformatora. Z oryginalnego modelu usunięto statyczny element drzwi, a następnie w silniku dodano własny model drzwi z przygotowaną teksturą zawierającą oznaczenie „UWAGA WYSOKIE NAPIĘCIE TRANSFORMATOR”. Zabieg ten był konieczny, aby drzwi mogły być otwierane przez gracza. Mechanizm ich działania został zrealizowany analogicznie do drzwi zastosowanych w modelu wozu strażackiego.

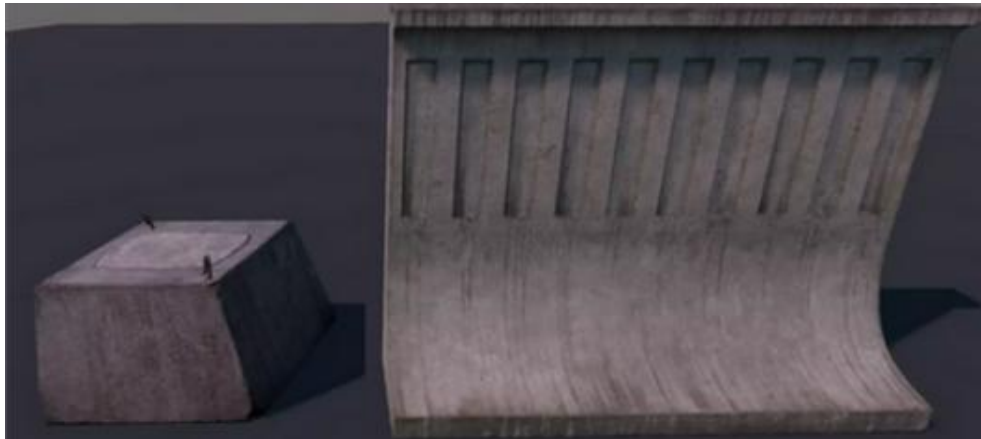
Wnętrze transformatorowni zostało przygotowane z uwzględnieniem realizmu wizualnego. Zastosowano kolorystykę nawiązującą do wyglądu zewnętrznego obiektu, uzupełnioną o detale w postaci zabrudzeń. Przy suficie umieszczono dwie lampy zapewniające podstawowe oświetlenie wnętrza oraz element oświetlenia awaryjnego. Na ścianie bocznej umieszczono przyciski oraz elementy interfejsu użytkownika informujące o stanie działania transformatora, które zostało powiązane z systemem elektrycznym i zostaną opisane w dalszej części pracy.



Rysunek 5.19. Model transformatorowni [Źródło własne].

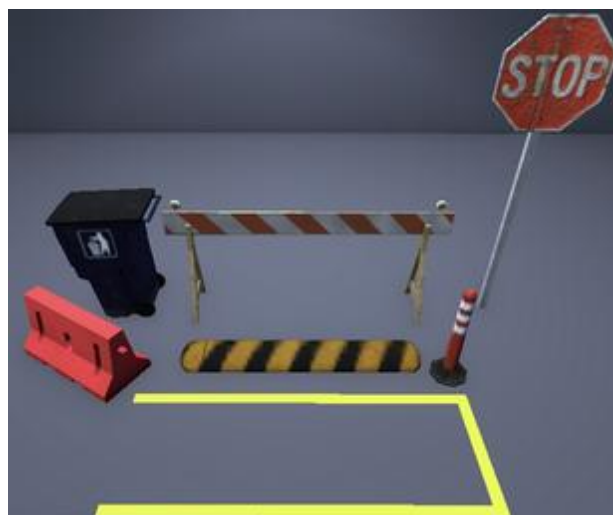
W celu ograniczenia widoczności obszarów znajdujących się poza właściwą sceną hali dystrybucyjnej, w otoczeniu obiektu zastosowano betonowe ogrodzenie [14]. Element ten pełni przede wszystkim funkcję wizualną i graniczną, maskując pustą przestrzeń środowiska gry oraz eliminując efekt „otwartego świata”, który mógłby negatywnie wpływać na odbiór realizmu symulacji.

Model ogrodzenia został przygotowany w stylistyce surowego betonu, z widocznymi śladami zabrudzeń i zużycia materiału, co odpowiada przemysłowemu charakterowi otoczenia hali dystrybucyjnej. Wysokość ogrodzenia została dobrana w taki sposób, aby skutecznie zasłaniać horyzont. Dodatkowo na terenie sceny rozmieszczono betonowe bloki, które pełnią funkcję uzupełniającą dla otoczenia zewnętrznego hali. Ich obecność zwiększa realizm przestrzeni przemysłowej, nadając jej bardziej autentyczny charakter. Bloki te stanowią elementy statyczne, które pełnią rolę małych przeszkód.



Rysunek 5.20. Model betonowego ogrodzenia oraz bloku [Źródło własne].

Kolejną grupę obiektów wykorzystanych w scenie hali dystrybucyjnej stanowią elementy dekoracyjne z infrastruktury drogowej, których zadaniem jest uwiarygodnienie przestrzeni przemysłowej symulacji. Modele te zostały dobrane wybiórczo z dostępnej paczki zasobów [15], tak aby odpowiadały charakterowi obiektu. Do sceny dodano między innymi kosz na śmieci, plastikową barierę drogową, próg spowalniający, słupek ostrzegawczy, znak drogowy STOP, żółty próg zwalniający, który został zmodyfikowany poprzez zmianę materiału oraz spłaszczenie geometrii. Zabieg ten pozwolił uzyskać lekko wypukły pas, wykorzystywany jako oznaczenie miejsca parkingowego na terenie hali. Ostatnim elementem tej grupy jest drewniana bariera, zastosowana w celu zainscenizowania zablokowanego wjazdu na teren hali dystrybucyjnej.



Rysunek 5.21. Modele rekwizytów ulicznych [Źródło własne].

Kolejnym elementem są modele dłoni [16] użytkownika w środowisku VR. Zostały dopracowane pod względem kolorystycznym tak, aby odpowiadały wyglądowi ciemnych rękawic ochronnych. Każda dłoń jest w pełni zrigowana i animowana, co umożliwia

realistyczne odwzorowanie ruchów wykonywanych przez użytkownika podczas korzystania z kontrolerów VR. W celu zapewnienia poprawnej interakcji z obiektami w środowisku symulacji, do każdej kości dłoni zostały przypisane kolidery, które umożliwiają wykrywanie kolizji z elementami otoczenia. Takie rozwiązanie pozwala na fizyczne reagowanie dłoni na kontakt z obiektami.

W modelach dłoni zaimplementowano dwie podstawowe animacje, powiązane bezpośrednio z przyciskami kontrolera VR. Pierwsza animacja odtwarzana jest w momencie naciśnięcia przycisku trigger, co odpowiada częściowemu zaciśnięciu dłoni. Druga animacja, przedstawiająca pełne zaciśnięcie dłoni w pięść, uruchamiana jest po naciśnięciu przycisku grip i odpowiada chwytaniu przedmiotów znajdujących się w otoczeniu.



Rysunek 5.22. Modele dłoni [Źródło własne].

Ostatnim i jednocześnie najbardziej rozbudowanym modelem wykorzystanym w symulacji jest model hali dystrybucyjnej [17]. Został on pozyskany z obszernej paczki zasobów „Warehouse Environment”, która została pierwotnie przygotowana na silnik Unreal Engine 5. Paczka ta charakteryzuje się bardzo wysokim poziomem szczegółowości oraz dużą liczbą elementów składowych, obejmujących zarówno elementy konstrukcyjne, jak i pełne wyposażenie wnętrza hali.

W celu wykorzystania zasobów w projekcie opartym na silniku Unity konieczne było zastosowanie dedykowanego narzędzia eksportującego [18], umożliwiającego przeniesienie modeli, materiałów oraz tekstur z Unreal Engine 5 do środowiska Unity. Sam proces eksportu nie był jednak wystarczający do uzyskania w pełni funkcjonalnej i spójnej sceny. Po zaimportowaniu zasobów wymagane było ręczne rozmieszczenie wszystkich elementów na scenie.

W wnętrzu hali rozmieszczono m.in. regały magazynowe, palety z ładunkiem, wózki widłowe, stoły robocze, kontenery, skrzynie oraz elementy infrastruktury technicznej. Układ wnętrza został zaprojektowany w taki sposób, aby odzwierciedlał rzeczywiste warunki panujące w obiektach dystrybucyjnych.

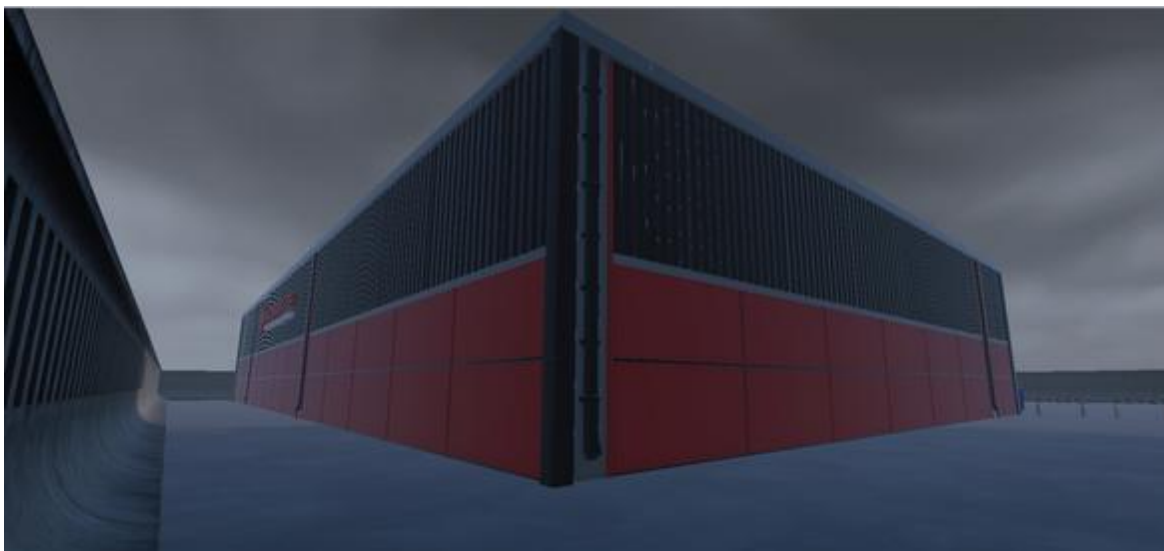
Kolejnym krokiem było ułożenie bryły zewnętrznej hali, wejść technicznych, bram załadunkowych oraz otoczenia obiektu, co pozwoliło na płynne poruszanie użytkownika pomiędzy przestrzenią zewnętrzną a wnętrzem hali. Całość została uzupełniona o wcześniej opisane elementy środowiskowe. Zagadnienia związane z optymalizacją wydajnościową oraz dostosowaniem sceny do pracy w środowisku VR zostały omówione w dalszej części pracy.



Rysunek 5.23. Widok hali z frontowego lewego narożnika sceny [Źródło własne].



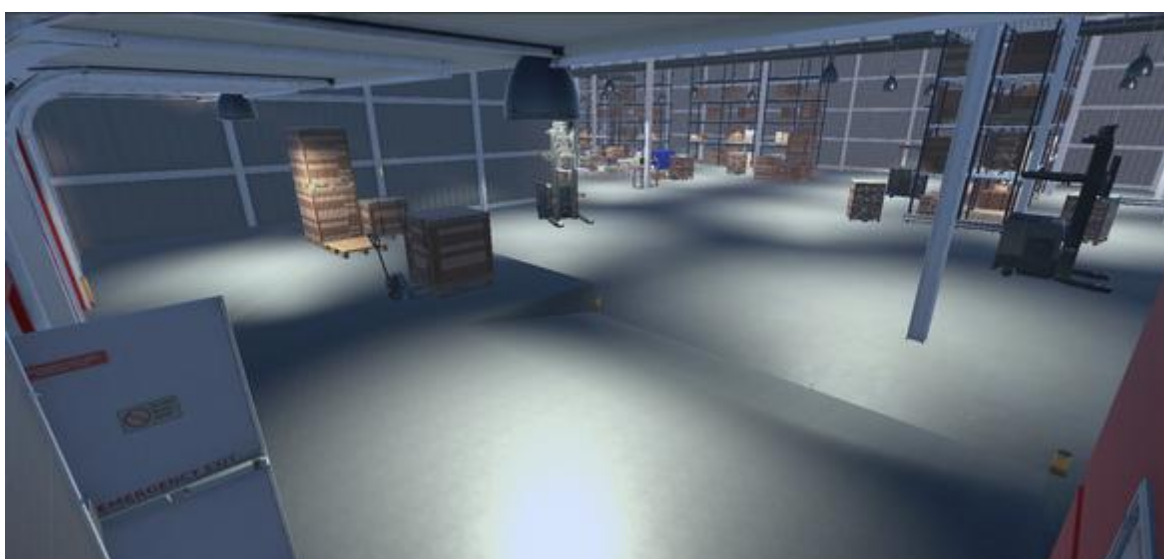
Rysunek 5.24. Widok hali z perspektywy bramy wjazdowej [Źródło własne].



Rysunek 5.25. Widok hali z tylnego lewego narożnika sceny [Źródło własne].



Rysunek 5.26. Widok hali z tylnego prawego narożnika sceny [Źródło własne].



Rysunek 5.27. Widok z wnętrza hali nad głównym wejściem/wyjściem [Źródło własne].



Rysunek 5.28. Widok z wnętrza hali z górnego prawego rogu [Źródło własne].



Rysunek 5.29. Widok z wnętrza hali na infrastrukturę techniczną [Źródło własne].

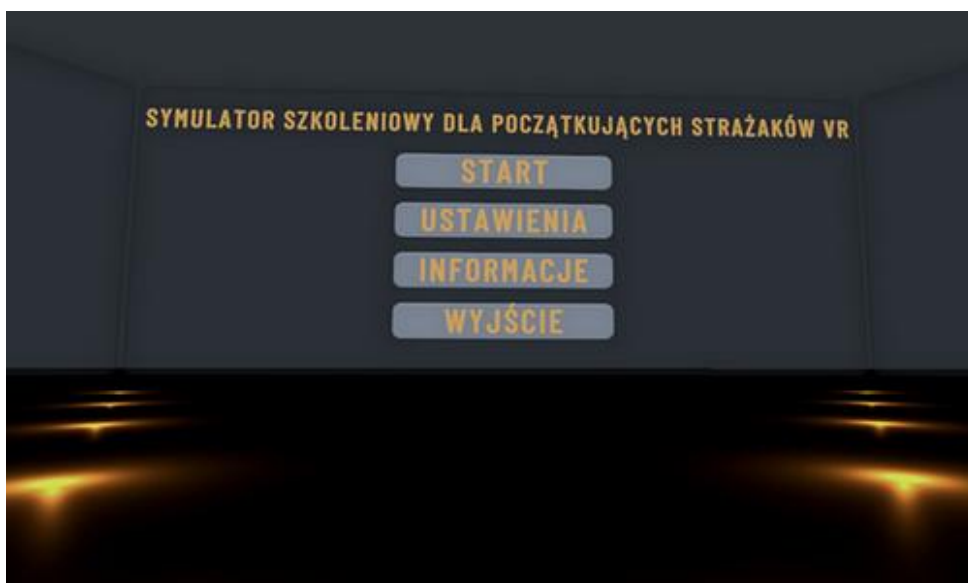


Rysunek 5.30. Widok z wnętrza hali na główne wejście/wyjście [Źródło własne].

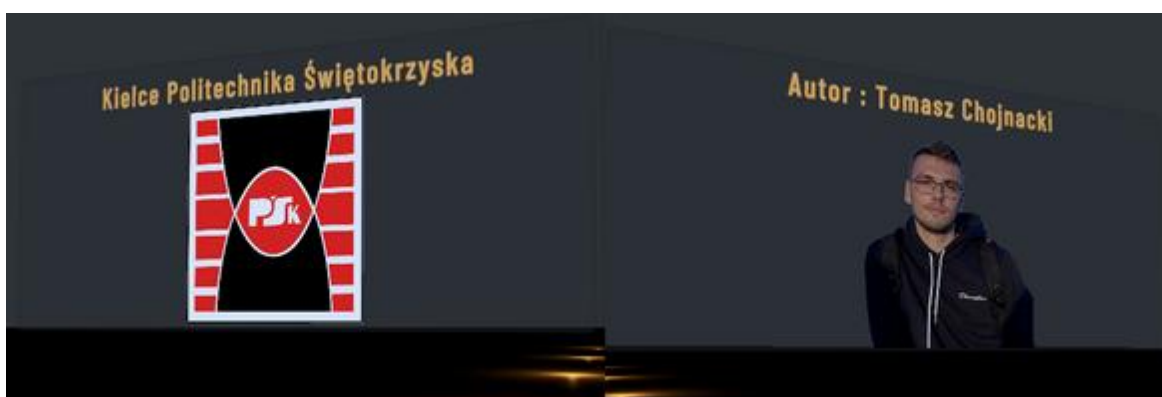
5.4. Główny interfejs użytkownika VR

Główny interfejs użytkownika w symulatorze stanowi menu VR, uruchamiane w osobnej scenie „Menu”. Interfejs został zaprojektowany w formie dużego, czytelnego panelu umieszczonego przed użytkownikiem, co ułatwia obsługę w goglach oraz minimalizuje potrzebę precyzyjnego celowania w małe elementy UI. Menu zawiera tytuł aplikacji oraz cztery podstawowe przyciski rozstawione w kolumnie: START, USTAWIENIA, INFORMACJE oraz WYJŚCIE.

Scena menu została dodatkowo zaaranżowana w sposób wzmacniający klimat symulatora. Po lewej stronie użytkownika znajduje się ekran z logo Politechniki Świętokrzyskiej, natomiast po prawej ekran zawierający podpis autora oraz zdjęcie. W przestrzeni sceny, za użytkownikiem, ustawiono dwa wozy strażackie zwrócone frontem w kierunku odpowiednich ekranów, a pomiędzy pojazdami umieszczono postać strażaka wykonującą animację machania ręką.



Rysunek 5.31. Wygląd panela Menu [Źródło własne].



Rysunek 5.32. Wygląd lewego i prawego panela [Źródło własne].



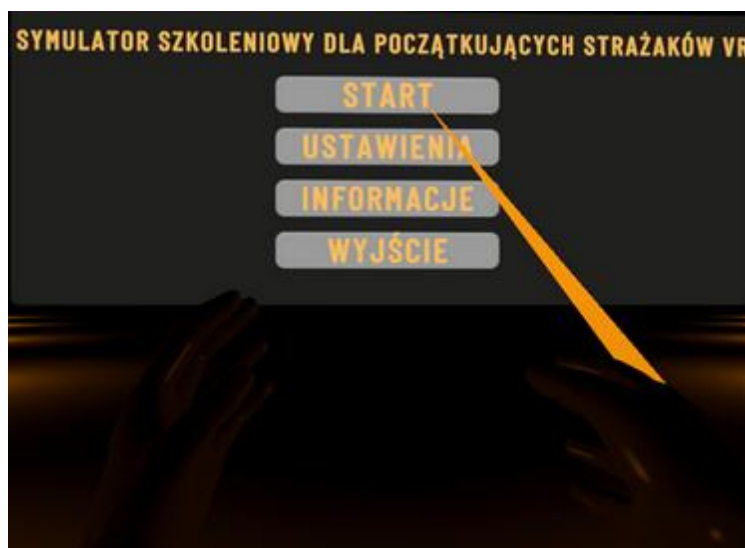
Rysunek 5.33. Wygląd otoczenia za użytkownikiem w menu [Źródło własne].

5.4.1. Interakcja z menu w VR

Obsługa menu realizowana jest za pomocą mechaniki wskazywania elementów UI promieniem z kontrolera. Użytkownik celuje w przyciski, a następnie zatwierdza wybór przyciskiem trigger na kontrolerze. Dodatkowym elementem, który został zaimplementowany do działania wskaźnika jest wyłączanie promienia wtedy, gdy nie trafia w element interfejsu. Dzięki temu w trakcie swobodnego rozglądania się po scenie użytkownik nie widzi ciągle aktywnej linii ray, a interakcja pojawia się dopiero w momencie faktycznego wejścia w tryb obsługi UI.

Mechanizm włączania/wyłączania wizualizacji ray został zaimplementowany w skrypcie PokazRayNaUI, który sprawdza, czy XRRayInteractor aktualnie wskazuje element UI, a następnie przełącza widoczność komponentu linii (XRInteractorLineVisual).

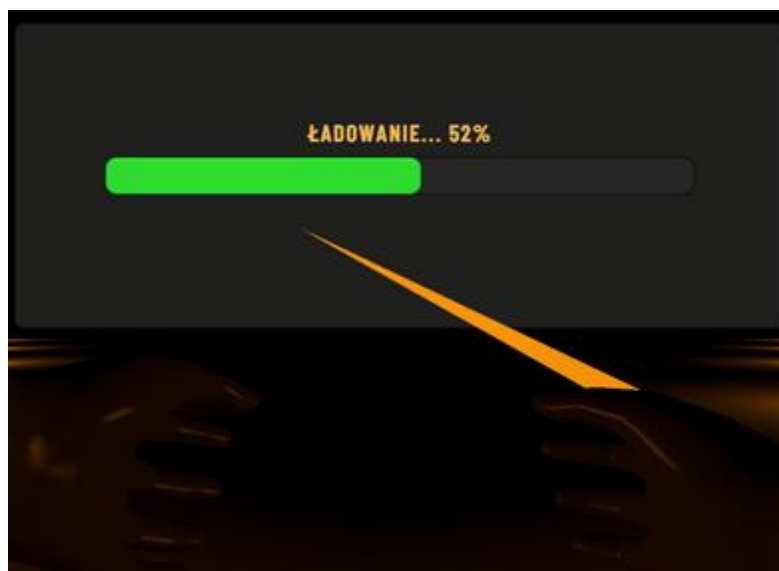
Po najechaniu wskaźnikiem na panel lub przycisk, system natychmiast udostępnia informację zwrotną w postaci widocznego promienia, a kliknięcie Triggerem powoduje aktywację wskazanego przycisku.



Rysunek 5.34. Wygląd najechania dłonią na UI Menu [Źródło własne].

5.4.2. Uruchamianie symulacji i ekran ładowania

Po potwierdzeniu zapoznania się z informacjami i kliknięciu ponownie przycisku START aplikacja ładuje scenę symulacji. Przejście zostało wzbogacone o ekran ładowania z paskiem postępu oraz tekstem procentowym. Funkcjonalność ta jest realizowana skryptem `SceneLoaderWithProgress`, który aktywuje panel ładowania i uruchamia asynchroniczne wczytywanie sceny (`LoadSceneAsync`). Takie rozwiązanie powoduje prawidłowe wczytanie wszystkich modeli, materiałów oraz tekstur. Skrypt dodatkowo wymusza minimalny czas wyświetlania ekranu ładowania, dzięki czemu pasek postępu nie przeskakuje natychmiastowo i użytkownik otrzymuje czytelną informację o stanie procentowym załadowanej symulacji.



Rysunek 5.35. Wygląd paska ładowania [Źródło własne].

5.4.3. Okno ustawień

W oknie USTAWIENIA dostępne są dwa obszary konfiguracji:

1. Grafika – realizowane poprzez `TMP_Dropdown`, który odzwierciedla aktualny poziom jakości ustawiony w Unity i pozwala na jego zmianę w trakcie działania aplikacji. Skrypt Ustawienia pobiera bieżący poziom jakości (`QualitySettings.GetQualityLevel()`), ustawia odpowiednią wartość w rozwijanej liście i reaguje na zmianę opcji poprzez `QualitySettings.SetQualityLevel(...)`.

Przykład implementacji (fragment skryptu Ustawienia):

```
int currentQuality = QualitySettings.GetQualityLevel();
private void ChangeLevel(int value){
    QualitySettings.SetQualityLevel(value, true);}

```

2. Obracanie – dostępne są dwa warianty: obracanie skokowe oraz obracanie ciągle. Zmiana ustawienia jest natychmiastowa i obowiązuje w trakcie bieżącej sesji uruchomienia aplikacji. Mechanizm zrealizowano w skrypcie UstawieniaObrotu, który przełącza aktywność odpowiednich skryptów obsługujących obrót oraz przechowuje wybór w statycznej zmiennej globalTrybObrotu.

Przykład implementacji (fragment skryptu UstawieniaObrotu):

```
private static int globalTrybObrotu = 0;
public void OnDropdownZmianaTrybu(int index)
{
    globalTrybObrotu = index;
    UstawTrybObrotu(index);
}
```



Rysunek 5.36. Wygląd okna ustawień [Źródło własne].

5.4.4. Okno informacji

Przycisk INFORMACJE prowadzi do panelu, w którym użytkownik wybiera temat, a następnie jest przenoszony do dedykowanego ekranu opisowego. Zastosowano zestaw przycisków odpowiadających kluczowym elementom symulatora: PROJEKT, CHODZENIE, CHWYTANIE, OBRACANIE, TELEPORTACJA, WĄŻ STRAŻACKI, POWIETRZE, ZDROWIE, SYSTEM PRĄDU. Na dole panelu znajduje się przycisk POWRÓT DO MENU, umożliwiający natychmiastowy powrót do ekranu głównego. Każde przejście pomiędzy ekranami UI w całym menu zostało wykonane dzięki funkcji On Click(Run time Only GameObject.SetActive). Użycie takiego rozwiązania uprościło całą procedurę implementacji do włączania i wyłączania poszczególnych okien po kliknięciu w odpowiedni przycisk.



Rysunek 5.37. Wygląd okna informacji wraz z wszystkimi informacjami [Źródło własne].

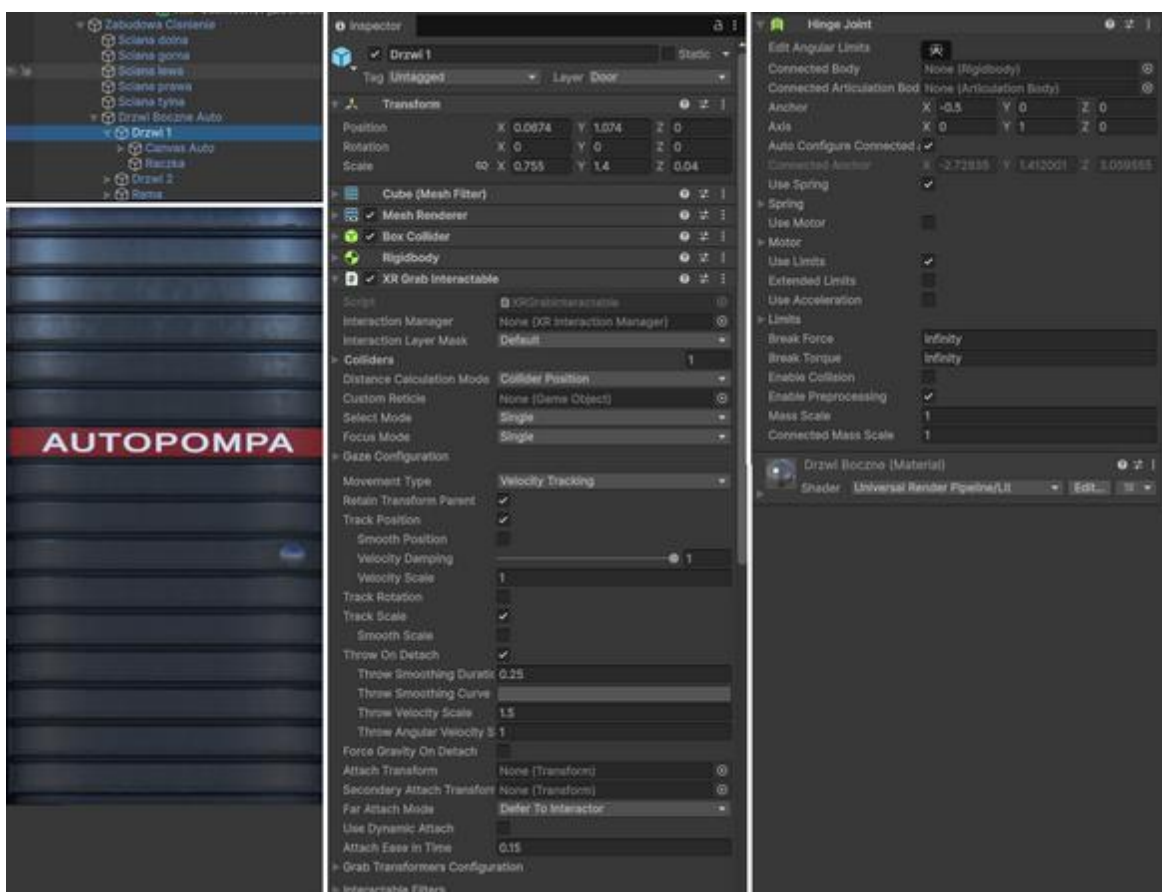
5.4.5. Zamykanie aplikacji

Ostatnią funkcją menu jest przycisk WYJŚCIE, którego zadaniem jest zamknięcie aplikacji. Implementacja została zrealizowana w skrypcie ZamykanieGry, który wywołuje Application.Quit(). Dodana została też obsługa pracy w edytorze w celu zatrzymania aplikacji w trybie Play (UnityEditor.EditorApplication.isPlaying). Zamykanie aplikacji z poziomu edytora przyczyniło się do optymalnej pracy przy testowaniu nowych funkcjonalności.

5.5. System autopompy

System autopompy w symulatorze został zaprojektowany jako mechanizm włączania i wyłączania ciśnienia wody, który warunkuje możliwość uruchomienia strumienia z zaworu końcowego węża. Na początku rozgrywki ciśnienie jest domyślnie ustawione jako wyłączone (OFF). Użytkownik może je aktywować dopiero po wykonaniu sekwencji określonych czynności.

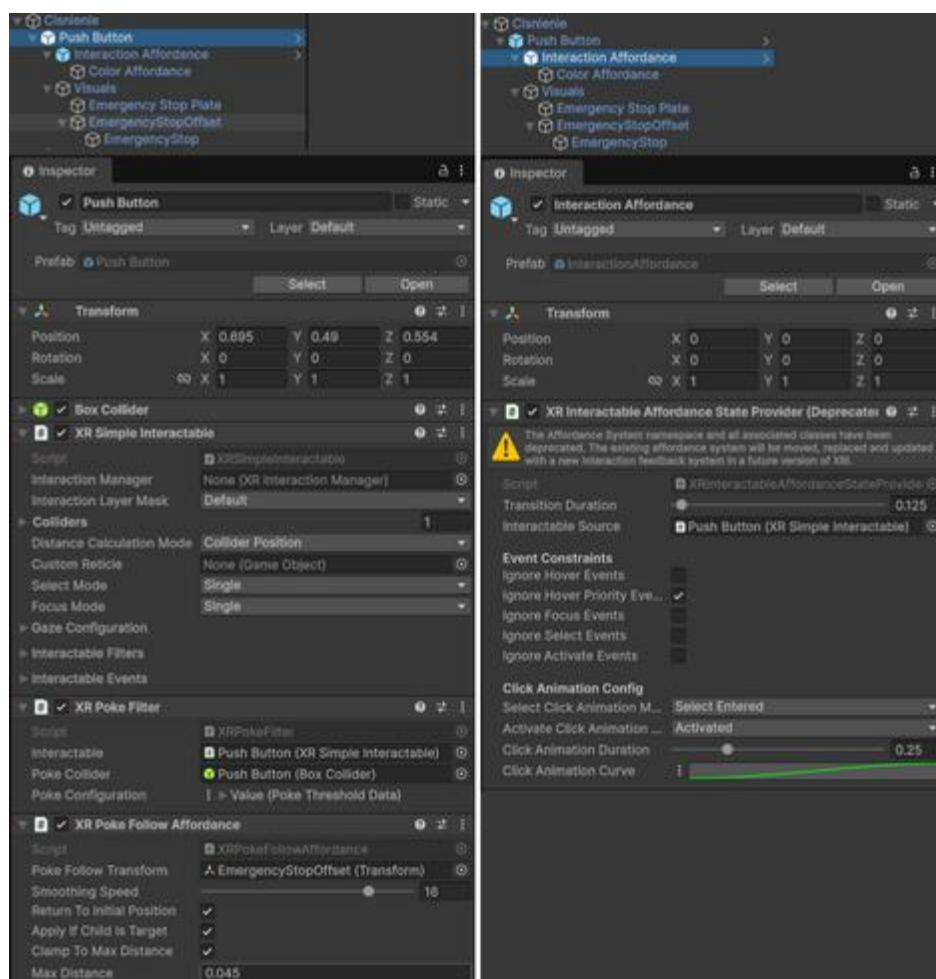
Pierwszym krokiem jest otwarcie podwójnych drzwi skrytki bocznej pojazdu strażackiego. Dopiero po otwarciu drzwi widoczny staje się panel ciśnienia z przyciskiem oraz monitorem informacyjnym. W momencie, gdy ciśnienie jest wyłączone, monitor prezentuje komunikat „CIŚNIENIE OFF”. Po aktywacji mechanizmu komunikat zmienia się na „ON”, co informuje użytkownika o gotowości układu do podawania wody.



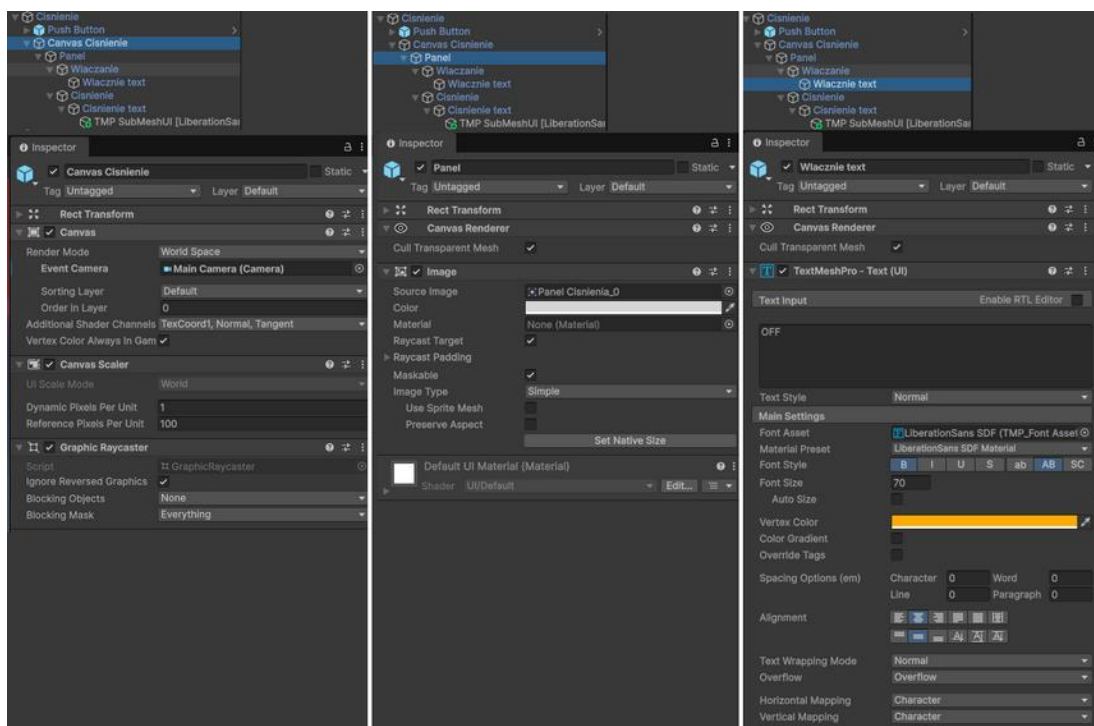
Rysunek 5.38. Drzwi boczne autopompy obiekt interaktywny (XR Grab Interactable) z fizyką (Rigidbody) oraz zawiasem (Hinge Joint) umożliwiającym otwieranie [Źródło własne].



Rysunek 5.39. Panel autopompy po otwarciu drzwi widok monitora z komunikatem stanu ciśnienia [Źródło własne].



Rysunek 5.40. Przycisk panelu ciśnienia XRSimpleInteractable z konfiguracją interakcji typu poke (XR Poke Filter) i elementami affordance [Źródło własne].



Rysunek 5.41. Canvas panelu ciśnienia w przestrzeni świata (World Space) z tekstem OFF/ON (TextMeshPro) [Źródło własne].

Włączanie/wyłączanie ciśnienia zostało zrealizowane w postaci skryptu PanelCisnienia sterującego stanem oraz synchronizującego go z UI i logiką strumienia wody. Naciśnięcie przycisku jest obsługiwane zdarzeniem `selectEntered` z komponentu `XRSimpleInteractable`, a następnie stan jest propagowany do modułu odpowiedzialnego za strumień.

Przykład implementacji (fragment skryptu PanelCisnienia):

```
private void OnEnable(){
    if (przycisk != null)
        przycisk.selectEntered.AddListener(OnPressed);
}

private void OnDisable(){
    if (przycisk != null)
        przycisk.selectEntered.RemoveListener(OnPressed);
}

private void OnPressed(SelectEnterEventArgs args){
    UstawStan(!isOn);
}

private void UstawStan(bool nowyStan){
    isOn = nowyStan;

    if (napisOnOff != null)
        napisOnOff.text = isOn ? "ON" : "OFF";

    if (strumienWody != null)
        strumienWody.CisnienieWlaczne = isOn;
}
```

Dzięki temu panel spełnia dwie role jednocześnie:

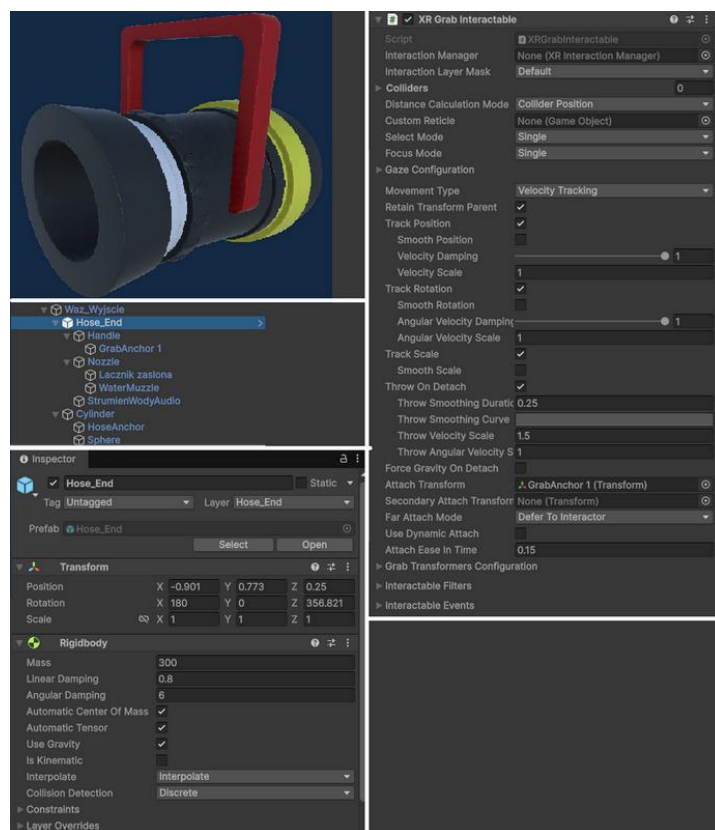
- dostarcza natychmiastowej informacji zwrotnej (zmiana „OFF/ON”),
- pełni funkcję przełącznika stanu układu wodnego, bezpośrednio blokując lub dopuszczając emisję wody z zaworu.

Uzupełnieniem całości jest warstwa audio. Dźwięk lania wody został nagrany za pomocą telefonu, a następnie obrobiony w programie Audacity w taki sposób, aby zapętlenie nie powodowało słyszalnych „przeskoków” i nie generowało negatywnego odczucia podczas długotrwałego użycia.

5.5.1. Implementacja zachowania węża strażackiego

Wąż strażacki w symulacji został przygotowany jako obiekt o zachowaniu fizycznym, który zachowuje się elastycznie, reaguje na ruch użytkownika w VR, umożliwia interakcję z końcowym zaworem poprzez chwytnanie.

Końcówka węża (zawór) jest obiektem możliwym do podniesienia posiada Rigidbody oraz XR Grab Interactable, co pozwala na chwytnanie przyciskiem grip zgodnie z mapowaniem XRI.



Rysunek 5.42. Końcówka węża (Hose_End) Rigidbody oraz XR Grab Interactable umożliwiające podniesienie i utrzymanie zaworu [Źródło własne].

Modelowanie zachowania elastycznego zrealizowano poprzez zbudowanie łańcucha punktów (segmentów) pomiędzy początkiem (start) i końcem (end). Dla każdego punktu tworzony jest obiekt z Rigidbody, a kolejne segmenty są łączone sprężynami (SpringJoint). Parametry sprężyn (siła, tłumienie, miękka odległość) są konfigurowane z poziomu Inspektora.

Przykład implementacji (fragment skryptu WazStrazacki):

```
private void ZbudujPunkty(){
    punkty = new Transform[numberOfInnerPoints + 2];
    rbPunkty = new Rigidbody[punkty.Length];
    punkty[0] = start;
    punkty[punkty.Length - 1] = end;
    for (int i = 1; i < punkty.Length - 1; i++){
        var p = Instantiate(pointPrefab, transform);
        p.name = $"Part_{i}_Point";
        punkty[i] = p.transform;

        var rb = p.GetComponent<Rigidbody>();
        rb.isKinematic = false;
        rbPunkty[i] = rb;
    }
}

private void ZbudujLaczniki(){
    laczniki = new SpringJoint[punkty.Length - 1];
    for (int i = 0; i < laczniki.Length; i++){
        var sj = punkty[i].gameObject.AddComponent<SpringJoint>();
        sj.connectedBody = rbPunkty[i + 1];
        laczniki[i] = sj;
    }
}
```

Ważnym elementem dostosowania pod VR jest stabilizacja końcówki (end). W praktyce, szybkie ruchy kontrolerem mogą powodować zaburzenia fizyki i nienaturalne szarpnięcia. Zastosowano więc ograniczenie prędkości liniowej i kątowej końcówki węża.

Przykład implementacji (fragment skryptu WazStrazacki):

```
var v = rbEnd.linearVelocity;
if (v.magnitude > maxPredkoscEnd)
    rbEnd.linearVelocity = v.normalized * maxPredkoscEnd;

var w = rbEnd.angularVelocity;
if (w.magnitude > maxPredkoscKatowaEnd)
    rbEnd.angularVelocity = w.normalized * maxPredkoscKatowaEnd;
```

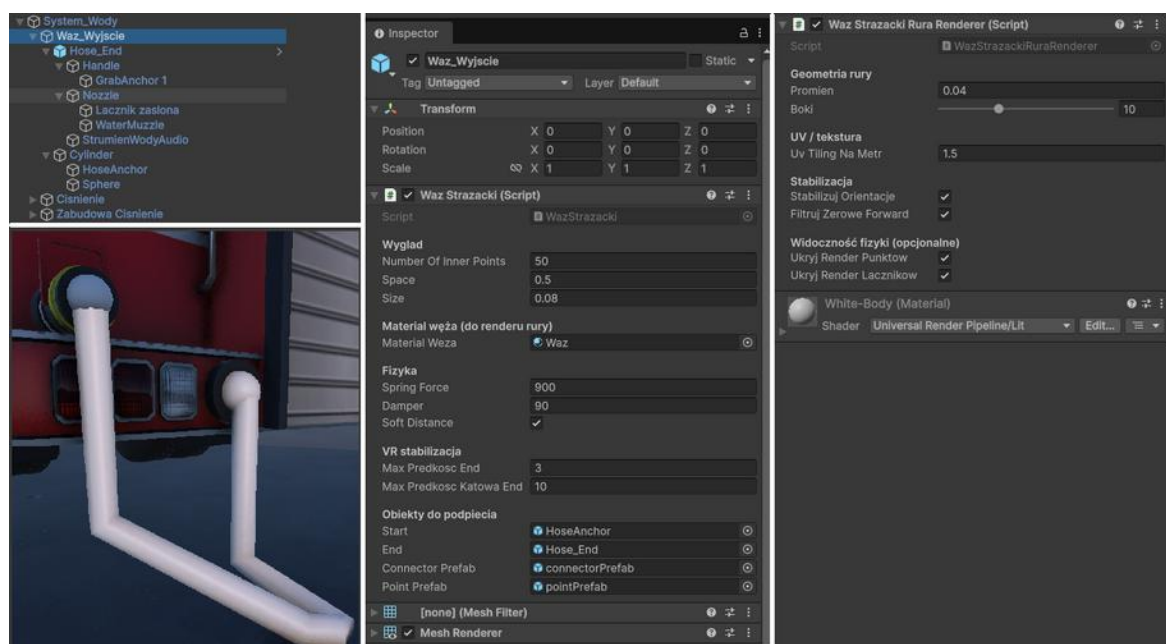
Wizualizacja węża została rozwiązana niezależnie od fizyki jako rura generowana proceduralnie wzdłuż punktów. Renderer tworzy siatkę (mesh) w formie rurki o zadanym promieniu i liczbie boków oraz mapuje UV w funkcji długości, co pozwala na poprawne powtarzanie tekstury.

Przykład implementacji (fragment skryptu WazStrazackiRuraRenderer):

```
[SerializeField] private float promien = 0.04f;
[SerializeField, Range(3, 32)] private int boki = 10;
[SerializeField] private float uvTilingNaMetr = 1.5f;

public void UstawPunkty(Transform[] punkty){
    this.punkty = punkty;
    mesh.Clear();
}
```

W całości użyciu użytkownik otrzymuje wąż, który ciągnie się za zaworem i reaguje na ruch podczas trzymania w dłoni zaworu, zachowuje się stabilnie w VR, a także charakteryzuje się ciężkością i sprężystością przy naprężeniach wynikających z zbyt dalekiego rozciągnięcia. Dodatkowo wyglądem jest bardzo zbliżony do rzeczywistego węża strażackiego dzięki ciągłej geometrii wizualnej nałożonej na elementy konstrukcyjne.



Rysunek 5.43. Wąż strażacki oraz ustawienia WazStrazacki i WazStrazackiRuraRenderer [Źródło własne].

5.5.2. System cząstek wody z zaworu końcowego

System wizualizacji wypływu wody z zaworu końcowego węża strażackiego został zrealizowany z użyciem dwóch niezależnych systemów cząstek (Particle System): strumienia wody (Water, Water_Particles) oraz efektu rozbryzgu (Splash). Takie rozdzielanie pozwala jednocześnie uzyskać wiarygodny wygląd ciągłego wypływu oraz poprawnie reagować na kontakt strumienia z powierzchniami sceny.

Główny strumień stanowi prefab `waterPrefab`, który jest inicjowany w momencie uruchomienia lania wody. Emisja cząstek odbywa się w osi wylotu (muzzle), dzięki czemu efekt jest zawsze poprawnie ustawiony względem orientacji zaworu. Do uniknięcia przenikania wody przez ściany i inne elementy zostało dodane dynamiczne skracanie widocznego strumienia. Zrealizowano to poprzez raycast wykonywany od punktu wylotu w kierunku strumienia. Jeżeli raycast wykryje trafienie, długość widocznego strumienia jest redukowana do odległości trafienia (z niewielkim marginesem), a w miejscu kontaktu generowany jest rozbryzg.

W praktyce długość strumienia jest sterowana nie skalowaniem obiektu, lecz parametrem lifetime cząstek, im krótszy czas życia, tym krótszy odcinek strumienia. Dzięki temu efekt pozostaje dobrze widoczny.

Przykład implementacji (fragment `StrumienWody`):

```
private void UstawDlugoscStrumienia(float odleglosc)
{
    odleglosc = Mathf.Clamp(odleglosc, minimalnyZasiegWizualny,
maksymalnyZasiegWizualny);

    var main = waterInstance.main;
    float speed = main.startSpeed.constant;
    if (speed < 0.01f) speed = domyslnaPredkosc;

    float lifetime = odleglosc / speed;
    lifetime = Mathf.Clamp(lifetime, 0.02f, 10f);

    main.startLifetime = lifetime;
}
```

Rozbryzg jest realizowany osobnym prefabem `splashPrefab`. Efekt jest ustawiany w miejscu trafienia, z rotacją dopasowaną do normalnej powierzchni, co sprawia, że rozbryzg układa się prawidłowo zarówno na podłodze, jak i na ścianach.

Przykład implementacji (fragment `StrumienWody`):

```
splashInstance.transform.position = pozycja;
splashInstance.transform.rotation = Quaternion.LookRotation(hit.normal);

if (!splashInstance.isPlaying){
    splashInstance.Clear(true);
    splashInstance.Play(true);
}
```

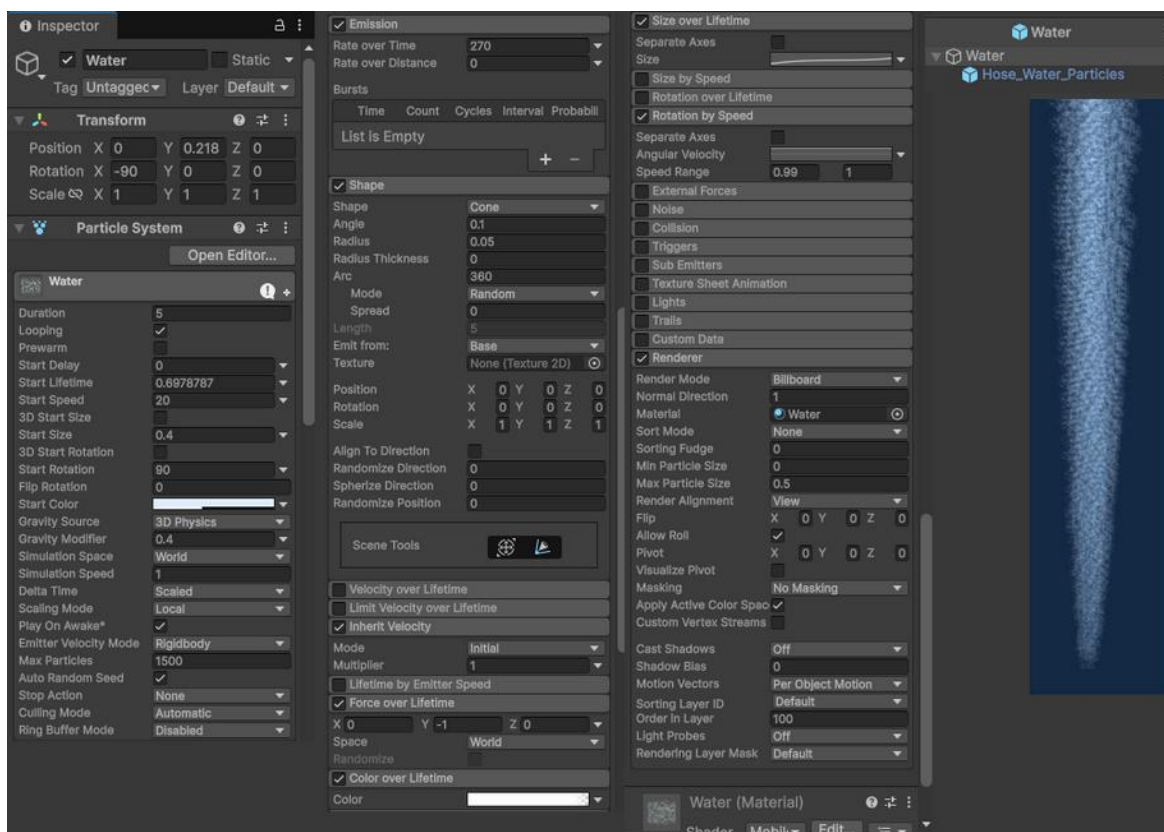
Zastosowano również mechanizm stabilizacji, który ogranicza ciągle restartowanie rozbryzgu przy minimalnych zmianach punktu trafienia. W praktyce rozbryzg jest

odświeżany dopiero wtedy, gdy punkt trafienia zmieni się bardziej niż ustalony próg, co poprawia płynność oraz zmniejsza efekt migotania.

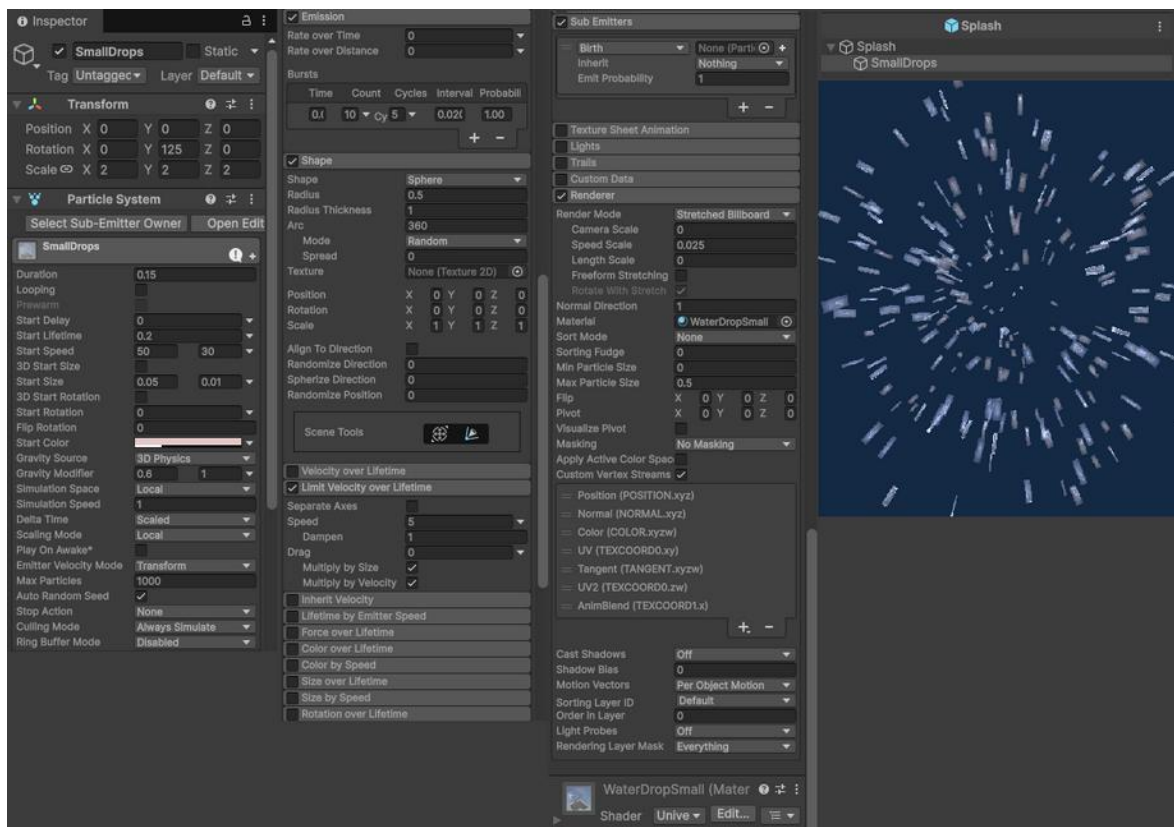
Przykład implementacji (fragment StrumienWody):

```
if (!mamHit || (p - ostatniHitPoint).sqrMagnitude > (progRuchuHit * progRuchuHit)){
    ostatniHitPoint = p;
    mamHit = true;
    UstawSplash(hit, p);
}
else{
    if (splashInstance && !splashInstance.isPlaying)
        splashInstance.Play(true);
}
```

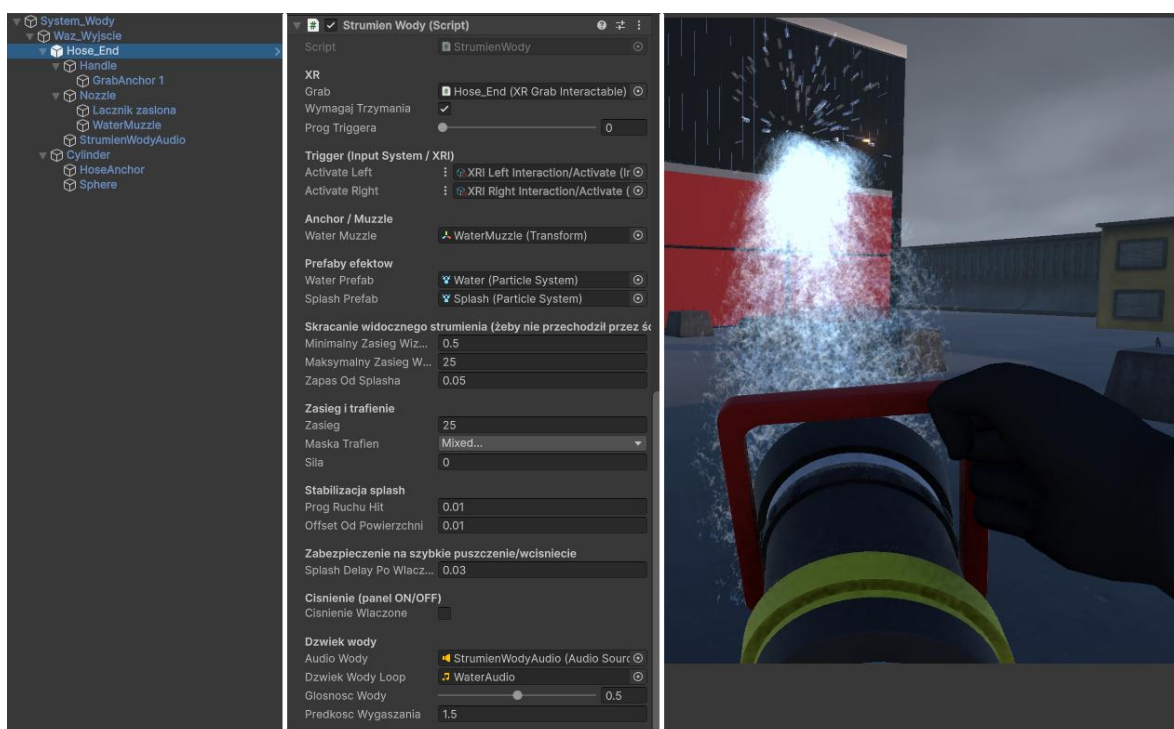
Dodatkowo wprowadzono krótki czas blokady rozbryzgu po włączeniu wody, co eliminuje niepożądany efekt przy szybkim kliknięciu triggera oraz przy uruchamianiu strumienia.



Rysunek 5.44. Konfiguracja Particle System (Water) oraz parametry emisji i kształtu strumienia [Źródło własne].



Rysunek 5.45. Konfiguracja Particle System (Splash) parametry rozbryzgu [Źródło własne].



Rysunek 5.46. Widok działania strumienia w scenie oraz inspektor komponentu StrumienWody z przypisanymi prefabami i parametrami zasięgu strumienia [Źródło własne].

5.6. System powietrza

System powietrza został zaimplementowany jako mechanizm kontrolujący dwa parametry stanu użytkownika: powietrze oraz zdrowie. Wartości te są aktualizowane w czasie rzeczywistym w metodzie Update(), co zapewnia aktualizowanie stanu podczas całej symulacji. W momencie uruchomienia sceny system inicjalizuje parametry startowe użytkownika, żeby w każdym podejściu do scenariusza parametry zdrowia i powietrza były pełne. Dodatkowo po załadowaniu sceny ustawiany jest stan „bez aparatu powietrznego” oraz konfiguracja źródła dźwięku gwizdka ostrzegawczego. Następnie stan jest wyświetlany na panelu UI, który jest dostępny po spojrzeniu od góry na lewą dłoń.

Jednym z elementów logiki jest rozróżnienie dwóch trybów pracy: przebywanie w strefie zagrożenia oraz poza strefą. W strefie następuje spadek powietrza (szybszy bez aparatu, wolniejszy z aparatem), a gdy powietrze spadnie do zera rozpoczyna się spadek zdrowia aż do zakończenia symulacji.

Przykład implementacji (fragment PlayerTlenZdrowie):

```
if (wStrefie){
    float rate = maAparat ? spadekZAparatem : spadekBezAparatu;
    powietrze = Mathf.Max(0f, powietrze - rate * dt);
    if (powietrze <= 0f){
        zdrowie = Mathf.Max(0f, zdrowie - spadekZdrowia * dt);
        if (zdrowie <= 0f){
            ZakonczGreIZaladujScene();
            return;
        }
    }
}
```

Poza strefą zagrożenia system realizuje regenerację powietrza, ale wyłącznie wtedy, gdy gracz nie ma założonego aparatu. W przypadku posiadania aparatu logika nie dodaje powietrza automatycznie, ponieważ stan butli pozostaje taki, jaki wynika ze zużycia.

Przykład implementacji (fragment PlayerTlenZdrowie):

```
else{
    if (!maAparat)
        powietrze = Mathf.Min(maxPowietrze, powietrze + regenPowietrza * dt);
}
```

System przewiduje również zakończenie gry poprzez automatyczne załadowanie sceny Przegrana po spadku zdrowia do zera.

5.6.1. Panel stanu zdrowia i powietrza

Panel stanu zdrowia i powietrza jest oparty o dwa komponenty typu Slider: jeden dla powietrza (powietrzeBar), drugi dla zdrowia (zdrowieBar). Wartości te są od 0 do 1, co upraszcza zarządzanie UI oraz zapewnia czytelność dla użytkownika. Aktywacja panelu odbywa się przy użyciu wektora z „Gaze intreactora”, który w momencie kolizji z kulą w „Main Camera” wyświetla panel stanu zdrowia i powietrza.

Aktualizacja UI została wydzielona do metody `OdswiezUI()`. W niej suwaki są ustawiane na zakres od 0 do 1.

Przykład implementacji (fragment `PlayerTlenZdrowie`):

```
private void OdswiezUI()
{
    if (powietrzeBar){
        powietrzeBar.minValue = 0f;
        powietrzeBar.maxValue = 1f;
        powietrzeBar.value = Powietrze01;
    }
    if (zdrowieBar){
        zdrowieBar.minValue = 0f;
        zdrowieBar.maxValue = 1f;
        zdrowieBar.value = Zdrowie01;
    }
}
```

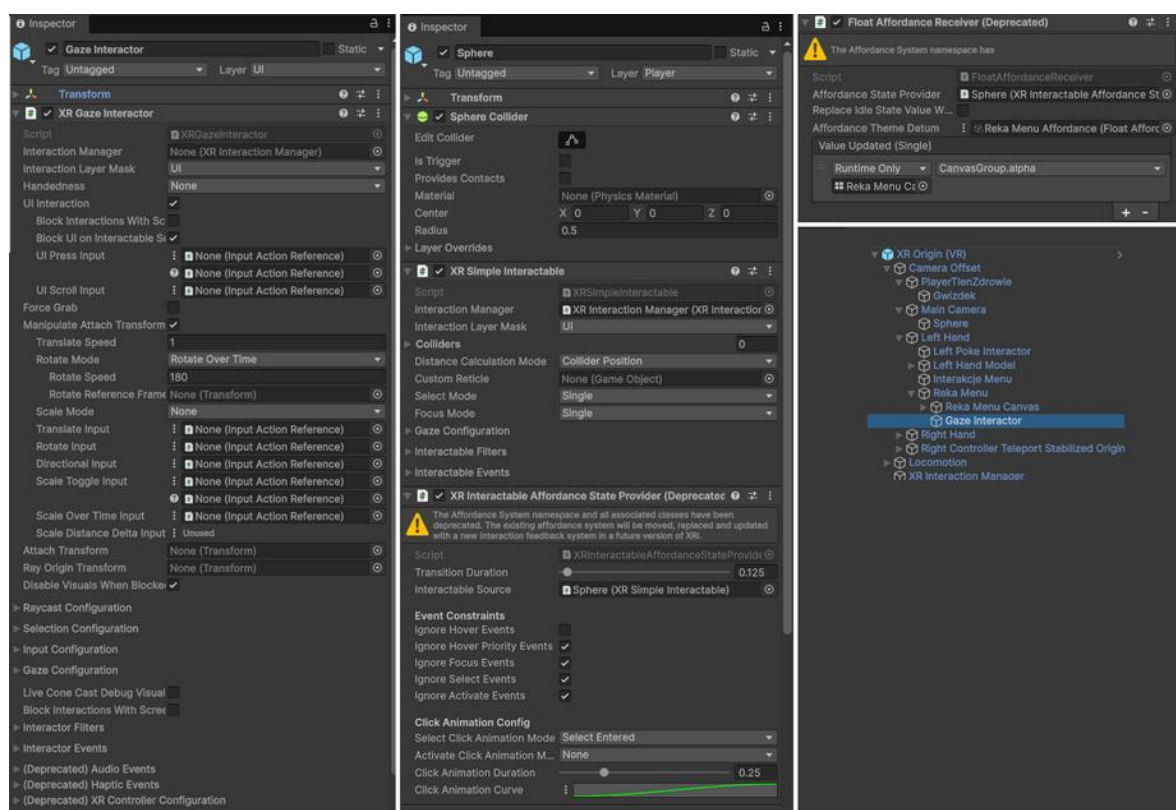
Panel zawiera dodatkowo informację tekstową realizowaną jako dwa obiekty `GameObject`: jeden dla stanu „założony aparat”, drugi dla stanu „brak aparatu”. Aktywacja odpowiedniej informacji odbywa się przez `SetActive()` w metodzie `OdswiezIkony()`. Ustawienie trybu aparatu jest za pomocą metody `UstawTrybAparatu()`, która natychmiast aktualizuje panel.

Przykład implementacji (fragment `PlayerTlenZdrowie`):

```
public void UstawTrybAparatu(bool czyMaAparat)
{
    maAparat = czyMaAparat;
    OdswiezIkony();
}
```



Rysunek 5.47. Wygląd panelu na lewej dłoni oraz jego działanie zakładania aparatu [Źródło własne].



Rysunek 5.48. Konfiguracja Gaze Interactor [Źródło własne].

Każda strefa zagrożenia została zrealizowana jako osobny obiekt sceny umieszczony w hali dystrybucyjnej, wyposażony w komponent Collider z aktywną opcją Is Trigger. Dzięki temu możliwe jest wykrywanie wejścia oraz wyjścia użytkownika z obszaru zagrożenia bez fizycznej kolizji.



Rysunek 5.49. Konfiguracja stref zagrożenia [Źródło własne].

5.6.2. Aparaty powietrzne

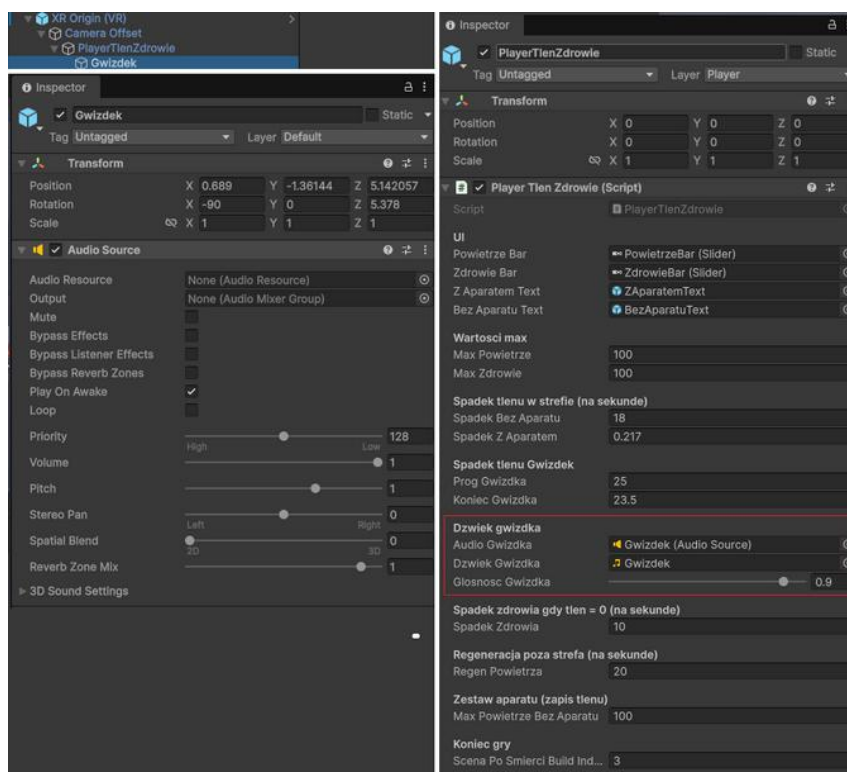
Aparaty powietrzne jako modele 3D zostały wiernie odwzorowane w programie blender na wzór sprzętu strażackiego [19]. Natomiast w symulatorze zostały zaimplementowane jako dwa niezależne zestawy (Zestaw 1 oraz Zestaw 2), które użytkownik może zakładać i zdejmować w trakcie rozgrywki [Rysunek 5.45]. Każdy zestaw posiada własną butlę, której stan jest zapisywany. Dzięki temu po ponownym założeniu tego samego zestawu gracz nie otrzymuje „pełnej” butli, lecz kontynuuje zużycie od poprzedniej wartości.

W projekcie przyjęto, że butla ma pojemność 100 bar. Zużycie powietrza przy założonym aparacie jest realizowane jako stały spadek w czasie przebywania w strefie zagrożenia. W skrypcie PlayerTlenZdrowie przyjęto parametr spadekZAparatem = 0.217 bar/s co odpowiada w przybliżeniu 7 minutom 40 sekundom pracy ($100 \text{ bar} / 0.217 \text{ bar/s} \approx 460 \text{ s} \approx 7 \text{ min } 40 \text{ s}$).

Stan aktualnego zestawu jest zapisywany na bieżąco, dzięki czemu poziom powietrza w butli pozostaje aktualny nawet przy nagłej zmianie sytuacji (np. opuszczeniu strefy zagrożenia lub zdjęciu aparatu). Zakładanie aparatu powoduje przełączenie systemu gracza w tryb „z aparatem”, natomiast zdejmowanie zapisuje aktualny stan butli i przywraca tryb „bez aparatu”.

Wybór zestawu realizowany jest poprzez interaktywne przyciski XR umieszczone na pojeździe ratowniczym. Każdy przycisk odpowiada konkretnemu zestawowi i wykorzystuje mechanizm `XRSimpleInteractable`. Po założeniu aparatu odpowiadający mu obiekt 3D zostaje ukryty w scenie, a po zdjęciu ponownie się pojawia, co zapewnia czytelną informację interakcji ze sprzętem przez użytkownika.

W symulatorze zastosowano dźwiękowy sygnał ostrzegawczy (gwizdek), który uruchamia się wyłącznie wtedy, gdy: gracz ma założony aparat, znajduje się w strefie zagrożenia, poziom powietrza w butli spadnie do przedziału 25 bar – początek ostrzegania, 23.5 bar – koniec przedziału ostrzegania. Przedział pomiędzy tymi wartościami daje przybliżony czas działania gwizdka, około 7 sekund ciągłego ostrzegania. W tym celu dźwięk został nagrany telefonem, a następnie przerobiony w programie Audacity w celu przyjemniejszego dla ucha zapętlania.



Rysunek 5.50. Konfiguracja dźwięku gwizdka [Źródło własne].

Źródło dźwięku gwizdka zostało wykonane programowo w metodzie `UstawGwizdek()`. Dźwięk jest uruchamiany i zatrzymywany automatycznie w momencie ostrzegania dzięki zmianie `gwizdekAktywny`. Działanie to zapobiegło zbędnemu obciążeniu systemu audio oraz nakładaniu się dźwięków na siebie.

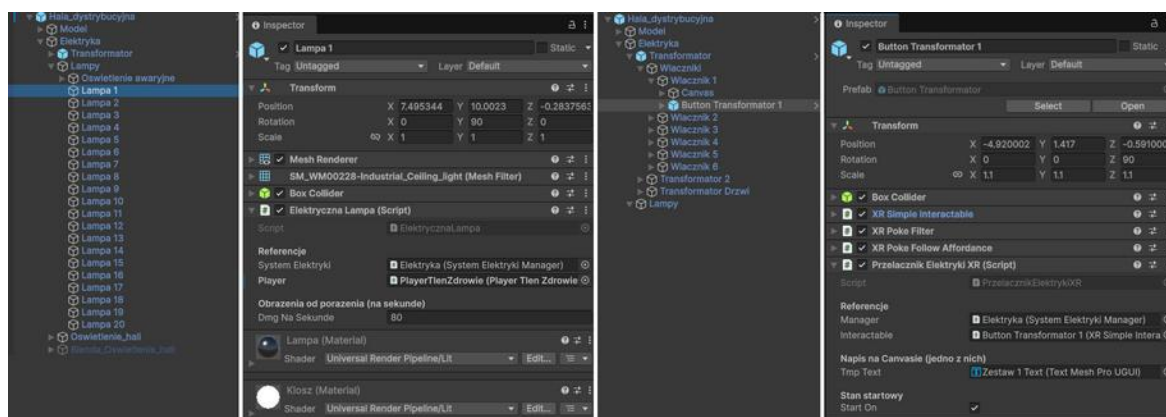
5.7. System prądu

System prądu został zaimplementowany w symulatorze jako mechanizm odpowiedzialny za kontrolę zasilania hali dystrybucyjnej oraz za uświadomienie użytkownika o konsekwencji polewania wodą urządzeń będących pod napięciem.

Centralnym elementem systemu jest skrypt `SystemElektrykiManager`, który przechowuje globalny stan zasilania w scenie. Skrypt ten analizuje stan wszystkich przełączników prądu rozmieszczonych w transformatorowni i na tej podstawie określa, czy prąd w hali jest włączony/wyłączony. Zasilanie pozostaje aktywne, jeżeli przynajmniej jeden z przełączników znajduje się w pozycji włączonej. Zmiana stanu zasilania wpływa bezpośrednio na system oświetlenia. Przy włączonym prądzie aktywne jest główne oświetlenie hali, natomiast po jego odcięciu automatycznie uruchamiane jest oświetlenie awaryjne.

Każdy z sześciu przełączników prądu w scenie posiada przypisany skrypt `PrzelacznikElektrykiXR`, który umożliwia jego obsługę w środowisku VR. Przełączniki są elementami interaktywnymi i reagują na akcję użytkownika wykonaną przy użyciu kontrolerów [Rysunek 5.19].

Każda lampa znajdująca się w hali posiada skrypt `ElektrycznaLampa`, który odpowiada za reakcję obiektu na kontakt z wodą. Skrypt ten jest połączony zarówno z menedżerem systemu elektrycznego, jak i z systemem zdrowia użytkownika. W momencie, gdy gracz poleje wodą lampę przy włączonym zasilaniu, dochodzi do symulacji porażenia prądem, co skutkuje odjęciem 80 punktów zdrowia w czasie rzeczywistym. Obrażenia naliczane są wyłącznie wtedy, gdy instalacja elektryczna pozostaje pod napięciem. Po odłączeniu prądu polewanie lamp wodą nie powoduje już utraty zdrowia.



Rysunek 5.51. Konfiguracja lampy oraz przycisku [Źródło własne].



Rysunek 5.52. Wygląd włączonego/wyłączonego oświetlenia [Źródło własne].

5.8. System ognia

System ognia został zaimplementowany jako centralny mechanizm zadaniowy symulacji. Opiera się na współpracy trzech głównych skryptów: SystemOgniaManager, FirePoint oraz FireAudioManager, a także na wykorzystaniu efektów wizualnych VFX [20]. Po uruchomieniu sceny menedżer systemu ognia automatycznie wyszukuje w hierarchii obiekty pełniące rolę punktów pożaru. Są to puste obiekty z dołączonymi Box Colliderami, których nazwy rozpoczynają się od ustalonego prefiksu „punkt_ognia”. Do każdego z takich punktów dynamicznie przypisywany jest prefab efektu ognia VFX, co pozwala na łatwą rozbudowę sceny.

Każdy punkt ognia posiada komponent FirePoint, który odpowiada za logikę wygaszania. Gaszenie realizowane jest poprzez kolizję strumienia wody z koliderem ognia. W momencie wykrycia takiej kolizji intensywność pożaru zaczyna stopniowo maleć, co wizualnie objawia się zmniejszaniem skali efektu VFX. Po osiągnięciu minimalnego progu intensywności punkt ognia zostaje uznany za ugaszony, a jego obiekt jest usuwany ze sceny.

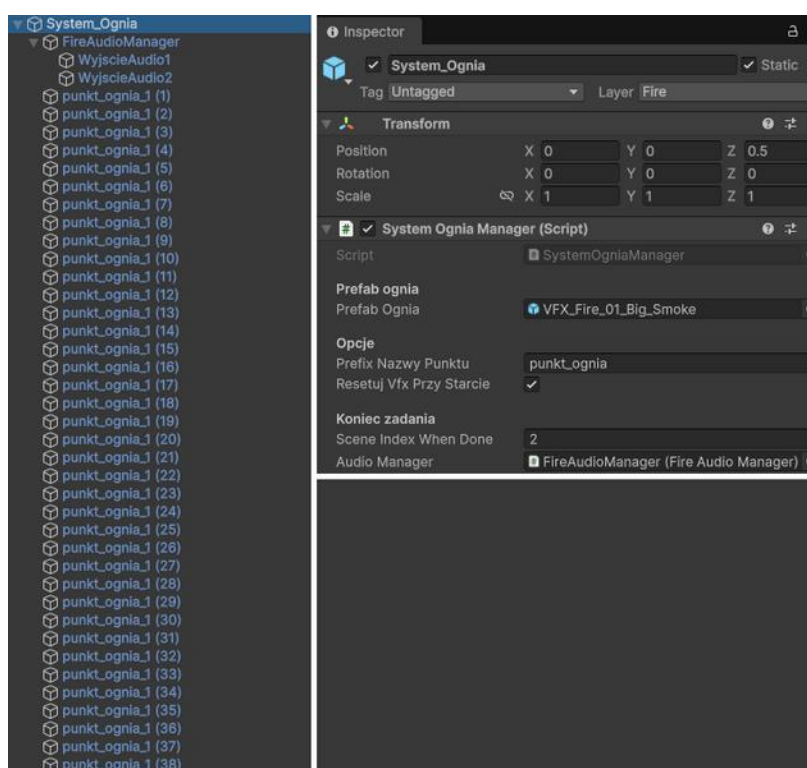
Dodatkowo zaimplementowano mechanizm ponownego rozpalenia ognia w sytuacji, gdy proces gaszenia nie zostanie doprowadzony do końca. Jeżeli strumień wody przestanie oddziaływać na punkt ognia przed osiągnięciem minimalnej intensywności, po upływie czasu intensywność efektu VFX wzrasta płynnie aż do osiągnięcia wartości bazowej. Wizualnie przedstawia się to jako ponowne rozpalenie i wymusza na użytkowniku sprawdzenie gaszonego punktu.

Przykład implementacji ponownego rozpalenia (fragment skryptu FirePoint):

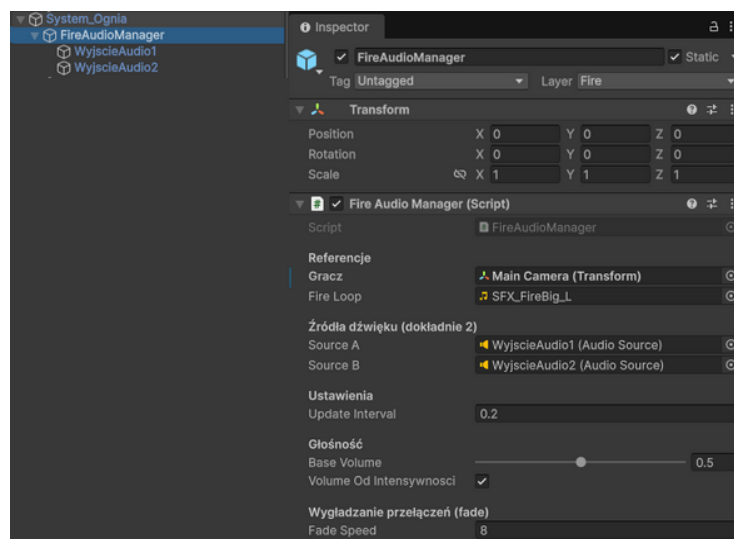
```
if (!jestGaszony && aktualnaIntensywnosc < maksymalnaIntensywnosc)
{
    aktualnaIntensywnosc += predkoscOdnowy * Time.deltaTime;
    aktualnaIntensywnosc = Mathf.Clamp(
        aktualnaIntensywnosc, minimalnaIntensywnosc, maksymalnaIntensywnosc);
    transform.localScale = Vector3.one * aktualnaIntensywnosc;
}
```

SystemOgniaManager prowadzi bieżące zliczanie aktywnych punktów pożaru. Po ugaszeniu ostatniego ognia automatycznie uruchamiany jest mechanizm zakończenia zadania, który przenosi gracza do sceny wygrana.

Efekty wizualne ognia nie były modyfikowane na poziomie parametrów graficznych. Wykorzystane zostały w formie domyślnej. Zmiany dotyczyły jedynie warstwy dźwiękowej. Aby uniknąć nakładania się wielu źródeł dźwięku w sytuacji, gdy w scenie występuje trzydzieści osiem punktów ognia, zaimplementowano dedykowany menedżer audio. Skrypt FireAudioManager dynamicznie wybiera dwa najbliższe graczowi punkty ognia i tylko dla nich aktywuje dźwięk płomieni, płynnie regulując głośność oraz pozycję źródła dźwięku na każde ucho.



Rysunek 5.53. Hierarchia punktów ognia [Źródło własne].



Rysunek 5.54. Ustawienia FireAudioManager [Źródło własne].



Rysunek 5.55. Punkty ognia rozmieszczone na scenie [Źródło własne].



Rysunek 5.56. Gaszenie punktu ognia [Źródło własne].

5.9. Sceny końcowe

W symulatorze zaimplementowano dwie odrębne sceny końcowe: scenę przegranej oraz scenę wygranej, które stanowią jednoznaczną informację zwrotną dla użytkownika o wyniku przeprowadzonej symulacji. Sceny te są ładowane automatycznie w zależności od stanu rozgrywki i pełnią zarówno funkcję informacyjną, jak i motywacyjną.

Scena przegranej uruchamiana jest w momencie, gdy poziom zdrowia gracza spadnie do zera, co może nastąpić m.in. w wyniku przebywania w strefie zagrożenia bez aparatu powietrznego lub po wyczerpaniu tlenu w butli i porażenia prądem. Warstwa wizualna sceny została utworzona w taki sposób, żeby podkreślić negatywny rezultat działań użytkownika. Strażacy w tle prezentują negatywne animacje. Implementacja przejścia do sceny przegranej:

Przykład implementacji (fragment skryptu `PlayerTlenZdrowie`):

```
if (zdrowie <= 0f){  
    SceneManager.LoadScene(scenaPoSmierciBuildIndex);  
}
```

Scena wygranej uruchamiana jest po ugaszeniu wszystkich aktywnych punktów ognia na scenie hali dystrybucyjnej. Jej wygląd jest zmieniony o elementy charakterystyczne względem sceny przegranej. W centrum panelu widnieje napis „WYGRANA” oraz przycisk powrotu do menu głównego. Strażacy w tle prezentują pozytywne animacje. Warstwa wizualna sceny została utworzona w taki sposób, aby podkreślić osiągnięty sukces działań użytkownika. Implementacja przejścia do sceny wygranej:

Przykład implementacji (fragment skryptu `SystemOgniaManager`):

```
if (aktywnychPunktowOgnia <= 0){  
    SceneManager.LoadScene(sceneIndexWhenDone);  
}
```

Dodatkowym elementem wzmacniającym przekaz wizualny sceny wygranej jest efekt konfetti, który symbolizuje sukces i nagrodę za poprawne wykonanie zadania. Zastosowany system konfetti pochodzi z pakietu XR Interaction Toolkit [8] i jest to standardowy efekt wykorzystywany w scenach demonstracyjnych tego pakietu. W ramach projektu nie modyfikowano logiki działania systemu cząstek. Jedyną wprowadzoną zmianą była korekta kolorystyki konfetti, aby lepiej dopasować je do stylistyki symulatora. Za czasowe sterowanie emisją konfetti odpowiada skrypt uruchamiający efekt na określony czas.

Przykład implementacji (fragment skryptu ConfettiTimer):

```
private void Start()
{
    particleSystem.Play();
    Invoke(nameof(StopConfetti), czasTrwania);
}

private void StopConfetti()
{
    particleSystem.Stop();
}
```



Rysunek 5.57. Widok sceny przegranej [Źródło własne].



Rysunek 5.58. Widok sceny wygranej [Źródło własne].

5.10. Optymalizacja i dostosowanie pod wydajność gogli VR

Optymalizacja aplikacji pod kątem pracy w goglach VR była jednym z długotrwałych etapów realizacji projektu, ponieważ środowiska wirtualnej rzeczywistości wymagają utrzymania stabilnej, wysokiej liczby klatek na sekundę oraz niskich opóźnień renderowania. W tym celu w projekcie zastosowano szereg działań optymalizacyjnych obejmujących tekstury, materiały, oświetlenie, cienie oraz jakość renderowania.

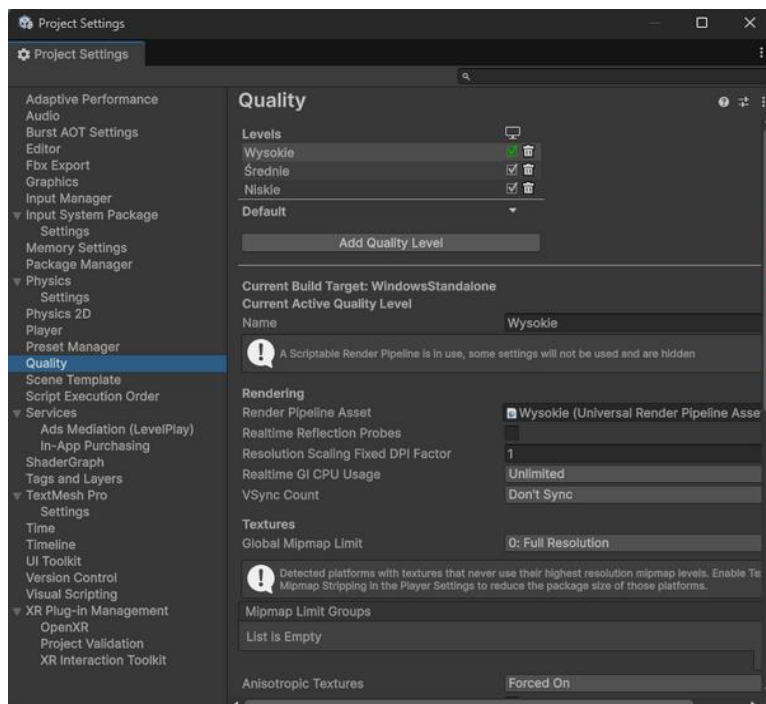
Wszystkie modele hali dystrybucyjnej zostały poddane optymalizacji tekstur poprzez ograniczenie maksymalnej rozdzielczości do 1024×1024 px.. Tekstury w rozdzielczości 4K, obecne w oryginalnych assetach, nie przynosiły zauważalnych korzyści wizualnych w goglach VR. Dodatkowo wszystkie materiały zaimportowane pierwotnie z silnika Unreal Engine 5 zostały uproszczone. Oryginalne materiały posiadały złożone systemy map metaliczności i parametrów PBR (Physically Based Rendering), które były kosztowne obliczeniowo. W projekcie utworzono nowe, uproszczone materiały URP, do których podpięto wyłącznie niezbędne tekstury (base map, normal map, occlusion map). Dzięki temu możliwe było oznaczenie większości obiektów hali jako Static, co pozwoliło silnikowi Unity na skuteczniejsze batchowanie.

W celu dalszej redukcji kosztów renderowania dla obiektów, których cienie nie są widoczne dla gracza (np. elementy techniczne, tyły obiektów), cienie zostały całkowicie wyłączone. Widoczne elementy sceny w trakcie symulacji dostały cienie wypiekane (Baked Shadows) zamiast dynamicznych. Został również ograniczony maksymalny dystans renderowania cieni oraz liczba kaskad w zależności od wybranego profilu jakości.

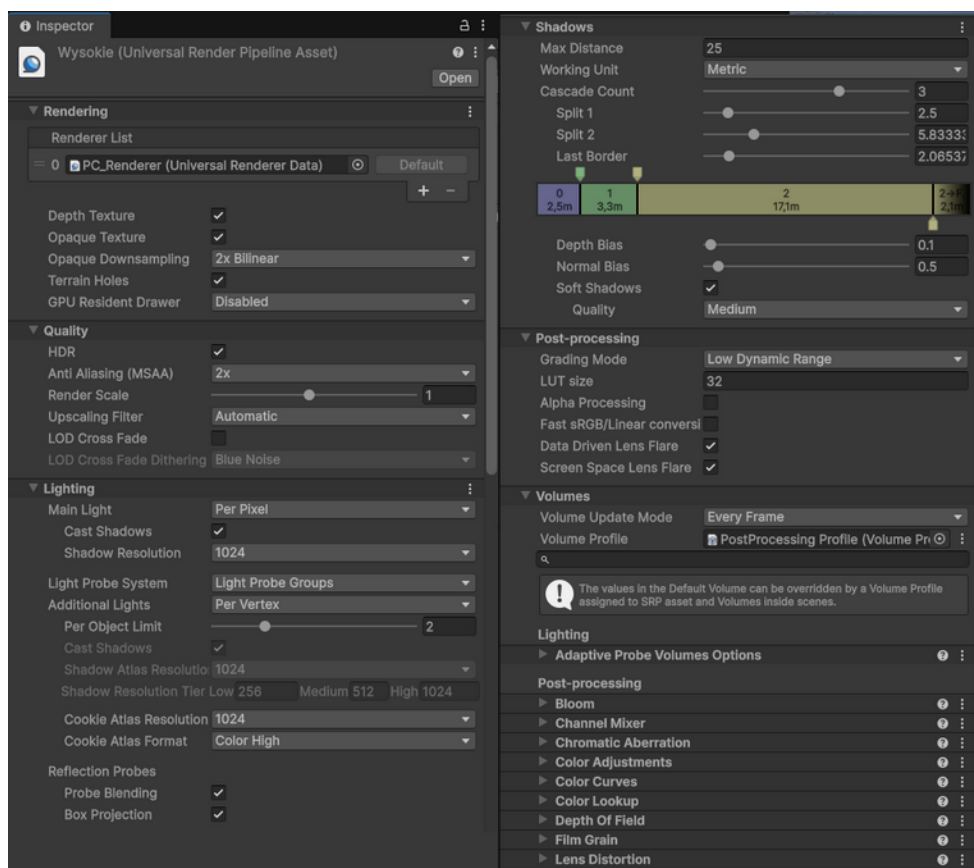
Projekt wykorzystuje trzy predefiniowane profile jakości dostępne w menu ustawień aplikacji:

- Wysokie – pełne cienie, większy dystans ich renderowania, HDR, post-processing,
- Średnie – ograniczona liczba kaskad cieni, uproszczony post-processing,
- Niskie – brak cieni, obniżona skala renderowania oraz minimalny zestaw efektów post-processingowych.

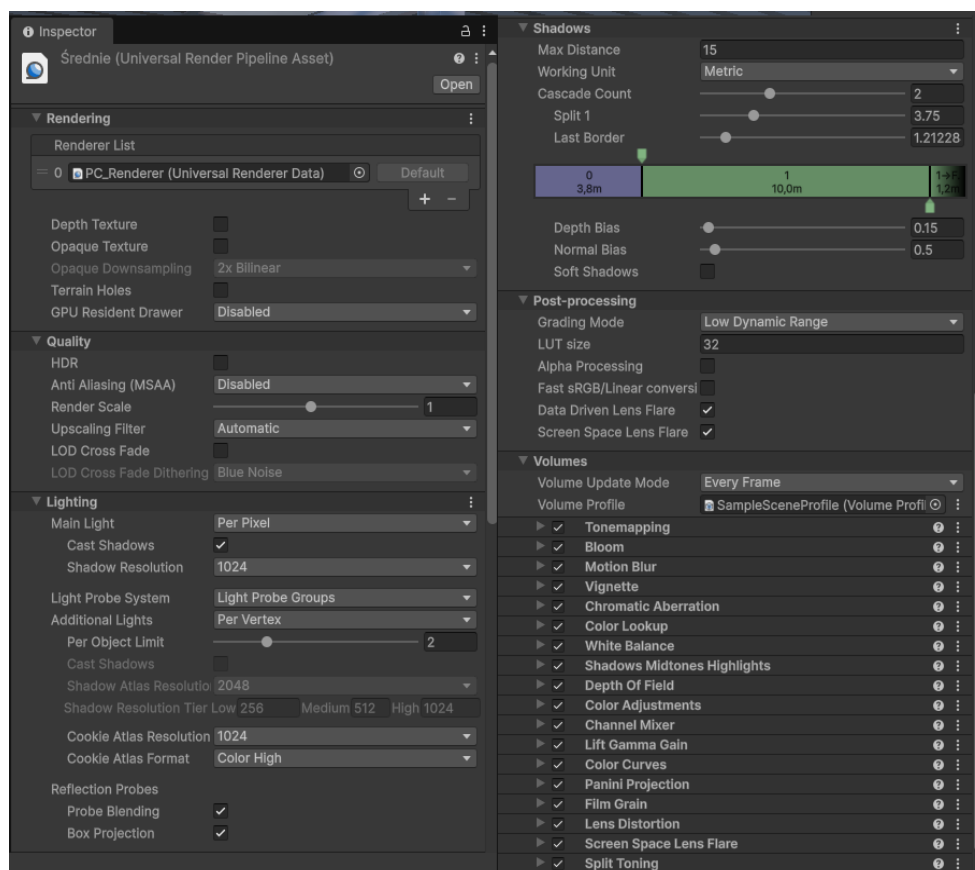
Profile oparte są na ustawieniach Universal Render Pipeline (URP) i pozwalają dostosować jakość grafiki do możliwości sprzętowych komputera użytkownika. Zmiana jakości odbywa się z poziomu interfejsu ustawień, bez konieczności restartu aplikacji.



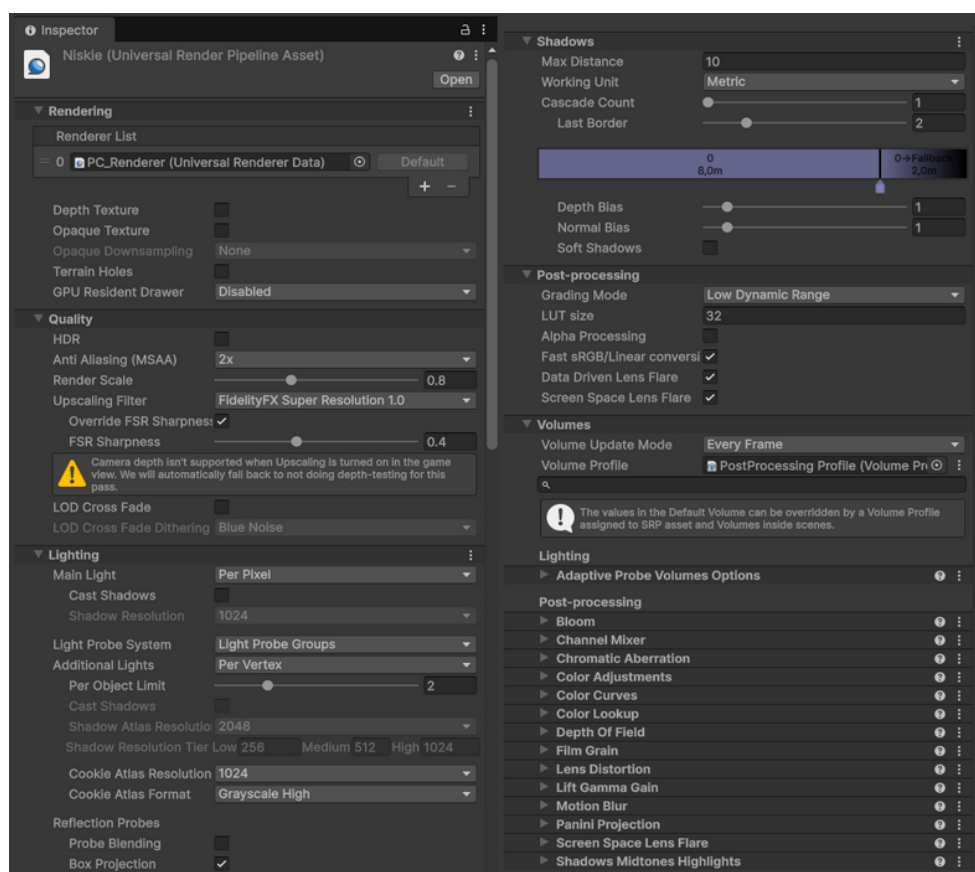
Rysunek 5.59. Widok profili jakości (Projekt Settings) [Źródło własne].



Rysunek 5.60. Ustawienia graficzne wysokie [Źródło własne].



Rysunek 5.61. Ustawienia graficzne średnie [Źródło własne].



Rysunek 5.62. Ustawienia graficzne niskie [Źródło własne].

6. TESTOWANIE SYMULATORA

Proces testowania symulatora został przeprowadzony dwuetapowo i obejmował zarówno testy wydajnościowe, jak i ocenę użytkowników. Pierwsza część testów miała na celu ocenę poprawności działania systemów symulacji oraz wydajności aplikacji w środowisku VR. Druga część polegała na testowaniu symulatora przez użytkowników, którzy następnie mieli odpowiedzieć na zestaw przygotowanych pytań.

Testy realizowano w warunkach najbardziej obciążających dla komputera, aby sprawdzić stabilność działania symulatora podczas renderowania dużej liczby złożonych obiektów. Szczególną uwagę zwrócono na płynność obrazu na goglach VR.

6.1. Testowanie funkcjonalne

Podczas testów funkcjonalnych szczególna uwaga została zwrócona na analizę wydajności symulatora przy różnych poziomach ustawień graficznych. Testy przeprowadzono na komputerze o specyfikacji sprzętowej:

- Procesor: 13th Gen Intel Core i7 – 13650HX (2.60 GHz),
- Pamięć RAM: 24,0 GB, 4800 MT/s,
- System operacyjny: Windows 11, 64-bit,
- Karta graficzna: NVIDIA GeForce RTX 5050 Laptop GPU.

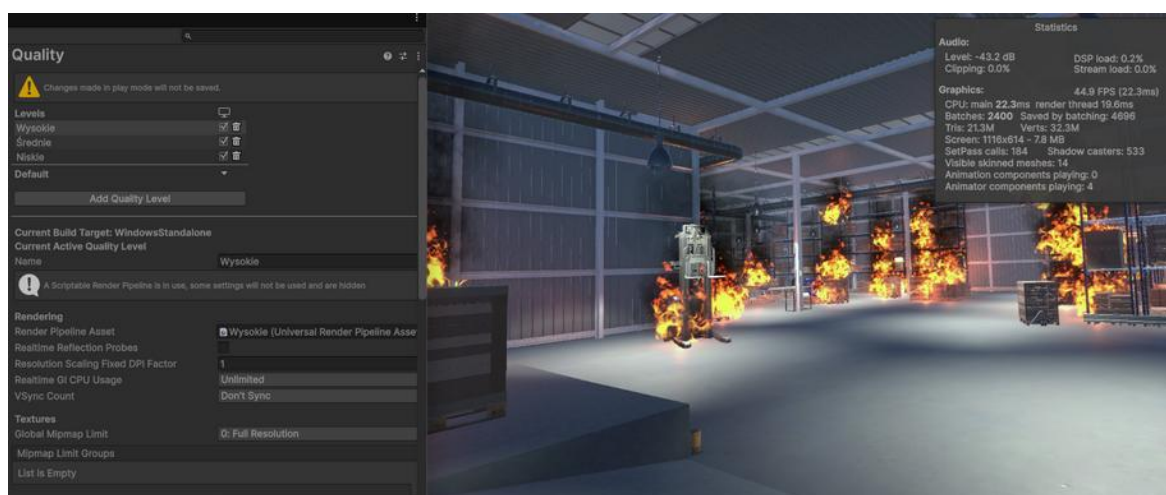
Testy wykonano z użyciem gogli VR, a pomiary liczby klatek na sekundę (FPS) odczytywano bezpośrednio z wbudowanych statystyk silnika Unity. Centralna część hali dystrybucyjnej była idealnym obszarem do przeprowadzenia testów, ponieważ obejmuje największą liczbę obiektów, które wpływają na zapotrzebowanie mocy obliczeniowej procesora i karty graficznej.

Wyniki testów dla poszczególnych profili graficznych:

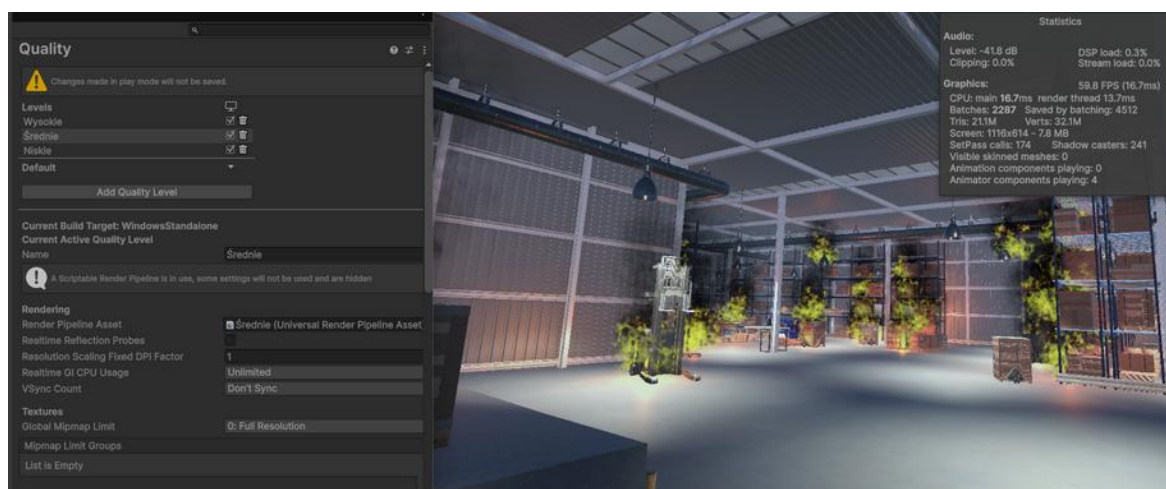
- Ustawienia wysokie – w tym trybie odnotowano średnią wydajność na poziomie 44,9 FPS. Uzyskanie takiej wartości FPS jest w pełni wystarczające do tego, aby symulator zachowywał stabilność działania i poprawne funkcjonowanie wszystkich systemów.
- Ustawienia średnie – w tym trybie uzyskano średnią wydajność na poziomie 59,8 FPS, co spowodowało zmniejszenie wysiłku komputera do uzyskania płynności przy zachowaniu zadowalającej jakości obrazu.

- Ustawienia niskie – dla tego profilu osiągnięto średni odczyt wydajności 72,5 FPS, zapewniając bardzo wysoką płynność obrazu, kosztem jakości wizualnej sceny.

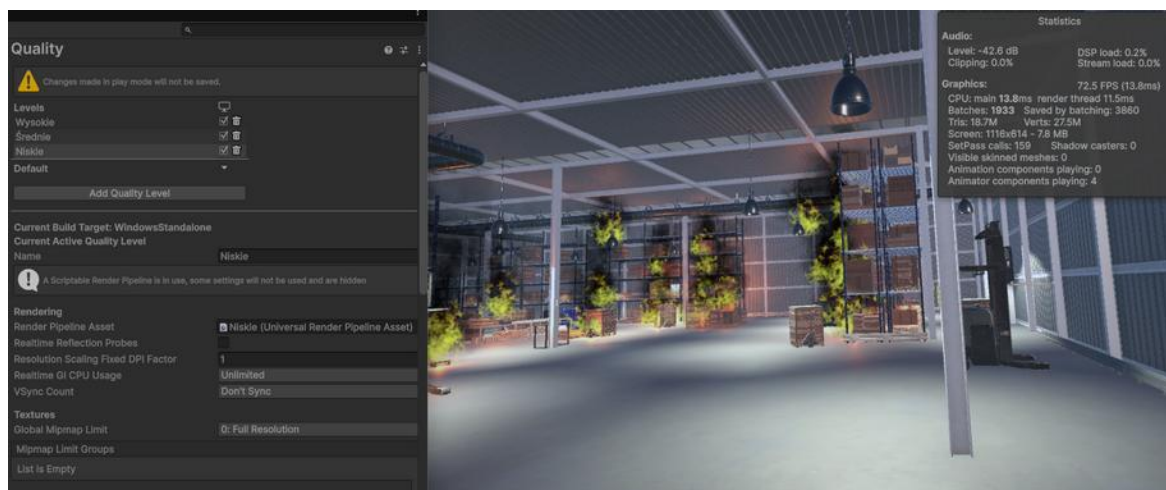
Po przeprowadzonych testach funkcjonalnych uzyskane wyniki demonstrują, że symulator poprawnie reaguje na zmianę poziomu jakości graficznej. Odnotowane wyniki dowodzą dobrego zoptymalizowania sceny hali dystrybucyjnej, zważając na dużą liczbę jednocześnie renderowanych obiektów, aktywnych efektów VFX ognia, źródeł światła oraz rozbudowanej geometrii hali. Symulacja przy najwyższych ustawieniach graficznych utrzymuje płynność umożliwiającą komfortowe korzystanie w środowisku VR, na wskazanej wyżej konfiguracji sprzętowej. Natomiast tryby średni i niski pozwalają na dalsze zwiększenie wydajności kosztem ograniczenia jakości.



Rysunek 6.63. Uzyskane FPS na ustawieniach graficznych wysokich [Źródło własne].



Rysunek 6.64. Uzyskane FPS na ustawieniach graficznych średnich [Źródło własne].



Rysunek 6.65. Uzyskane FPS na ustawieniach graficznych niskich [Źródło własne].

6.2. Opis odczuć użytkowników po testach symulatora

Przed rozpoczęciem testów użytkownicy zostali zapoznani wyłącznie z podstawową obsługą kontrolerów VR oraz zasadami bezpiecznego korzystania z gogli wirtualnej rzeczywistości. Nie przekazywano im szczegółowych informacji dotyczących przebiegu symulacji, ani sposobu realizacji poszczególnych zadań. Cały proces testowy odbywał się w ciszy, bez dodatkowych podpowiedzi. Miało to na celu sprawdzenie intuicyjności interfejsu oraz czytelności informacji przekazywanych bezpośrednio przez samą aplikację. Do badań dobrano osoby o zróżnicowanym poziomie doświadczenia zarówno w zakresie obsługi sprzętu VR, jak i wiedzy dotyczącej postępowania podczas gaszenia pożarów. Pozwoliło to na uzyskanie bardziej zróżnicowanych opinii oraz obserwację różnic w sposobie interakcji z symulatorem.

Podczas testów zauważono odmienne strategie zachowania użytkowników. Część uczestników przed rozpoczęciem symulacji dokładnie zapoznawała się z dostępnymi informacjami instruktażowymi. Inni decydowali się na przerwanie symulacji w jej trakcie, aby wrócić do materiałów informacyjnych i lepiej zrozumieć działanie poszczególnych mechanizmów. Kolejna grupa użytkowników przechodziła scenariusz metodą prób i błędów, ucząc się obsługi systemów w trakcie symulacji. W przypadku tej grupy odnotowano wyższe oceny poziomu trudności symulatora w późniejszych pytaniach.

W ramach testów symulator został również udostępniony czynnemu strażakowi Ochotniczej Straży Pożarnej w Osinach, co pozwoliło na uzyskanie opinii osoby posiadającej praktyczne doświadczenie w działaniach gaśniczych. Jego obserwacje i odczucia stanowiły cenne odniesienie do oceny realizmu oraz przydatności symulatora jako narzędzia szkoleniowego.

Pytania, które zostały przedstawione użytkownikom to:

1. Ogólna ocena symulatora w skali od 0 do 10.
2. Ocena realizmu przedstawionego scenariusza od 0 do 10.
3. Ocena realizmu symulacji od 0 do 10.
4. Ocena trudności symulatora od 0 do 10.
5. Ocena dostosowania symulatora pod preferencje użytkownika (zakres ustawień) od 0 do 10.
6. Jakie emocje towarzyszyły Ci podczas odgrywanego scenariusza w symulacji?

Tabela 6.1. Odpowiedzi użytkowników do pytań od 1 do 5.

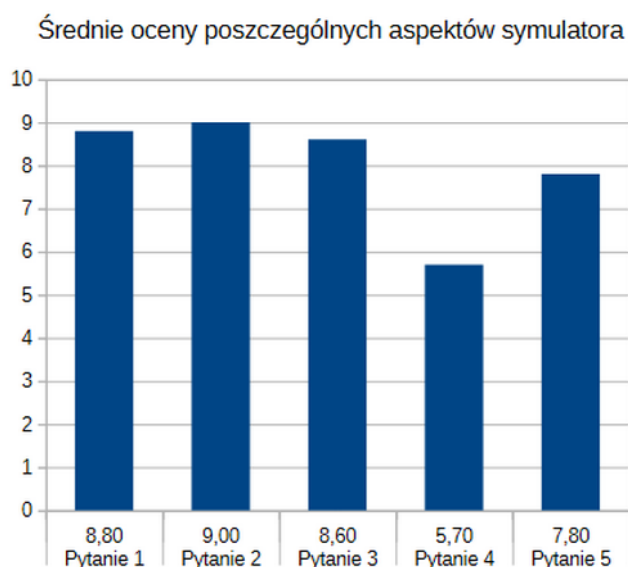
Nazwa Użytkownika	Pytanie 1	Pytanie 2	Pytanie 3	Pytanie 4	Pytanie 5
Użytkownik (Strażak)	9	9	8	6	10
Użytkownik (2)	9	8	8	6	10
Użytkownik (3)	8	7	7	6	8
Użytkownik (4)	9	10	9	8	9
Użytkownik (5)	10	10	10	8	5
Użytkownik (6)	10	10	10	6	8
Użytkownik (7)	8	9	8	5	7
Użytkownik (8)	8	8	8	3	8
Użytkownik (9)	9	9	9	4	3
Użytkownik (10)	8	10	9	5	10

Tabela 6.2. Odpowiedzi użytkowników do pytania 6.

Nazwa Użytkownika	Pytanie 6
Użytkownik (Strażak)	„Scenariusz w odgrywanej symulacji był bardzo wciągający. Grając w okularach VR wszystko wydawało się jakby to było w prawdziwym życiu. Na początku było lekkie napięcie ze względu na nowe otoczenie, nie wiedziałem jaki będzie scenariusz i można było się spodziewać wszystkiego. Pojawiło się również skupienie, żeby zrobić wszystko dobrze i nie popełnić błędu, co by mogło skutkować przegraną. Po opanowaniu sytuacji pojawiła się ulga, że wszystko zmierza w dobrym kierunku i wygranej. Po ukończeniu scenariusza pojawiła się satysfakcja, że udało się wygrać oraz przemyślenia co można było zrobić lepiej lub co się zrobiło źle.”
Użytkownik (2)	„Było to ciekawe doświadczenie, należało wcielić się w rolę strażaka i przemyśleć jakie czynności muszą wykonać takie służby, by ugasić pożar. Przy pierwszej rozgrywce nie wszystko było tak proste do przewidzenia, co sprawiło, że łatwo było przegrać. Gaszenie pożaru było również świetną zabawą.”
Użytkownik (3)	„Strach przed kończącym się powietrzem oraz presja czasu.”
Użytkownik (4)	„Podczas zbliżania się zbyt blisko ognia doświadczyłam uczucia, że mogę się zaraz poparzyć. Innymi uczuciami jakie mi towarzyszyły to: strach, napięcie, niepewność, satysfakcja.”
Użytkownik (5)	„Nowe doświadczenia dzięki obsłudze padów i okularów VR. Chęć wykonania zadania, zaskoczenie, jeśli chodzi o stopień realistyczności rozgrywki, zrozumiałe i przejrzyste sposoby wykonywania ruchów i czynności.”
Użytkownik (6)	„Nowe doświadczenie, chęć wykonania zadania, zrozumienia wszystkich funkcji, starałam się skupić na otoczeniu, co się, gdzie znajduje, tak żeby ukończyć zadanie, towarzyszyła mi ciekawość, bo wszystko było dla mnie nowe, zaangażowanie (chęć wykonania zadania), radość (że mogłam spróbować czegoś nowego, była to forma gry i zabawy), frustracja.”

Użytkownik (7)	„Symulacja była angażująca i wymagała skupienia. Po czasie odczułem presję czasu, irytację oraz satysfakcję z gaszonego pożaru. Całość była zupełnie nowym doświadczeniem.”
Użytkownik (8)	„Gra ciekawa, realistyczna, poruszanie się bardzo intuicyjne. W grę bardzo się wczułem, po kilku minutach poczułem napływ gorąca.”
Użytkownik (9)	„Fajna zabawa, ciekawie było się wcielić w strażaka. Satysfakcjonujące uczucie ugaszenia pożaru od razu za pierwszym razem.”
Użytkownik (10)	„Dobrze się bawiłam podczas gry, ale wystraszyłam się dźwięku, kiedy zaczęło mi się kończyć powietrze w butli.”

Analiza wyników z odpowiedzi użytkowników z wykorzystaniem wykresu słupkowego.



Rysunek 6.66. Średnie z uzyskanych odpowiedzi na pytania [Źródło własne].

Najwyżej oceniony został realizm przedstawionego scenariusza (średnia 9,0), co potwierdza skuteczność zastosowanych rozwiązań wizualnych i dźwiękowych. Wysokie oceny uzyskały również ogólna ocena symulatora (8,8) oraz realizm samej symulacji (8,6).

Najniższą średnią ocenę uzyskał poziom trudności symulatora (5,7), co wynikało z różnych strategii przyjętych przez użytkowników. Ocena dostosowania symulatora do preferencji użytkownika wyniosła 7,8, co wskazuje na wystarczającą ilość ustawień.

Odpowiedzi opisowe potwierdziły poziom immersji symulatora. Użytkownicy najczęściej wskazywali emocje takie jak napięcie, stres, skupienie, ciekawość oraz

satysfakcję po ukończeniu scenariusza. Uzyskane wyniki potwierdzają, że symulator został odebrany pozytywnie i spełnia swoje zadanie, otrzymując ogólną średnią ocenę ze wszystkich pytań równą 7.98.

6.3. Możliwości rozwoju

Przeprowadzona implementacja symulatora VR stanowi dobrą podstawę do dalszego rozwoju aplikacji pod względem technologicznym. Jednym z kierunków rozbudowy systemu jest możliwość zastosowania nowszych i bardziej zaawansowanych efektów wizualnych ognia. Aktualnie wykorzystany model VFX spełnia swoje zadanie dydaktyczne, jednak w przyszłości możliwe jest zastąpienie go nowszym systemem, co pozwoliłoby na jeszcze większą ilość ognia na scenach.

Kolejnym istotnym obszarem rozwoju jest rozszerzenie liczby dostępnych scenariuszy szkoleniowych. Obecna symulacja skupia się na gaszeniu pożaru w hali dystrybucyjnej, jednak system może zostać łatwo zaadaptowany do innych typów zdarzeń.

Dodatkową możliwością rozwoju jest integracja symulatora z bieżnią VR, umożliwiającą fizyczne poruszanie się użytkownika w przestrzeni wirtualnej. Zastosowanie tego typu urządzeń pozwoliłoby na bardziej realistyczne odwzorowanie przemieszczania się strażaka podczas akcji.

7. PODSUMOWANIE

Celem niniejszej pracy było zaprojektowanie oraz realizacja szkoleniowej gry symulacyjnej w wirtualnej rzeczywistości, przeznaczonej do wspomagania procesu szkolenia początkujących strażaków. Założeniem projektu było stworzenie immersyjnego środowiska VR, które w bezpieczny i kontrolowany sposób umożliwia użytkownikowi zapoznanie się z podstawowymi elementami działań gaśniczych.

W ramach pracy przeanalizowano podstawy technologii wirtualnej rzeczywistości oraz możliwości i ograniczenia gogli Meta Quest 2 w zastosowaniach szkoleniowych. Część projektowa i implementacyjna pracy objęła opracowanie scenariusza pożaru w hali dystrybucyjnej, zaprojektowanie interakcji użytkownika oraz implementację kluczowych systemów symulacyjnych, takich jak: system autopompy, system ognia, system prądu, system powietrza oraz mechanizmy odpowiedzialne za wygraną i przegraną w symulacji.

Ważnym elementem projektu było uwzględnienie realizmu procedur oraz konsekwencji błędnych decyzji użytkownika. Zaimplementowane mechanizmy, takie jak: ograniczona ilość powietrza w aparacie oddechowym, zagrożenie porażeniem prądem, konieczność zachowania odpowiedniej kolejności działań czy możliwość ponownego

rozpalenia się ognia, pozwalają użytkownikowi lepiej zrozumieć specyfikę pracy strażaka. Jednocześnie symulator umożliwia wielokrotne powtarzanie scenariusza, co stanowi jedną z zalet wykorzystania technologii VR w szkoleniach.

Przeprowadzone testy funkcjonalne wykazały, że aplikacja poprawnie reaguje na zmianę jakości graficznej, co dodatkowo zostało poparte wskazaniem FPS przy poszczególnych ustawieniach. Testy z udziałem użytkowników o zróżnicowanym doświadczeniu w zakresie VR oraz działań strażackich pozwoliły ocenić czytelność interfejsu, stopień trudności symulatora oraz poziom realizmu symulacji. Uzyskane opinie potwierdziły, że aplikacja spełnia swoją funkcję szkoleniową.

Na podstawie zrealizowanych założeń, wykonanej implementacji oraz wyników testów można stwierdzić, że główny cel pracy został osiągnięty. Opracowany symulator stanowi funkcjonalne i spójne narzędzie szkoleniowe. Praca potwierdza, że technologia wirtualnej rzeczywistości posiada duży potencjał w obszarze szkoleń specjalistycznych.

LITERATURA

[1] Audacity 3.7 Reference Manual.

<https://manual.audacityteam.org/>

[online, dostęp 29.12.2025]

[2] Piotr Chlipalski., Blender. Architektura i projektowanie. Wydanie II, Helion Gliwice 2018.

[3] Harrison Ferrone., Learning C# by Developing Games with Unity, Helion Gliwice 2021.

[4] Ignaciuk, M. (2022). Wirtualna rzeczywistość w edukacji – analiza opinii nauczycieli na temat wykorzystania technologii VR w edukacji szkolnej. Ars Educandi, 19(19). <https://doi.org/10.26881/ae.2022.19.05> [online, dostęp 21.11.2025]

[5] Kęsy, M. (2016). Symulatory wirtualnej rzeczywistości. Dydaktyka Informatyki, 11, 125 – 131. <https://doi.org/10.15584/di.2016.11.16> [online, dostęp 21.11.2025]

[6] Jonathan Linowes, Unity Virtual Reality Projects, Packt Publishing, 2020

[7] Wykorzystanie wirtualnej rzeczywistości w szkoleniu strażaków: szanse i zagrożenia Artur Luzar, Bogdan Kosowski

https://biblioteka.apoz.edu.pl/pl/search_results/2267947?q%5Bsort_attributes%5D%5Bfield_name%5D=title.sort&q%5Bsort_attributes%5D%5Border%5D=desc

[online, dostęp 11.01.2026]

[8] Unity 6.0 User Manual

<https://docs.unity3d.com/6000.0/Documentation/Manual/UnityManual.html>

[online, dostęp 15.08.2025]

[9] Artykuł „VR w straży pożarnej – korzyści, wyzwania i najlepsze praktyki wdrożenia szkoleń wirtualnych”

<https://fire-flames.com/szkolenia-vr-dla-strazy-pozarnej-korzysci/>

[online, dostęp 11.01.2026]

[10] UML tools - Generator scenariuszy przypadków użycia

<https://kronos.tu.kielce.pl/uml/scenarios.html>

[11] Model Strażaka

<https://sketchfab.com/3d-models/firefighter-964fa8b171df41dd8fe6b1305da24812>

[online, dostęp 17.08.2025]

[12] Model wozu strażackiego

<https://www.cgtrader.com/3d-models/vehicle/truck/low-poly-rusty-emergency-fire-truck-vehicle-textured>

[online, dostęp 26.11.2025]

[13] Model transformatora

<https://assetstore.unity.com/packages/3d/environments/industrial/transformer-box-194940>

[online, dostęp 06.01.2026]

[14] Model betonowego ogrodzenia

<https://assetstore.unity.com/packages/3d/props/modular-self-stand-fence-105862>

[online, dostęp 09.12.2025]

[15] Rekwizyty uliczne

<https://assetstore.unity.com/packages/3d/props/street-props-prototype-collection-291021>

[online, dostęp 09.12.2025]

[16] Modele dłoni

<https://assetstore.unity.com/packages/3d/characters/stylized-simple-hands-221297>

[online, dostęp 18.08.2025]

[17] Warehouse Environment

<https://www.fab.com/listings/ef0311b7-fd62-414a-b2c3-66ba95d8a21d>

[online, dostęp 08.11.2025]

[18] Unreal To Unity Exporter 2025

<https://www.fab.com/listings/6acd4c02-aaad-4169-99c4-df5082f64692>

[online, dostęp 08.11.2025]

[19] Model asteam

<https://asteam.ch/produkt/113/Atenschutz/Atenschutzger%C3%A4te/propak-uno>

[online, dostęp 19.12.2025]

[20] Free Fire VFX – URP

<https://assetstore.unity.com/packages/vfx/particles/fire-explosions/free-fire-vfx-urp-266226>

[online, dostęp 02.01.2026]

SPIS TABEL

Tabela 6.1. Odpowiedzi użytkowników do pytań od 1 do 5.

Tabela 6.2. Odpowiedzi użytkowników do pytania 6.

SPIS RYSUNKÓW

Rysunek 3.1. Soczewki gogli Meta Quest 2 [Źródło własne].

Rysunek 3.2. Kamery gogli Meta Quest 2 [Źródło własne].

Rysunek 3.3. Kontrolery gogli Meta Quest 2 [Źródło własne].

Rysunek 4.1. Scenariusz przypadku użycia: „Uruchomienie symulacji” [Opracowanie własne].

Rysunek 4.2. Scenariusz przypadku użycia: „Zmiana ustawień” [Opracowanie własne].

Rysunek 4.3. Scenariusz przypadku użycia: „Przegląd informacji” [Opracowanie własne].

Rysunek 4.4. Scenariusz przypadku użycia: „Symulacja w hali dystrybucyjnej – osiągnięcie wygranej” [Opracowanie własne].

Rysunek 4.5. Scenariusz przypadku użycia: „Symulacja w hali dystrybucyjnej – osiągnięcie przegranej” [Opracowanie własne].

Rysunek 5.1. Konfiguracja Build Profiles [Źródło własne].

Rysunek 5.2. XR Plug-in Management – OpenXR dla platformy PC [Źródło własne].

Rysunek 5.3. OpenXR – Interaction Profiles [Źródło własne].

Rysunek 5.4. Hierarchia XR Origin (VR) oraz widok inspektora [Źródło własne].

Rysunek 5.5. Widok inspektora obiektu Locomotion [Źródło własne].

Rysunek 5.6. Widok inspektora obiektu Move (poruszania) [Źródło własne].

Rysunek 5.7. Widok inspektora komponentu Continuous Turn Provider [Źródło własne].

Rysunek 5.8. Widok inspektora komponentu Snap Turn Provider [Źródło własne].

Rysunek 5.9. Widok inspektora komponentu Teleportation [Źródło własne].

Rysunek 5.10. Widok inspektora obiektu Teleport Ray [Źródło własne].

Rysunek 5.11. Widok inspektora obiektu, w którym przyczepiony jest punkt startowy promienia Right Controller Teleport Stabilized Origin [Źródło własne].

Rysunek 5.12. Czerwony wskaźnik teleportacji [Źródło własne].

Rysunek 5.13. Wskaźnik teleportacji Directional Teleport Reticle [Źródło własne].

Rysunek 5.14. Widok inspektora Left Hand, XR Direct Interactor oraz Sphere Collider [Źródło własne].

Rysunek 5.15. Widok inspektora Right Hand, XR Direct Interactor oraz Sphere Collider [Źródło własne].

Rysunek 5.16. Model strażaka [Źródło własne].

Rysunek 5.17. Jednak klatka przedstawiająca animacje modelu strażaka [Źródło własne].

Rysunek 5.18. Model wozu strażackiego [Źródło własne].

Rysunek 5.19. Model transformatorowni [Źródło własne].

Rysunek 5.20. Model betonowego ogrodzenia oraz bloku [Źródło własne].

Rysunek 5.21. Modele rekwizytów ulicznych [Źródło własne].

Rysunek 5.22. Modele dłoni [Źródło własne].

Rysunek 5.23. Widok hali z frontowego lewego narożnika sceny [Źródło własne].

Rysunek 5.24. Widok hali z perspektywy bramy wjazdowej [Źródło własne].

Rysunek 5.25. Widok hali z tylnego lewego narożnika sceny [Źródło własne].

Rysunek 5.26. Widok hali z tylnego prawego narożnika sceny [Źródło własne].

Rysunek 5.27. Widok z wnętrza hali nad głównym wejściem/wyjściem [Źródło własne].

Rysunek 5.28. Widok z wnętrza hali z górnego prawego rogu [Źródło własne].

Rysunek 5.29. Widok z wnętrza hali na infrastrukturę techniczną [Źródło własne].

Rysunek 5.30. Widok z wnętrza hali na główne wejście/wyjście [Źródło własne].

Rysunek 5.31. Wygląd panela Menu [Źródło własne].

Rysunek 5.32. Wygląd lewego i prawego panela [Źródło własne].

Rysunek 5.33. Wygląd otoczenia za użytkownikiem w menu [Źródło własne].

Rysunek 5.34. Wygląd najechania dłonią na UI Menu [Źródło własne].

Rysunek 5.35. Wygląd paska ładowania [Źródło własne].

Rysunek 5.36. Wygląd okna ustawień [Źródło własne].

Rysunek 5.37. Wygląd okna informacji wraz z wszystkimi informacjami [Źródło własne].

Rysunek 5.38. Drzwi boczne autopompy obiekt interaktywny (XR Grab Interactable) z fizyką (Rigidbody) oraz zawiasem (Hinge Joint) umożliwiającym otwieranie [Źródło własne].

Rysunek 5.39. Panel autopompy po otwarciu drzwi widok monitora z komunikatem stanu ciśnienia [Źródło własne].

Rysunek 5.40. Przycisk panelu ciśnienia XRSimpleInteractable z konfiguracją interakcji typu poke (XR Poke Filter) i elementami affordance [Źródło własne].

Rysunek 5.41. Canvas panelu ciśnienia w przestrzeni świata (World Space) z tekstem OFF/ON (TextMeshPro) [Źródło własne].

Rysunek 5.42. Końcówka węża (Hose_End) Rigidbody oraz XR Grab Interactable umożliwiające podniesienie i utrzymanie zaworu [Źródło własne].

Rysunek 5.43. Wąż strażacki oraz ustawienia WazStrazacki i WazStrazackiRuraRenderer [Źródło własne].

Rysunek 5.44. Konfiguracja Particle System (Water) oraz parametry emisji i kształtu strumienia [Źródło własne].

Rysunek 5.45. Konfiguracja Particle System (Splash) parametry rozbryzgu [Źródło własne].

Rysunek 5.46. Widok działania strumienia w scenie oraz inspektor komponentu StrumienWody z przypisanymi prefabami i parametrami zasięgu strumienia [Źródło własne].

Rysunek 5.47. Wygląd panelu na lewej dłoni oraz jego działanie zakładania aparatu [Źródło własne].

Rysunek 5.48. Konfiguracja Gaze Interactor [Źródło własne].

Rysunek 5.49. Konfiguracja stref zagrożenia [Źródło własne].

Rysunek 5.50. Konfiguracja dźwięku gwizdka [Źródło własne].

Rysunek 5.51. Konfiguracja lampy oraz przycisku [Źródło własne].

Rysunek 5.52. Wygląd włączonego/wyłączonego oświetlenia [Źródło własne].

Rysunek 5.53. Hierarchia punktów ognia [Źródło własne].

Rysunek 5.54. Ustawienia FireAudioManager [Źródło własne].

Rysunek 5.55. Punkty ognia rozmieszczone na scenie [Źródło własne].

Rysunek 5.56. Gaszenie punktu ognia [Źródło własne].

Rysunek 5.57. Widok sceny przegranej [Źródło własne].

Rysunek 5.58. Widok sceny wygranej [Źródło własne].

Rysunek 5.59. Widok profilów jakości (Projekt Settings) [Źródło własne].

Rysunek 5.60. Ustawienia graficzne wysokie [Źródło własne].

Rysunek 5.61. Ustawienia graficzne średnie [Źródło własne].

Rysunek 5.62. Ustawienia graficzne niskie [Źródło własne].

Rysunek 6.63. Uzyskane FPS na ustawieniach graficznych wysokich [Źródło własne].

Rysunek 6.64. Uzyskane FPS na ustawieniach graficznych średnich [Źródło własne].

Rysunek 6.65. Uzyskane FPS na ustawieniach graficznych niskich [Źródło własne].

Rysunek 6.66. Średnie z uzyskanych odpowiedzi na pytania [Źródło własne].